

SIEMENS

SINUMERIK

SINUMERIK ONE Synchronized actions

Function Manual

Introduction

1

Fundamental safety
instructions

2

Brief description

3

Detailed description

4

Examples

5

Appendix

A

Valid for

controller
SINUMERIK ONE

CNC software Version 6.23

01/2024

A5E48053843B AH

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury will result if proper precautions are not taken.
--

WARNING
--

indicates that death or severe personal injury may result if proper precautions are not taken.

CAUTION
--

indicates that minor personal injury can result if proper precautions are not taken.
--

NOTICE

indicates that property damage can result if proper precautions are not taken.
--

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

WARNING
--

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.
--

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Table of contents

1	Introduction	7
1.1	About SINUMERIK	7
1.2	About this documentation	7
1.3	Documentation on the internet	10
1.3.1	Documentation overview SINUMERIK ONE	10
1.3.2	Documentation overview SINUMERIK operator components	10
1.4	Feedback on the technical documentation	11
1.5	mySupport documentation	11
1.6	Service and Support.....	12
1.7	Using OpenSSL	13
1.8	Compliance with the General Data Protection Regulation	13
2	Fundamental safety instructions.....	15
2.1	General safety instructions.....	15
2.2	Warranty and liability for application examples	15
2.3	Cybersecurity information	15
3	Brief description	17
4	Detailed description.....	19
4.1	Definition of a synchronized action.....	19
4.2	Components of synchronized actions	20
4.2.1	Validity, identification number (ID, IDS)	20
4.2.2	Frequency (WHENEVER, FROM, WHEN, EVERY)	21
4.2.3	G command (condition)	22
4.2.4	Condition	22
4.2.5	G command (action)	23
4.2.6	Actions with condition fulfilled (DO).....	24
4.2.7	Actions with condition unfulfilled (ELSE)	24
4.3	System variables for synchronized actions	25
4.3.1	Reading and writing.....	26
4.3.2	Operators and arithmetic functions	26
4.3.3	Type conversions	29
4.3.4	Marker/counter (\$AC_MARKER)	30
4.3.5	Parameters (\$AC_PARAM)	31
4.3.6	R parameters (\$R)	32
4.3.7	Machine and setting data (\$\$M, \$\$S).....	33
4.3.8	Timer (\$AC_TIMER)	34
4.3.9	FIFO variables (\$AC_FIFO)	35
4.3.10	Path tangent angle (\$AC_TANEB)	40
4.3.11	Override (\$A...OVR)	40

4.3.12	Capacity evaluation (\$AN_IPO ... , \$AN/AC_SYNC ... , \$AN_SERVO).....	42
4.3.13	Working-area limitation (\$SA_WORKAREA_...).....	45
4.3.14	SW cam positions and times (\$\$SN_SW_CAM_...)	45
4.3.15	Path length evaluation / machine maintenance (\$AA_TRAVEL ... , \$AA_JERK ...)	46
4.3.16	Polynomial coefficients, parameters (\$AC_FCT ...).....	47
4.3.17	Overlaid movements (\$AA_OFF)	49
4.3.18	Online tool length compensation (\$AA_TOFF)	52
4.3.19	Current block in the interpolator (\$AC_BLOCKTYPE, \$AC_BLOCKTYPEINFO, \$AC_SPLITBLOCK)	56
4.3.20	Initialization of array variables (SET, REP)	58
4.3.21	Grinding-specific system variables (\$AC_IN_KEY_G...)	58
4.3.22	Status Synchronized action disabled (\$AC_SYNA_STATE).....	62
4.4	User-defined variables for synchronized actions	63
4.5	Language elements for synchronized actions and technology cycles	64
4.6	Language elements for technology cycles only	70
4.7	Actions in synchronized actions.....	70
4.7.1	Output of M, S and H auxiliary functions to the PLC	70
4.7.2	Reading and writing of system variables	71
4.7.3	Polynomial evaluation (SYNFCT).....	72
4.7.4	Online tool offset (FTOC).....	77
4.7.5	Programmed read-in disable (RDISABLE)	79
4.7.6	Cancel preprocessing stop (STOPREOF).....	80
4.7.7	Delete distance-to-go (DELDTG).....	80
4.7.8	Traversing axes, to position (POS).....	82
4.7.9	Setting the measuring system (G70, G71, G700, G710)	86
4.7.10	Position in specified reference range (POSRANGE).....	87
4.7.11	Traversing axes, endless (MOV)	88
4.7.12	Axial feedrate (FA)	89
4.7.13	Axis replacement (GET, RELEASE, AXTOCHAN)	90
4.7.14	Traversing spindles (M, S, SPOS).....	95
4.7.15	Actual value setting with loss of the referencing status (PRESETON)	97
4.7.16	Actual value setting without loss of the referencing status (PRESETONS)	103
4.7.17	Couplings (CP..., LEAD..., TRAIL..., CTAB...)	109
4.7.18	Measurement (MEAWA, MEAC)	113
4.7.19	Travel to fixed stop (FXS, FXST, FXSW, FOCON, FOCOF, FOC)	116
4.7.20	Channel synchronization (SETM, CLEARM).....	118
4.7.21	User-specific error reactions (SETAL)	119
4.7.22	Cancel the actual subprogram level (CANCELSUB)	119
4.8	Technology cycles	120
4.8.1	General	120
4.8.2	Processing mode (ICYCON, ICYCOF)	122
4.8.3	Definitions (DEF, DEFINE)	123
4.8.4	Parameter transfer	124
4.8.5	Context variable (\$P_TECCYCLE).....	124
4.9	Protected synchronized actions	125
4.10	Coordination via part program and synchronized action (LOCK, UNLOCK, CANCEL)	126
4.11	Coordination via PLC	127
4.12	Configuration.....	128

4.13	Control behavior in specific operating states	129
4.13.1	Power On.....	129
4.13.2	NC reset.....	130
4.13.3	NC stop.....	130
4.13.4	Operating mode change	131
4.13.5	End of program.....	132
4.13.6	Block search.....	132
4.13.7	Program interruption by ASUB.....	133
4.13.8	REPOS.....	133
4.13.9	Response to alarms	134
4.14	Diagnostics	134
4.14.1	Displaying the status of synchronized actions	134
5	Examples.....	135
5.1	Examples of conditions in synchronized actions.....	135
5.2	Reading and writing of SD/MD from synchronized actions.....	135
5.3	Examples of adaptive control	137
5.3.1	Clearance control with variable upper limit.....	137
5.3.2	Feedrate control	138
5.3.3	Control velocity as a function of normalized path	139
5.4	Monitoring a safety clearance between two axes	140
5.5	Store execution times in R parameters	141
5.6	"Centering" with continuous measurement.....	142
5.7	Axis couplings via synchronized actions.....	144
5.7.1	Coupling to leading axis.....	144
5.7.2	Non-circular grinding via master value coupling	146
5.7.3	On-the-fly parting	149
5.8	Technology cycles position spindle	150
5.9	Synchronized actions in the TC/MC area.....	152
A	Appendix.....	157
A.1	List of abbreviations.....	157
	Index.....	163

Introduction

1.1 About SINUMERIK

From simple, standardized CNC machines to premium modular machine designs – the SINUMERIK CNCs offer the right solution for all machine concepts. Whether for individual parts or mass production, simple or complex workpieces – SINUMERIK is the highly dynamic automation solution, integrated for all areas of production. From prototype construction and tool design to mold making, all the way to large-scale series production.

Visit our website for more information SINUMERIK (<https://www.siemens.com/sinumerik>).

1.2 About this documentation

This documentation is part of the documentation category of SINUMERIK Function Manuals.

SINUMERIK Function Manuals

The SINUMERIK Function Manuals describe the NC functions of a SINUMERIK control.

The target group are configuring engineers, technologists, commissioning engineers and programmers.

Each Function Manual covers a specific topic area and contains all the function descriptions for this topic area.

The following table shows which Function Manuals are available for your SINUMERIK control and which topic areas a Function Manual covers:

Function Manual	Topic area
Basic functions	Basic functions of a CNC control
Axes and spindles	Axis and spindle functions, axis couplings
Transformations	Transformation functions
Monitoring and compensating	Axis monitoring and compensation functions, collision avoidance
Tools	Functions for selection, correction and monitoring of tools
Synchronized actions	Functionality of synchronized actions
PLC	Structure and functions of the PLC
Tool management	Function, commissioning and programming of tool management
Technologies	Extended technological functionalities

Overview of contents

You will find an overview of the main chapters of the descriptions of functions that are contained in a Function Manual on the title page.

Validity

The title page also contains all data about the validity of a document, i.e. for which SINUMERIK control and for which software version this edition of the Function Manual is valid.

System data

In the descriptions of functions, the system data relevant for a function (machine data, setting data, system variables, interface signals and alarms) are only described with the level of detail that is strictly necessary to understand the function. You will find detailed information on this data in the relevant List Manuals and for alarms in the Diagnostics Manual.

Basic Program Plus and Basic Program

In the TIA Portal, two different types of basic program are available for programming the PLC functionality:

- Basic Program Plus works only with symbolic addressing. The functionality is implemented via data types, PLC blocks, functions and instructions. Basic Program Plus is available only for SINUMERIK ONE.
- Basic Program works with both symbolic and absolute addressing. The programs are thus largely compatible with already existing programs from the SIMATIC Manager. Basic Program is available for ONE and MC controllers.

Basic Program Plus and Basic Program cannot be used together.

PLC blocks and data types

The descriptions of functions often refer to the PLC signals and functions. In the text, they are primarily the blocks and data types of Basic Program Plus, which is only available for SINUMERIK ONE.

You can find detailed information in the PLC Function Manual for the relevant controller.

Interface signals

The symbolic addressing of Basic Program Plus is the first choice for addressing interface signals. To derive signals for Basic Program, there are tables at the end of the relevant section with the PLC signals used for the different signal directions, as shown in the following example:

NC → PLC

Basic Program Plus	Basic Program	
<Axis>.basic.in.enc1Synchronized	LBP_Axis*.E_RefSyn1	DB31,DBX60.4
...		

PLC → NC

Basic Program Plus	Basic Program	
<Axis>.basic.out.refPointApproachCam	LBP_Axis*.A_DelayRef	DB31,DBX12.7
...		

Note**Quantity structure**

In Basic Program Plus, axes, channels etc. are represented via instances of the corresponding objects.

The absolute addressing in Basic Program is based on the absolute maximum number of the following components: Mode groups (DB11), channels (DB21, ...), axes/spindles (DB31, ...)

Standard scope

This documentation only describes the functionality of the standard version. This may differ from the scope of the functionality of the system that is actually supplied. Please refer to the ordering documentation only for the functionality of the supplied drive system.

It may be possible to execute other functions in the system which are not described in this documentation. This does not, however, represent an obligation to supply such functions with a new control or when servicing.

For reasons of clarity, this documentation cannot include all of the detailed information on all product types. Further, this documentation cannot take into consideration every conceivable type of installation, operation and service/maintenance.

The machine manufacturer must document any additions or modifications they make to the product themselves.

Websites of third-party companies

This document may contain hyperlinks to third-party websites. Siemens is not responsible for and shall not be liable for these websites and their content. Siemens has no control over the information which appears on these websites and is not responsible for the content and information provided there. The user bears the risk for their use.

1.3 Documentation on the internet

1.3.1 Documentation overview SINUMERIK ONE

Comprehensive documentation about the functions provided in SINUMERIK ONE Version 6.13 and higher is provided in the Documentation overview SINUMERIK ONE (<https://support.industry.siemens.com/cs/ww/en/view/109768483>).



You can display documents or download them in PDF and HTML5 format.

The documentation is divided into the following categories:

- User: Operating
- User: Programming
- Manufacturer/Service: Functions
- Manufacturer/Service: Hardware
- Manufacturer/Service: Configuration/Setup
- Manufacturer/Service: Safety Integrated
- Information and training
- Manufacturer/Service: SINAMICS

1.3.2 Documentation overview SINUMERIK operator components

Comprehensive documentation about the SINUMERIK operator components is provided in the Documentation overview SINUMERIK operator components (<https://support.industry.siemens.com/cs/document/109783841/technische-dokumentation-zu-sinumerik-bedienkomponenten?dti=0&lc=en-WW>).

You can display documents or download them in PDF and HTML5 format.

The documentation is divided into the following categories:

- Operator Panels
- Machine control panels
- Machine Pushbutton Panel
- Handheld Unit/Mini handheld devices
- Further operator components

An overview of the most important documents, entries and links to SINUMERIK is provided at SINUMERIK Overview - Topic

Page (<https://support.industry.siemens.com/cs/document/109766201/sinumerik-an-overview-of-the-most-important-documents-and-links?dti=0&lc=en-WW>).

1.4 Feedback on the technical documentation

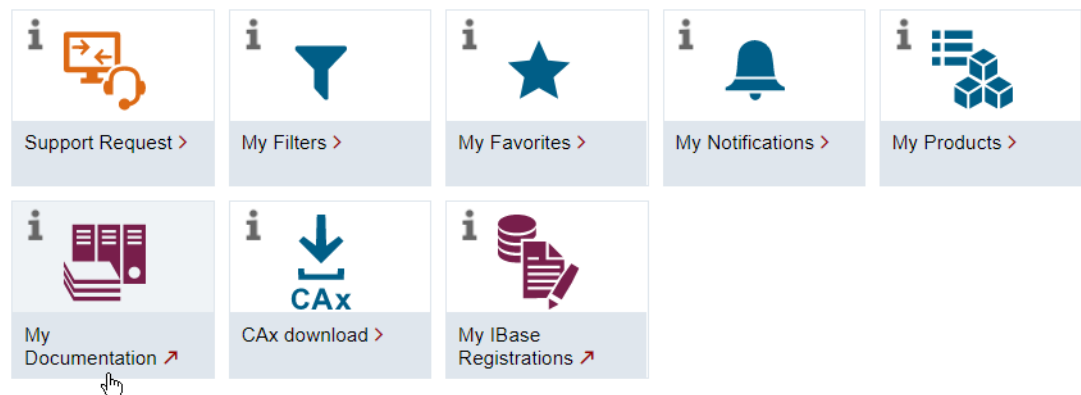
If you have any questions, suggestions or corrections regarding the technical documentation which is published in the Siemens Industry Online Support, use the link "Send feedback" link which appears at the end of the entry.

1.5 mySupport documentation

With the "mySupport documentation" web-based system you can compile your own individual documentation based on Siemens content, and adapt it for your own machine documentation.

To start the application, click on the "My Documentation" tile on the "mySupport links and tools" (<https://support.industry.siemens.com/cs/ww/en/my>) portal page:

mySupport Links and Tools



The configured manual can be exported in RTF, PDF or XML format.

Note

Siemens content that supports the mySupport documentation application can be identified by the presence of the "Configure" link.

1.6 Service and Support

Product support

You can find more information about products on the internet:

Product support (<https://support.industry.siemens.com/cs/ww/en/>)

The following is provided at this address:

- Up-to-date product information (product announcements)
- FAQs (frequently asked questions)
- Manuals
- Downloads
- Newsletters with the latest information about your products
- Global forum for information and best practice sharing between users and specialists
- Local contact persons via our Contacts at Siemens database (→ "Contact")
- Information about field services, repairs, spare parts, and much more (→ "Field Service")

Technical support

Country-specific telephone numbers for technical support are provided on the internet at address (<https://support.industry.siemens.com/cs/ww/en/sc/4868>) in the "Contact" area.

If you have any technical questions, please use the online form in the "Support Request" area.

Training

You can find information on SITRAIN at the following address (<https://www.siemens.com/sitrain>).

SITRAIN offers training courses for automation and drives products, systems and solutions from Siemens.

Siemens support on the go



With the award-winning "Industry Online Support" app, you can access more than 300,000 documents for Siemens Industry products – any time and from anywhere. The app can support you in areas including:

- Resolving problems when implementing a project
- Troubleshooting when faults develop
- Expanding a system or planning a new system

Furthermore, you have access to the Technical Forum and other articles from our experts:

- FAQs
- Application examples
- Manuals
- Certificates
- Product announcements and much more

The "Industry Online Support" app is available for Apple iOS and Android.

1.7 Using OpenSSL

This product can contain the following software:

- Software developed by the OpenSSL project for use in the OpenSSL toolkit
- Cryptographic software created by Eric Young.
- Software developed by Eric Young

You can find more information on the internet:

- OpenSSL (<https://www.openssl.org>)
- Cryptsoft (<https://www.cryptsoft.com>)

1.8 Compliance with the General Data Protection Regulation

Siemens observes standard data protection principles, in particular the data minimization rules (privacy by design).

1.8 Compliance with the General Data Protection Regulation

For this product, this means:

The product does not process or store any personal data, only technical function data (e.g. time stamps). If the user links this data with other data (e.g. shift plans) or if he/she stores person-related data on the same data medium (e.g. hard disk), thus personalizing this data, he/she must ensure compliance with the applicable data protection stipulations.

Fundamental safety instructions

2.1 General safety instructions

 WARNING
Danger to life if the safety instructions and residual risks are not observed If the safety instructions and residual risks in the associated hardware documentation are not observed, accidents involving severe injuries or death can occur. • Observe the safety instructions given in the hardware documentation. • Consider the residual risks for the risk evaluation.

 WARNING
Malfunctions of the machine as a result of incorrect or changed parameter settings As a result of incorrect or changed parameterization, machines can malfunction, which in turn can lead to injuries or death. • Protect the parameterization against unauthorized access. • Handle possible malfunctions by taking suitable measures, e.g. emergency stop or emergency off.

2.2 Warranty and liability for application examples

Application examples are not binding and do not claim to be complete regarding configuration, equipment or any eventuality which may arise. Application examples do not represent specific customer solutions, but are only intended to provide support for typical tasks.

As the user you yourself are responsible for ensuring that the products described are operated correctly. Application examples do not relieve you of your responsibility for safe handling when using, installing, operating and maintaining the equipment.

2.3 Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions constitute one element of such a concept.

2.3 Cybersecurity information

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit
<https://www.siemens.com/cybersecurity-industry>.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under
<https://new.siemens.com/cert>.

Further information is provided on the Internet:

Industrial Security Configuration Manual (<https://support.industry.siemens.com/cs/ww/en/view/108862708>)



WARNING

Unsafe operating states resulting from software manipulation

Software manipulations, e.g. viruses, Trojans, or worms, can cause unsafe operating states in your system that may lead to death, serious injury, and property damage.

- Keep the software up to date.
- Incorporate the automation and drive components into a state-of-the-art, integrated industrial cybersecurity concept for the installation or machine.
- Make sure that you include all installed products in the integrated industrial cybersecurity concept.
- Protect files stored on exchangeable storage media from malicious software by with suitable protection measures, e.g. virus scanners.
- Carefully check all cybersecurity-related settings once commissioning has been completed.

Brief description

General

A synchronized action consists of a series of related statements within a part program that is called cyclically in the interpolator clock cycle synchronously to the machining blocks.

A synchronized action is essentially divided into two parts, the optional condition and the obligatory action part. The time at which the actions are executed can be made dependent on a specific system state using the condition part. The conditions are evaluated cyclically in the interpolator clock cycle. The actions are then a reaction to user-definable system states. Their execution is not bound to block limits.

Furthermore, the validity of the synchronized action (non-modal, modal or static) and the frequency of the execution of the actions (once, repeatedly) can be defined.

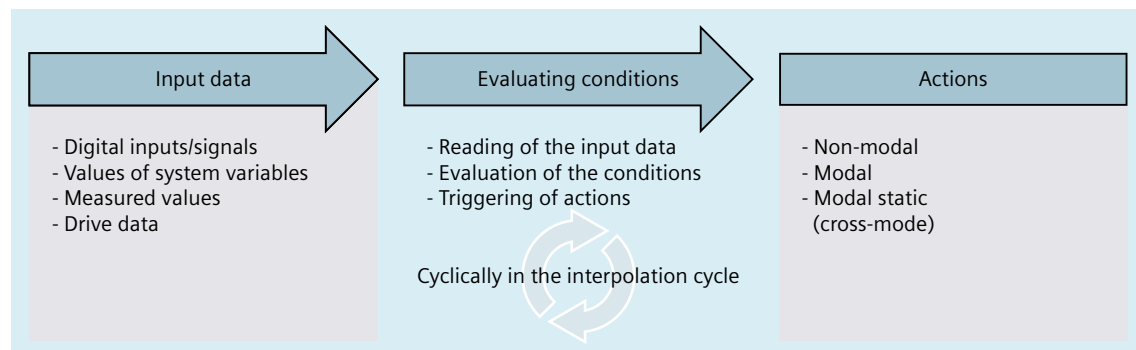
Examples of permissible actions

- Output of auxiliary functions to PLC
- Writing and reading of main run variables
- Traversing of positioning axes
- Activation of synchronous procedures, such as:
 - Read-in disable
 - Delete distance-to-go
 - End preprocessing stop
- Activation of technology cycles
- Calculation of function values
- Tool offsets
- Activating/deactivating couplings
- Measuring
- Enabling/disabling of synchronized actions

Examples of non-permissible actions

- Traversing of path axes

Schematic diagram of synchronous action



Detailed description

4.1 Definition of a synchronized action

A synchronized action is defined in a block of a part program. Any further commands that are not part of the synchronized action, must not be programmed within this block.

Components of a synchronized action

A synchronized action consists of the following components:

Validity, ID no. (Page 20) (optional)	Condition part (optional)			Action part with condition fulfilled			Action part with condition unfulfilled (optional)		
	Frequency (Page 21)	G command (Page 22) (optional)	Condition (Page 22)	Keyword	G command (Page 23) (optional)	Actions (Page 24)	Keyword	G command (Page 23) (optional)	Actions (Page 24)
--- ¹⁾ ID=<no.> IDS=<no.> >	--- ¹⁾ WHENEVER FROM WHEN EVERY	G...	Logical Expressio n	DO	G...	Action 1 ... Action n	ELSE	G...	Action 1 ... Action n

¹⁾ Not programmed

Syntax

Examples:

- DO <Action 1...n>
- <frequency> [<G function>] <condition> DO <action 1...n>
- ID=<no.> <frequency> [<G function>] <condition> DO <action 1...n>
- IDS=<no.> <frequency> [<G function>] <condition> DO <action 1...n>
- IDS=<no.> <frequency> [<G function>] <condition> DO <action 1...n>
ELSE <action 1...n>

4.2 Components of synchronized actions

4.2.1 Validity, identification number (ID, IDS)

Validity

The validity defines when and where the synchronized action will be processed:

Validity	Meaning
--- 1)	Non-modal synchronized action A non-modal synchronized action applies: - As long as the main run block following the synchronized action is active - Only in the AUTOMATIC mode Example: The synchronized action from N10 takes effect as long as N20 is active. <pre>N10 WHEN \$A_IN[1]==TRUE DO \$A_OUTA[1]=10 N20 G90 F1000 X100</pre>
ID=<ID number>	Modal synchronized action A modal synchronized action applies: - Until the part program has been completed - Only in the AUTOMATIC mode Value range: See the paragraph below "Identification number" > "Value range" Example: <pre>N20 ID=1 EVERY \$A_IN[1]==TRUE DO \$A_OUTA[1]=10</pre>
IDS=<ID number>	Static synchronized action A static synchronized action applies: In all operating modes for an unlimited period of time Value range: See the paragraph below "Identification number" > "Value range" Example: <pre>N30 IDS=1 EVERY \$A_IN[1]==TRUE DO \$A_OUTA[1]=10</pre>

1) Not programmed

Note

Static synchronized actions

Static synchronized actions (IDS) can be defined in an ASUB and activated at any time by activation of the ASUB via the PLC user program.

Identification number ID/IDS

Range of values

The identification numbers ID/IDS lie in various number ranges. The number ranges are assigned to different users.

ID/IDS	User	Directory
1 ... 999	"General area"	Any directory
1000 ... 1199	Machine manufacturer	/ _N_CMA_DIR
1200 ... 1399	Siemens	/ _N_CST_DIR

Parallelization

If several synchronized actions are to be active in parallel in a channel, then they must have different identification numbers ID/IDS. Synchronized actions with the same identification number replace each other within a channel.

Sequence of execution

Modal and static synchronized actions are executed in the order of their identification numbers ID/IDS.

Non-modal synchronized actions are executed after execution of the modal synchronized actions in the order of their programming.

Coordination via part programs and synchronized actions

Synchronized actions can be coordinated using part programs and synchronized actions (Page 126) based on the identification numbers ID/IDS.

Coordination via PLC

Synchronized actions with identification numbers ID/IDS in the range from 1 to 64 can be coordinated via the NC/PLC interface from the PLC user program (Page 127).

4.2.2 Frequency (WHENEVER, FROM, WHEN, EVERY)

The frequency specifies how often the condition is queried and, when the condition is fulfilled, how often the action should be executed. The frequency is part of the condition.

Frequency	Meaning
--- ¹⁾	If no frequency is specified, the action is executed cyclically in every interpolator clock cycle.
WHENEVER	If the condition is fulfilled, the action is executed cyclically in every interpolator clock cycle.
FROM	After the condition has been fulfilled once, the action is executed cyclically in every interpolator clock cycle for as long as the synchronized action is active.
WHEN	If the condition is fulfilled, the action is executed once and then the condition is no longer checked.
EVERY	In the following cases, the action is executed once: - The condition is already satisfied at the start of the synchronized action (state: TRUE) - At every state transition of the condition from FALSE to TRUE (rising edge)

¹⁾ Not programmed

See also

Technology cycles (Page 120)

4.2.3 G command (condition)

Defined initial state

With regard to the part program sequence, synchronized actions can be executed at any time depending on fulfillment of the condition. It is therefore recommended that the measuring system (inch or metric) be defined in a synchronized action **before** the condition and/or in the action part. This generates a defined initial position for the evaluation of the condition and the execution of the action, irrespective of the current part program state.

G commands

The following G commands are permissible:

- G70 (Inch dimensions for geometric specifications (lengths))
- G71 (Metric dimensions for geometric specifications (lengths))
- G700 (Inch dimensions for geometric and technological specifications (lengths, feedrate))
- G710 (Metric dimensions for geometric and technological specifications (lengths, feedrate))

Note

No other G commands are permitted in synchronized actions except G70, G71, G700 and G710.

Validity

A G command programmed in the condition part also applies for the action part even if no G command has been programmed in the action part itself.

A G command programmed in the action part only applies within the action part.

4.2.4 Condition

Execution of the action can be made dependent on the fulfillment of a condition. As long as the synchronized action is active, the condition is checked cyclically in the interpolator clock cycle. If no condition is specified, the action is executed cyclically in every interpolator clock cycle.

All operations that return a truth value (TRUE/FALSE) as the result can be programmed as a condition:

- Comparisons of system variables with constants
- Comparisons of system variables with system variables

- Comparisons of system variables with results of arithmetic operations
- Linking of comparisons through Boolean expressions

Examples

Comparisons

Program code

```
ID=1 WHENEVER $AA_IM[X] > $$AA_IM[Y] DO ...
ID=2 WHENEVER $AA_IM[X] > (10.5 * SIN(45)) DO ...
```

Boolean operations

Program code

```
ID=1 WHENEVER ($A_IN[1]==1) OR ($A_IN[3]==0) DO ...
```

See also

Reading and writing (Page 26)

Examples of conditions in synchronized actions (Page 135)

System variables for synchronized actions (Page 25)

4.2.5 G command (action)

Defined initial state

With regard to the part program sequence, synchronized actions can be executed at any time depending on fulfillment of the condition. Therefore, it is advisable to define the required measuring system (inch or metric) in the action part in a synchronized action. This generates a defined initial position for the execution of the action, irrespective of the current part program state.

G commands

The following G commands are permissible:

- G70 (Inch dimensions for geometric specifications (lengths))
- G71 (Metric dimensions for geometric specifications (lengths))
- G700 (Inch dimensions for geometric and technological specifications (lengths, feedrate))
- G710 (Metric dimensions for geometric and technological specifications (lengths, feedrate))

Validity

A G command programmed in the condition part also applies for the action part even if no G command has been programmed in the action part itself.

A G command programmed in the action part only applies within the action part.

4.2.6 Actions with condition fulfilled (DO)

The action part of a synchronized action is initiated with the keyword **DO**.

One or more actions can be programmed in the action part. These are executed when the appropriate condition is fulfilled. If several actions are programmed in one synchronized action, they are all executed in the same interpolator clock cycle.

Example:

If the actual value of the Y axis is greater than or equal to 35.7, the auxiliary function M135 is output on the PLC and, at the same time, digital output 1 = 1 is set.

Program code

```
WHEN $AA_IM[Y] >= 35.7 DO M135 $A_OUT[1]=1
```

Technology cycle

A technology cycle can be called as an action (see Section "Technology cycles (Page 120)").

4.2.7 Actions with condition unfulfilled (ELSE)

With the keyword **ELSE**, every synchronized action can be expanded by actions which are to be executed if the condition is not fulfilled. This allows the combination of synchronized actions which have contrary conditions.

Example:

```
ID=101 WHENEVER $VA_IM[x] < 100 DO $AC_OVR=100
```

```
ID=102 WHENEVER $VA_IM[x] >= 100 DO $AC_OVR=50
```

The two synchronized actions can be brought together by programming **ELSE**:

```
ID=101 WHENEVER $VA_IM[x] < 100 DO $AC_OVR=100 ELSE $AC_OVR=50
```


Additional properties

- In the ELSE branch, the same constraints apply as for the actions with the condition fulfilled (Page 24)
- ELSE is always possible with every condition. The practical use is the responsibility of the user.
Example:

```
WHEN $AA_IW[X] > 100 DO $R1=1 ELSE $R1=2
G0 X100 F10
```

Here, the expression \$R1=2 is executed in each interpolation cycle and the R-parameter is written, even if this is not essential. This must be taken into account by the user.
- The language command ELSE can be used together with all frequency variants.
Examples:
 - The ELSE branch is always executed when the condition is not fulfilled:

```
WHENEVER $AA_IW[X] > 100 DO $R1=1 ELSE $R1=2
EVERY $AA_IW[X] > 100 DO $R1=1 ELSE $R1=2
```
 - As long as the condition is not fulfilled, the ELSE branch is executed, then always the action:

```
FROM $AA_IW[X] > 100 DO $R1=1 ELSE $R1=2
```
 - As long as the condition is not fulfilled, the ELSE branch is executed, then the action and the synchronized action are ended:

```
WHEN $AA_IW[X] > 100 DO $R1=1 ELSE $R1=2
```

4.3 System variables for synchronized actions

The system variables of the NC are listed in the "System Variables" Parameter Manual with their respective properties. System variables that can be read or written in synchronized actions, are marked with an "X" in the corresponding line (Read or Write) of the "SA" (synchronized action) column.

Note

System variables used in synchronized actions are implicitly read and written synchronous to the main run.

More information

For detailed descriptions of the system variables, see the System Variables Parameter Manual.

4.3.1 Reading and writing

System variables

All system variables that can be used in synchronized actions are read/written in the **main run**, with the exceptions below.

The system variables that can be used in synchronized actions are marked with "X" in the "SA" (Synchronized Action) column in the "Read" and/or "Write" line in the "System Variables" List Manual.

Further information

List Manual System Variables

System of the identifiers

The identifiers of the system variables, which are read/written in the main run when used in synchronized actions, have the following systematics:

\$A...	Current main run data
\$V...	Servo data
\$R...	R parameters to be read/written in the main run
\$\$M...	Machine data to be read/written in the main run
\$\$S...	Setting data to be read/written in the main run

Exceptions

The following variables are read/written in the pre-run even when used in synchronized actions:

- User-defined variables: LUD, GUD
- Machine data: \$\$M...
- Setting data: \$\$S...
- R parameters: R<number> or R[<index>]

4.3.2 Operators and arithmetic functions

Operators

Arithmetic operators

System variables of the REAL and INT type can be linked by the following operators:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication

Operator	Meaning
/	- Division in synchronized actions: $\text{INT} / \text{INT} \Rightarrow \text{INT}$ - Division in synchronized actions with REAL result by using the function ITOR(): $\text{ITOR}(\text{INT}) / \text{ITOR}(\text{INT}) \Rightarrow \text{REAL}$ - Division in NC programs: $\text{INT} / \text{INT} \Rightarrow \text{REAL}$
DIV	Integer division: $\text{INT} / \text{INT} \Rightarrow \text{INT}$
MOD	Modulo division (only for type INT) supplies remainder of an INT division Example: $3 \text{ MOD } 4 = 3$

Note

Only variables of the same type may be linked by these operations.

Relational operators

Operator	Meaning
==	Equal to
<>	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

Boolean operators

Operator	Meaning
NOT	NOT
AND	AND
OR	OR
XOR	Exclusive OR

Bit logic operators

Operator	Meaning
B_OR	Bit-by-bit OR
B_AND	Bit-by-bit AND
B_XOR	Bit-by-bit exclusive OR
B_NOT	Bit-by-bit negation

4.3 System variables for synchronized actions

Priority of the operators

The operators have the following priorities for execution in the synchronized action (highest priority: 1):

Priority	Operators	Meaning
1	NOT, B_NOT	Negation, bit-by-bit negation
2	*, /, DIV, MOD	Multiplication, division
3	+, -	Addition, subtraction
4	B_AND	Bit-by-bit AND
5	B_XOR	Bit-by-bit exclusive OR
6	B_OR	Bit-by-bit OR
7	AND	AND
8	XOR	Exclusive OR
9	OR	OR
10	<<	Concatenation of strings, result type STRING
11	==, <>, <, >, >=, <=	Relational operators

Note

It is strongly recommended that the individual operators are clearly prioritized by setting parentheses "(...)" when several operators are used in an expression.

Example of a condition with an expression with several operators:

Program code

```
... WHEN ($AA_IM[X] > VALUE) AND ($AA_IM[Y] > VALUE1) DO ...
```

Arithmetic functions

Operator	Meaning
SIN()	Sine
COS()	Cosine
TAN()	Tangent
ASIN()	Arc sine
ACOS()	Arc cosine
ATAN2()	Arc tangent 2
SQRT()	Square root
ABS()	Absolute value
POT()	Power function
TRUNC()	Integer component The accuracy for comparison commands can be set using TRUNC
ROUND()	Round to an integer
LN()	Natural logarithm
EXP()	Exponential function

Indexing

The index of a system variable of type "Array of ..." can in turn be a system variable. The index is also evaluated in the main run in the interpolator clock cycle.

Example

Program code

```
... WHEN ... DO $AC_PARAM[$AC_MARKER[1]]=3
```

Restrictions

- It is not permissible to nest indices with further system variables.
- The index must not be formed via preprocessing variables. The following example is therefore **not** permitted since \$P_EP is a preprocessing variable:
\$AC_PARAM[1] = \$P_EP[\$AC_MARKER[0]]

4.3.3 Type conversions

An implicit type conversion is performed between the following data types for value assignments and parameter transfers with different data types:

- REAL
- INT
- BOOL

Note

Conversion REAL to INT

For the conversion from REAL to INT, a decimal place value ≥ 0.5 rounded up to the next higher integer. For a decimal place value < 0.5 , rounding is to the next lower integer. Behavior in accordance with the `ROUND` function.

If the REAL value is outside the INT value range, an alarm is displayed and a conversion is not performed.

Conversion from REAL or INT to BOOL

- Value $\neq 0 \rightarrow \text{TRUE}$
 - Value $= 0 \rightarrow \text{FALSE}$
-

Examples

Conversion: **INT** \$AC_MARKER $\rightarrow$ **REAL** \$AC_PARAM

Program code

```
$AC_MARKER[1]=561
ID=1 WHEN TRUE DO $AC_PARAM[1] = $AC_MARKER[1]
```

Conversion: **REAL** \$AC_PARAM $\rightarrow$ **INT** \$AC_MARKER

Program code

```
$AC_PARAM[1]=561.0
```

4.3 System variables for synchronized actions

Program code
ID=1 WHEN TRUE DO \$AC_MARKER[1] = \$AC_PARAM[1]
Conversion: INT \$AC_MARKER → BOOL \$A_OUT
Program code
\$AC_MARKER[1]=561
ID=1 WHEN \$A_IN[1] == TRUE DO \$A_OUT[0]=\$AC_MARKER[1]
Conversion: REAL \$R401 → BOOL \$A_OUT
Program code
R401 = 100.542
WHEN \$A_IN[0] == TRUE DO \$A_OUT[2]=\$R401
Conversion: BOOL \$A_OUT → INT \$AC_MARKER
Program code
ID=1 WHEN \$A_IN[2] == TRUE DO \$AC_MARKER[4] = \$A_OUT[1]
Conversion: BOOL \$A_OUT → REAL \$R10
Program code
WHEN \$A_IN[3] == TRUE DO \$R10 = \$A_OUT[3]

4.3.4 Marker/counter (\$AC_MARKER)

The \$AC_MARKER[<index>] variables are channel-specific arrays of system variables for use as markers or counters.

Data type: INT (integer)
<Index>: Array index: 0, 1, 2, ... (max. number – 1)

Number per channel

The maximum number of \$AC_MARKER variables per channel can be set via the machine data:
MD28256 \$MC_MM_NUM_AC_MARKER = <maximum number>

Storage location

The storage location of the \$AC_MARKER variables can be defined channel-specifically via the machine data:
MD28257 \$MC_MM_BUFFERED_AC_MARKER = <value>

Value	Storage location
0	Dynamic memory (default setting)
1	Static memory

Note**Data backup and memory space**

- The \$AC_MARKER variables created in the static memory can be saved channel-specifically via the data backup. Data block: _N_CH<channel number>_ACM
- Please ensure that sufficient memory is available in the selected memory area. An array element requires 4 bytes of memory space.

Reset behavior

The reset behavior depends on the storage location of the \$AC_MARKER variables:

- Dynamic memory: Initialization with the value "0"

Static memory: Retention of the current value

4.3.5 Parameters (\$AC_PARAM)

The \$AC_PARAM[<index>] variables are channel-specific arrays of system variables for use as general buffers.

Data type: REAL

<Index>: Array index: 0, 1, 2, ... (max. number - 1)

Number per channel

The maximum number of \$AC_PARAM variables per channel can be set via the machine data:

MD28254 \$MC_MM_NUM_AC_PARAM = <maximum number>

Storage location

The storage location of the \$AC_PARAM variables can be defined channel-specifically via the machine data:

MD28255 \$MC_MM_BUFFERED_AC_PARAM = <value>

Value	Storage location
0	Dynamic memory (default setting)
1	Static memory

Note**Data backup and memory space**

- The \$AC_PARAM variables created in the static memory can be saved channel-specifically via the data backup. Data block: _N_CH<channel number>_ACP
 - Please ensure that sufficient memory is available in the selected memory area. An array element requires 4 bytes of memory space.
-

Reset behavior

The reset behavior depends on the storage location of the \$AC_PARAM variables:

- Dynamic memory: Initialization with the value "0"
- Static memory: Retention of the current value

4.3.6 R parameters (\$R)

Whether R parameters are treated as preprocessing or main run variables depends on whether they are written with or without \$ characters. In principle, the notation is freely selectable. For use in synchronized actions, R parameters should be used as main run variables, i.e. with \$ characters:

- \$R[<index>]
- \$R<number>

Data type: REAL

<Index>: Array index: 0, 1, 2, ...

<Number>: Number of the R parameter: 0, 1, 2, ...

The notations with index or number are equivalent.

Parameterizable number per channel

The maximum number of R parameters per channel can be set via the machine data:

MD28050 \$MC_MM_NUM_R_PARAM = <maximum number>

Reset behavior

R parameters are saved persistently in the static memory of the NC. Therefore, R parameters retain their values with all reset types:

- Power on reset
- NC reset
- End of part program reset

Example

Value assignment to R10 in the action part of the synchronized action and subsequent evaluation in the part program

```
WHEN $A_IN[1]==1 DO $R[10]=$AA_IM[Y] ; assignment
G1 X100 F150
STOPRE
IF R[10] > 50 ... ; evaluation in the part program
```

4.3.7 Machine and setting data (\$\$M, \$\$S)**Reading and writing MD and SD**

When machine and setting data is used in synchronized actions, a distinction must be made as to whether this remains unchanged during the execution of the synchronized action, or is changed through parallel processes:

- Data that remain **unchanged** can already be read or written in the **pre-run**.
- Data that is **changed** may only be read or written in the **main run**.

Data access during preprocessing

Machine and setting data that can already be read and written in synchronized actions during preprocessing, is programmed with the same identifiers as in the part program:

\$M... or \$S...

Program code

```
; The reversal position of the Z axis $SA_OSCILL_REVERSE_POS2[Z]
; remains unchanged over the entire machining period
ID=2 WHENEVER $AA_IM[z]<$SA_OSCILL_REVERSE_POS2[Z]-6 DO $AA_OVR[X]=0
```

Data access during the main run

Machine and setting data that are only to be read or written in synchronized actions in the main run must be preceded by an additional character "\$":

\$\$M... or \$\$S...

Program code

```
; The reversal position of the Z axis $SA_OSCILL_REVERSE_POS2[Z]
; can be changed by operator input at any time
ID=1 WHENEVER $AA_IM[z] < $$SA_OSCILL_REVERSE_POS2[Z] DO $AA_OVR[X] = 0
```

4.3 System variables for synchronized actions

Writing during the main run

The following requirements must be satisfied for writing during the main run:

- The access authorization at the time of writing must be sufficient for writing.
- The machine or setting data must have the property "Effective immediately".

Program code

```
; The switching position of the SW cam $SN_SW_CAM_ ... must,
; depending on the current setpoint of the X axis in WCS $AA_IW[X],
; only be written during the main run
ID=2 WHEN $AA_IW[X] > 10 DO $$SN_SW_CAM_PLUS_POS_TAB_1[0] = 20
                                $$SN_SW_CAM_MINUS_POS_TAB_1[0]=20
```

A complete overview of the properties of the machine and setting data can be found in:

More information

- List Manual System Variables
- List Manual Machine Data

4.3.8 Timer (\$AC_TIMER)

The \$AC_TIMER[<index>] variables are channel-specific arrays of system variables.

Data type:	REAL
<Index>:	Array index: 0, 1, 2, ... (max. number - 1)
Unit:	Seconds

Number per channel

The maximum number of \$AC_TIMER variables per channel can be set via the machine data:

MD28258 \$MC_MM_NUM_AC_TIMER = <maximum number>

Function

Starting

A timer is started by assigning a value ≥ 0 :

\$AC_TIMER[<index>] = <starting value>; with starting value ≥ 0

Incrementing

The value of the timer is incremented by the duration of the set interpolator clock cycle (MD10071 IPO_CYCLE_TIME) for each interpolator clock cycle.

\$AC_TIMER[<index>] += <interpolator clock cycle>

Stopping

A timer is stopped by assigning a value < 0 :

$\$AC_TIMER[<index>] = <stopping\ value>;$ with stopping value < 0

When a stopping value is assigned, only the further incrementing of the timer is stopped. The stopping value is not assigned. After the timer is stopped, the last valid value is retained and can still be read.

Note

The current value of a timer can be read when the timer is running or stopped.

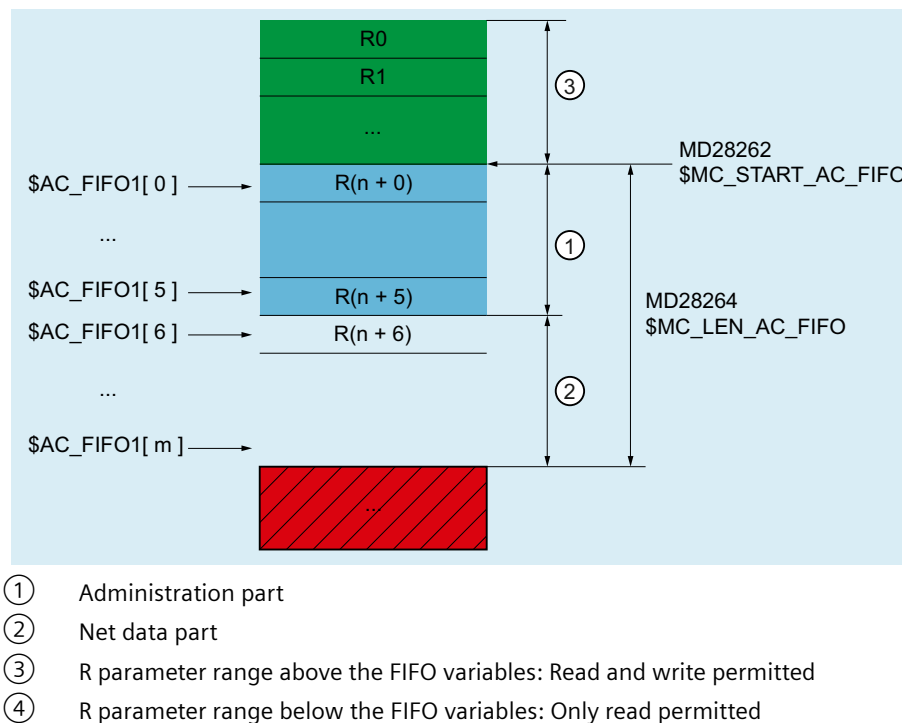
Example

Output the actual value of the X axis as voltage value via analog output $\$A_OUTA[3]$, 500 ms after the detection of digital input $\$A_IN[1]$:

Program code	Comment
WHEN $\$A_IN[1] == 1$ DO $\$AC_TIMER[1]=0$	; Start timer, starting value 0
WHEN $\$AC_TIMER[1]>=0.5$ DO $\$A_OUTA[3]=\$AA_IM[X]$ $\$AC_TIMER[1]=-1$	

4.3.9 FIFO variables ($\$AC_FIFO$)

A FIFO variable provides a complex data structure based on R parameters. The data structure comprises one administration part and one net data part. The net data part is structured as a stack according to the FIFO principle (first in, first out). Using the index of the FIFO variables, the different functions are addressed in the administration part as well as the net data.



Note

The statements regarding R parameters also apply to FIFO variables. See Chapter "R parameters (\$R) (Page 32)".

Syntax

Write

```
$AC_FIFO<n>[<i>] = <value>
$AC_FIFO[<n>, <i>] = <value>
```

Read

```
<variable> = $AC_FIFO<n>[<i>]
<variable> = $AC_FIFO[<n>, <i>]
```

Meaning

\$AC_FIFO:	FIFO data structure in the R parameters, starting from value in MD28262 \$MC_START_AC_FIFO	
	Data type:	REAL
<n>:	Number of FIFO variables	
	Data type:	INT
	Value range:	1, 2, ... max. number (see additional information)

<i>:	Index of the FIFO variables with which the various functions and data within the data structure of the FIFO variables is accessed.	
	Value range:	0, 1, 2, ... (MD28264 \$MC_LEN_AC_FIFO - 1)
	Value	Meaning
	Administrative data	
	0	Write A value is written to the FIFO stack by assigning a value to the FIFO stack via index 0 (\$AC_FIFO[0] = <value>). The assigned value is written to the next free location in the FIFO stack. If all memory locations in the FIFO stack are already occupied, an alarm is displayed: - When writing in an NC program: Alarm 20149 - When writing in a synchronized action: Alarm 17020 Read A value is read from the FIFO stack by assigning the FIFO stack to a variable with index 0 (<variable> = \$AC_FIFO[0]). The oldest value is read and then removed from the FIFO stack. Note - Reading in the NC program / synchronized action If a value is read in an NC program / synchronized action with index 0, the oldest value is read and removed from the FIFO stack as described above. - Reading on the user interface, e.g. SINUMERIK Operate If a value is read or displayed with index 0 on the user interface, e.g. SINUMERIK Operate: "Diagnosis" > "NC/PLC Variables", the value is read internally with index 1 (oldest value) without changing the FIFO stack.
	1	Write / Read: The "oldest" net data is addressed; the FIFO stack is not changed
	2	Write / Read: The "newest" net data is addressed; the FIFO stack is not changed
	3	Read: Returns the sum of the values of all net data Enable via MD28266 \$MC_MODE_AC_FIFO, bit 0 required. See paragraph below "Summation across all net data"
	4	Read: Supplies the number of net data currently stored in the FIFO stack. Write: Reset to the initial state is realized by writing the value of 0 to FIFO variable, index 4. Example: \$AC_FIFO1[4] = 0
	5	Read: Returns the current write index, relative to the beginning of the FIFO stack
	Net data	
	6	Write/read: The 1st field element of the net data range is addressed
	7	Write/read: The 2nd field element of the net data range is addressed
	n	Write/read: The nth field element of the net data range is addressed
	More information List Manual System Variables	

Machine data

Number of FIFO variables per channel

The number of FIFO variables per channel is set using:

MD28260 \$MC_NUM_AC_FIFO = <number of FIFO variables per channel>

Beginning of the R parameter range of FIFO variables

The R-parameter, from which the range of FIFO variables for the channel begins, is set using:

MD28262 \$MC_START_AC_FIFO = <number of the start R parameter>

Note

Free R-parameters

Only the R-parameters whose numbers are below the start R parameter of the FIFO variables, can be written to the NC program.

Number of field elements for each FIFO variable

The maximum number of field elements per FIFO variable is set using:

MD28264 \$MC_LEN_AC_FIFO = <number of field elements per FIFO variable>

Total number of R parameters in the channel

The total number of R parameters, which are required in the channel, is set using:

MD28050 \$MC_MM_NUM_R_PARAMETERS = MD28262 \$MC_START_AC_FIFO +
MD28260 \$MC_NUM_AC_FIFO *
(MD28264 \$MC_LEN_AC_FIFO + 6)

Summation of all net data

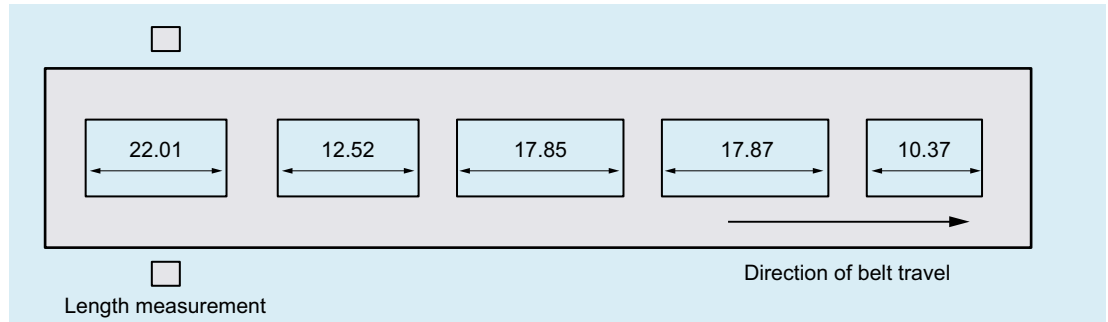
The sum of the values of all net data is only provided via \$AC_FIFO[3] if the function is activated via machine data:

MD28266 \$MC_MODE_AC_FIFO, bit 0 = <value>

Value	Meaning
FALSE	The sum of the values of all net data is not provided
TRUE	The sum of the values of all net data is provided

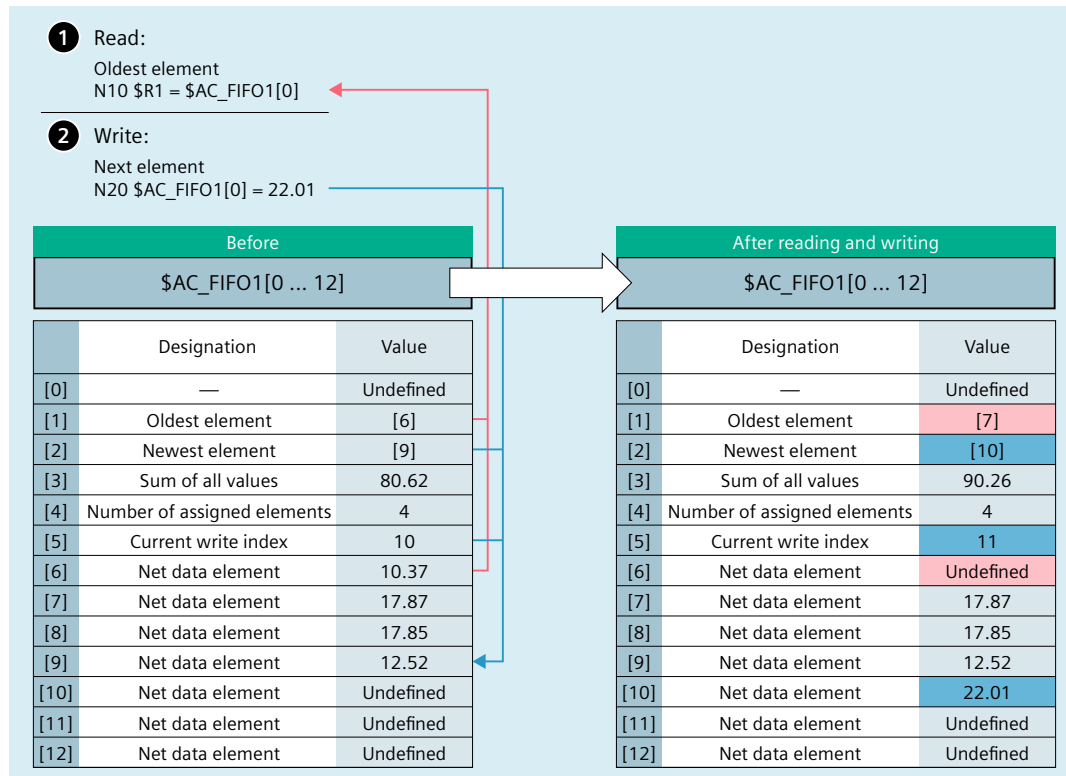
Example

Serial determination of the length of workpieces that move past an automatic measuring station on a conveyor belt.



The measurement results are written to or read from the \$AC_FIFO1 system variable via synchronized actions.

- **Read:** With index 0, the "oldest" net data element is read and deleted from the FIFO stack.
- **Write:** With index 0, the value is written to the next free net data element.



4.3.10 Path tangent angle (\$AC_TANEB)

The angle between the tangent at the end point of the current block and the tangent at the start point of the following block can be read via the channel-specific system variable \$AC_TANEB (Tangent **A**ngle at **E**nd of **B**lock).

Data type: REAL

The tangent angle is always specified positive in the range 0.0 to 180.0°.

If the tangent angle cannot be determined, the value -180.0° is output.

Used only with programmed blocks

It is recommended that the tangent angle only be read for programmed blocks, not for intermediate blocks generated by the system. A distinction can be made via the system variable \$AC_BLOCKTYPE:

\$AC_BLOCKTYPE == 0 (programmed block)

Example:

Program code

```
ID=2 EVERY $AC_BLOCKTYPE==0 DO $R1=$AC_TANEB
```

4.3.11 Override (\$A...OVR)

Current override

Channel-specific override

The path feedrate can be changed via the channel-specific system variable \$AC_OVR.

Data type: REAL

Unit: %

Value range: 0.0 to machine data

- For **binary**-coded override switch
MD12100 \$MN_OVR_FACTOR_LIMIT_BIN
- For **gray**-coded override switch
MD12030 \$MN_OVR_FACTOR_FEEDRATE[30]

The system variable \$AC_OVR must be written in every interpolation cycle, otherwise the value "100%" is effective.

Channel-specific rapid traverse override

With G0 blocks (rapid traverse), the rapid traverse feedrate can also be influenced via the setting data SD42122 \$SC_OVR_RAPID_FACTOR in addition to the system variable \$AC_OVR.

Requirement: Release of the rapid traverse override via the user interface.

Axis-specific override

The axial feedrate can be changed via the axis-specific system variable \$AA_OVR:

Data type: REAL
Unit: %
Value range: 0.0 to machine data

- For **binary**-coded override switch
MD12100 \$MN_OVR_FACTOR_LIMIT_BIN
- For **gray**-coded override switch
MD12030 \$MN_OVR_FACTOR_FEEDRATE[30]

The system variable \$AA_OVR must be written in every interpolation cycle, otherwise the value "100%" is effective.

PLC override

Channel-specific override

The channel-specific override (`<Chan>.basic.out.pathFeedrateOvrFactor`) set via the machine control panel can be read via the channel-specific system variable \$AC_PLC_OVR:

Data type: REAL
Unit: %
Value range: 0.0 to maximum value

Axis-specific override

The axis-specific override (`<Axis>.basic.out.feedrateOvrFactor`) set via the machine control panel can be read via the axis-specific system variable \$AA_PLC_OVR:

Data type: REAL
Unit: %
Value range: 0.0 to maximum value

Effective override

Effective channel-specific override

The effective channel-specific override can be read via the channel-specific system variable \$AC_TOTAL_OVR:

Data type: REAL
Unit: %
Value range: 0.0 to maximum value

4.3 System variables for synchronized actions

Effective axis-specific override

The effective axis-specific override can be read via the axis-specific system variable \$AA_TOTAL_OVR:

Data type: REAL
 Unit: %
 Value range: 0.0 to maximum value

PLC signals**NC → PLC**

Basic Program Plus	Basic Program	
<Chan>.basic.out.pathFeedrateOvrFactor	LBP_Chan*.A_FD_OR	DB21,DBB4
<Axis>.basic.out.feedrateOvrFactor	LBP_Axis*.A_FD_OR	DB31,DBB0

4.3.12 Capacity evaluation (\$AN_IPO ... , \$AN/AC_SYNC ... , \$AN_SERVO)

The values of the current, maximum and average system utilization due to synchronized actions can be read via the following system variables:

NC-specific system variable	Meaning
\$AN_IPO_ACT_LOAD	Current computing time of the interpolator level (incl. synchronized actions of all channels)
\$AN_IPO_MAX_LOAD	Longest computing time of the interpolator level (incl. synchronized actions of all channels)
\$AN_IPO_MIN_LOAD	Shortest computing time of the interpolator level (incl. synchronized actions of all channels)
\$AN_IPO_LOAD_PERCENT	Current computing time of the interpolator level in relation to the interpolation cycle (%)
\$AN_SYNC_ACT_LOAD	Current computing time for synchronized actions over all channels
\$AN_SYNC_MAX_LOAD	Longest computing time for synchronized actions over all channels
\$AN_SYNC_TO_IPO	Percentage share that the synchronized actions have of the total computing time (over all channels)
\$AN_SERVO_ACT_LOAD	Current computing time of the position controller
\$AN_SERVO_MAX_LOAD	Longest computing time of the position controller
\$AN_SERVO_MIN_LOAD	Shortest computing time of the position controller

Channel-specific system variable	Meaning
\$AC_SYNC_ACT_LOAD	Current computing time for synchronized actions in the channel
\$AC_SYNC_MAX_LOAD	Longest computing time for synchronized actions in the channel
\$AC_SYNC_AVERAGE_LOAD	Average computing time for synchronized actions in the channel

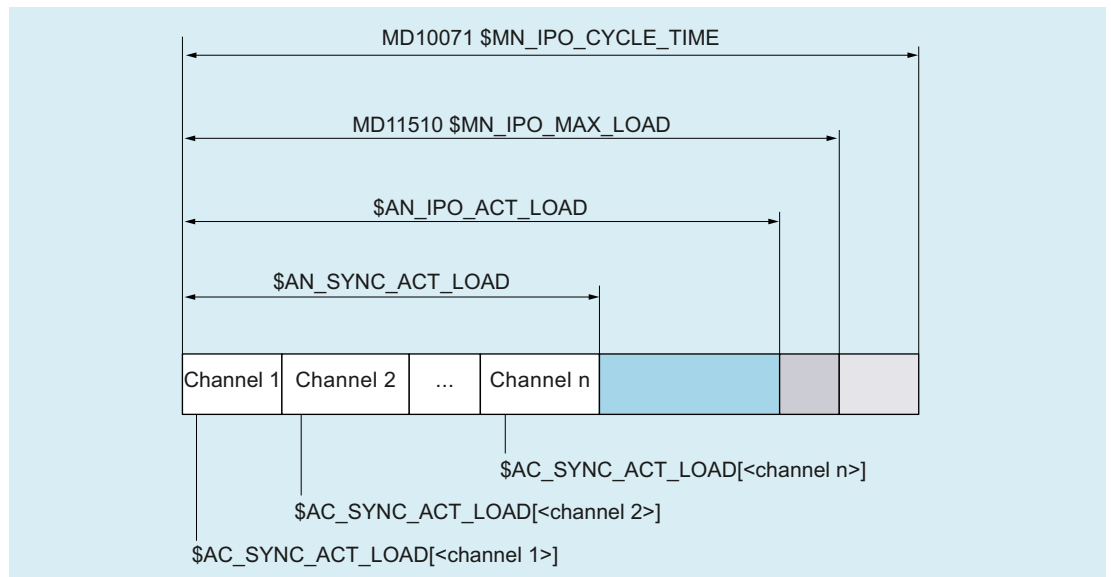


Figure 4-1 Computing time shares of the synchronized actions on the interpolation cycle

Activation

The system variables only contain valid values when the "Utilization evaluation via synchronized actions" diagnostic function is active.

For this, the following machine data must be greater than zero:

MD11510 \$MN_IPO_MAX_LOAD > 0 (maximum permissible interpolator utilization)

When the function is active, the current values are displayed in the "Time required for synchronized actions" line in the "Diagnostics" > "System utilization" operating area.

Note

The system variables always contain the values of the previous interpolation cycle.

Overload limit

An overload limit is specified via the value set via MD11510 \$MN_IPO_MAX_LOAD:

MD11510 \$MN_IPO_MAX_LOAD = <maximum permissible utilization in % of the interpolation cycle>

If the value set in the machine data is exceeded, the following system variable is set:

\$AN_IPO_LOAD_LIMIT = TRUE

If the value falls below the set value again, the system variable is reset:

\$AN_IPO_LOAD_LIMIT = FALSE

Application

A user-specific strategy to avoid a level overflow can be implemented via the system variable \$AN_IPO_LOAD_LIMIT.

4.3 System variables for synchronized actions

Resetting of min./max. values

The following system variables for min./max. values are reset by writing arbitrary values:

System variable	Meaning
\$AN_SERVO_MAX_LOAD	Longest computing time of the position controller
\$AN_SERVO_MIN_LOAD	Shortest computing time of the position controller
\$AN_IPO_MAX_LOAD	Longest computing time of the interpolator level (incl. synchronized actions of all channels)
\$AN_IPO_MIN_LOAD	Shortest computing time of the interpolator level (incl. synchronized actions of all channels)
\$AN_SYNC_MAX_LOAD	Longest computing time for synchronized actions over all channels
\$AC_SYNC_MAX_LOAD	Longest computing time for synchronized actions in the channel

Example

Program code	Comment
\$MN_IPO_MAX_LOAD=80	; Overload limit
;	
; Initialization of the min./max. values	
N01 \$AN_SERVO_MAX_LOAD=0	
N02 \$AN_SERVO_MIN_LOAD=0	
N03 \$AN_IPO_MAX_LOAD=0	
N04 \$AN_IPO_MIN_LOAD=0	
N05 \$AN_SYNC_MAX_LOAD=0	
N06 \$AC_SYNC_MAX_LOAD=0	
;	
; Alarm 63111 when the overload limit is exceeded	
N10 IDS=1 WHENEVER \$AN_IPO_LOAD_LIMIT == TRUE DO M4711 SETAL(63111)	
;	
; Alarm 63222 when the computing time share of the	
; synchronized actions over all channels exceeds 30% of the interpolation cycle	
N20 IDS=2 WHENEVER \$AN_SYNC_TO_IPO > 30 DO SETAL(63222)	
;	
N30 G0 X0 Y0 Z0	
...	
N999 M30	

4.3.13 Working-area limitation (\$SA_WORKAREA_ ...)

Only the activation via the setting data is effective for the traversable command axes in synchronized actions with regard to the programmable working area limitation G25/G26:

- \$SA_WORKAREA_PLUS_ENABLE
- \$SA_WORKAREA_MINUS_ENABLE

Switching the working area limitation on and off via the commands WALIMON/WALIMOF in the part program has no effect on the command axes traversable via synchronized actions.

4.3.14 SW cam positions and times (\$\$SN_SW_CAM_ ...)

The values of the SW cam positions and times can be read and written via the following setting data:

NC-specific setting data	Meaning
\$SN_SW_CAM_MINUS_POS_TAB_1[0..7]	Minus cam positions
\$SN_SW_CAM_MINUS_POS_TAB_2[0..7]	Minus cam positions
\$SN_SW_CAM_PLUS_POS_TAB_1[0..7]	Plus cam positions
\$SN_SW_CAM_PLUS_POS_TAB_2[0..7]	Plus cam positions
\$SN_SW_CAM_MINUS_TIME_TAB_1[0..7]	Minus cam lead or delay time
\$SN_SW_CAM_MINUS_TIME_TAB_2[0..7]	Minus cam lead or delay time
\$SN_SW_CAM_PLUS_TIME_TAB_1[0..7]	Plus cam lead or delay time
\$SN_SW_CAM_PLUS_TIME_TAB_2[0..7]	Plus cam lead or delay time

Note

The setting of a software cam via synchronized actions must not be performed immediately before the cam is reached. At least three interpolation cycles must be available before the cam is reached.

A detailed description of the "Software cam" function can be found in:

Further information

Function Manual Axes and Spindles

Examples

Program code

```
; Changing a cam position:
ID=1 WHEN $AA_IW[x] > 0 DO $$SN_SW_CAM_MINUS_POS_TAB_1[0] = 50.0
...
; Changing a lead time
ID=1 WHEN $AA_IW[x] > 0 DO $$SN_SW_CAM_MINUS_TIME_TAB_1[0] = 1.0
```

4.3 System variables for synchronized actions

See also

Machine and setting data (\$\$M, \$\$S) (Page 33)

4.3.15 Path length evaluation / machine maintenance (\$AA_TRAVEL ... , \$AA_JERK ...)

The data of the path length evaluation, e.g. for machine maintenance, can be read via the system variables listed below.

Activation

The activation for the recording of the path length evaluation data is performed via:

MD18860 \$MN_MM_MAINTENANCE_MON = 1

The data to be recorded for the specific axis can be selected via the following axis-specific machine data:

MD33060 \$MA_MAINTENANCE_DATA[<axis>], bit n = 1

Bit	Meaning
0	Recording of total traversing distance, total traversing time and number of traversing operations of the axis.
1	Recording of total traversing distance, total traversing time and number of traversing operations of the axis at high speed.
2	Recording of total number of axis jerks, the time during which the axis is traversed with jerk and the number of traversing operations with jerk.

System variable

System variable	Meaning	n
\$AA_TRAVEL_DIST	Total travel distance: Sum of all set position changes in MCS in [mm] or [deg.].	0
\$AA_TRAVEL_TIME	Total travel time: Sum of IPO cycles of set position changes in MCS in [s] (resolution: 1 IPO cycle)	
\$AA_TRAVEL_COUNT	Total travel count: A complete machine axis trip is defined by the following succession of states, as based on set position: standstill > traversing > standstill	
\$AA_TRAVEL_DIST_HS	Total traversing distance at high axis velocities ¹⁾	1
\$AA_TRAVEL_TIME_HS	Total traversing time at high axis velocities ¹⁾	
\$AA_TRAVEL_COUNT_HS	Total number of traversing operations at high axis velocities ³⁾	
\$AA_JERK_TOT	Total sum of axis jerks: Sum of all jerk setpoints in [m/s ³] or [deg./ s ³].	2
\$AA_JERK_TIME	Total travel time with jerk: Sum of IPO cycles from jerk setpoint changes in [s] (resolution: 1 IPO cycle)	
\$AA_JERK_COUNT	Total number of traversing operations with jerk	
¹⁾ Actual machine axis velocity ≥ 80% of the maximum parameterized axis velocity (MD32000 MAX AX VELO)		

Further information

For a detailed description of the function, refer to:

Function Manual Axes and Spindles

4.3.16 Polynomial coefficients, parameters (\$AC_FCT ...)

Function

Using the FCTDEF function, as a maximum, a 3rd degree polynomial can be defined:

$$f(x) = a_0 + a_1 \cdot x + a_2 \cdot x^2 + a_3 \cdot x^3$$

Note

The definition must be made in a **part program**.

Syntax

FCTDEF (<Poly_No>, <Lo_Limit>, <Up_Limit>, a₀, a₁, a₂, a₃)

Meaning

Parameter	Meaning
<Poly_No>:	Number of the polynomial function
<Lo_Limit>:	Lower limit of the function values
<Up_Limit>:	Upper limit of the function values
a ₀ , a ₁ , a ₂ , a ₃ :	Polynomial coefficient

Note

Polynomial coefficients (a₂, a₃) that are not required can be omitted when programming the FCTDEF (. . .) function.

System variable

Read and write access to polynomial coefficients and parameters is also possible from synchronized actions via the following system variables:

System variable	Meaning
\$AC_FCTLL [<Poly_No>]:	Lower limit for function value
\$AC_FCTUL [<Poly_No>]:	Upper limit for function value
\$AC_FCT0 [<Poly_No>]:	a ₀
\$AC_FCT1 [<Poly_No>]:	a ₁

4.3 System variables for synchronized actions

System variable	Meaning
\$AC_FCT2[<Poly_No>]:	a_2
\$AC_FCT3[<Poly_No>]:	a_3
<Poly_No>:	The number specified during the definition of the polynomial function (see above: Syntax)

Part program

When writing system variables in the part program, preprocessing stop `STOPRE` must be programmed explicitly for block-synchronous writing.

Note**Block-synchronous writing in the part program**

So that the system variables can be written block-synchronously in the part program, the `STOPRE` command (preprocessing stop) must be used after writing the system variables.

Synchronized action

When writing system variables in synchronized actions, they take effect immediately.

Use

The function value $f(x)$ of the polynomial can be used as input value in synchronized actions, e.g. for the following functions:

- "Polynomial evaluation (SYNFCT) (Page 72)"
- "Online tool offset (FTOC) (Page 77)"

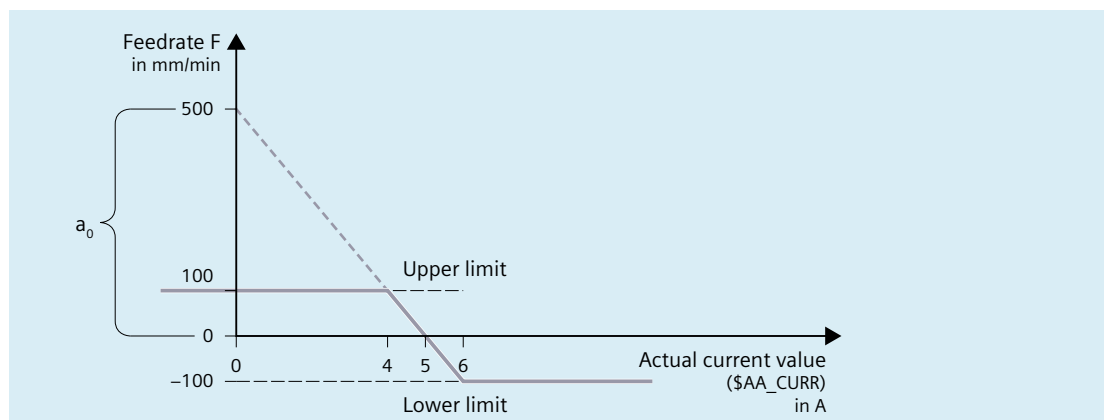
Example: Linear dependency

Figure 4-2 Example of linear dependency

Parameter	Meaning
<Poly_No>:	Number of the polynomial, e.g. = 1
<Lo_Limit>:	Lower limit of the function values = -100

Parameter	Meaning
<Up_Limit>:	Upper limit of the function values = 100
a ₀ :	Axis section on the ordinate (feedrate): $(5 - 4) / 100 = 5 / a_0$ $a_0 = 100 * 5 / (5 - 4) = \mathbf{500}$
a ₁ :	Gradient of the straight line: $a_1 = 100 / (4 - 5) = \mathbf{-100}$
a ₂ :	= 0 (no square component)
a ₃ :	= 0 (no cubic component)

Program code

```
FCTDEF(1, -100, 100, 500, -100, 0, 0)
; Or in abbreviated notation without parameters a2 and a3
FCTDEF(1, -100, 100, 500, -100)
```

4.3.17 Overlaid movements (\$AA_OFF)**Overlaid movements**

The system variable \$AA_OFF can be used to specify a position offset in a channel axis which is traversed immediately:

$\$AA_OFF[\text{<channel axis>}] = \text{<position offset>}$

The following machine data can be used to set whether the position offset of the system variable is to be assigned or summed up (integrated):

MD36750 \$MA_AA_OFF_MODE, bit 0 = <value>

<value>	Meaning
0	Assignment: $\$AA_OFF = \text{<position offset>}$
1	Summing (integration): $\$AA_OFF += \text{<position offset>}$

Limitation of the overlay velocity

The maximum permissible velocity with which the position offset can be traversed can be set via the machine data:

MD32070 \$MA_CORR_VELO (axis velocity for overlay)

Axial jerk limitation

Setting the following machine data activates an axial jerk limitation for the \$AA_OFF overlaying:

MD32420 \$MA_JOG_AND_POS_JERK_ENABLE (basic position of axial jerk limitation) = **1**

4.3 System variables for synchronized actions

The axial jerk is limited to the value set in MD32430 \$MA_JOG_AND_POS_MAX_JERK (axial jerk).

Note

No predictive velocity control can be made for the overlaid \$AA_OFF motion. This can cause a discontinuous velocity change, in particular for clocked specification (via synchronous actions) for \$AA_OFF overlay values. In such cases, we recommend the deactivation of the jerk limitation when possible.

Upper limit of the compensation value

The value of \$AA_OFF can be limited via the following setting data:

SD43350 \$SA_AA_OFF_LIMIT (upper limit of the compensation value \$AA_OFF in case of clearance control)

The status of the limitation can be read via the following system variable:

\$AA_OFF_LIMIT[<axis>] == <value>

Value	Meaning
-1	Compensation value is limited in the negative direction.
1	Compensation value is limited in the positive direction.
0	No limitation of the offset value

Reset behavior

With static synchronous actions (IDS = <number> DO \$AA_OFF = <value>), deselection of the position offset effective in \$AA_OFF results in an immediate new overlaid movement. The reset behavior with regard to \$AA_OFF can therefore be set via the following machine data:

MD36750 \$MA_AA_OFF_MODE, bit 1 = <value>

<value>	Meaning
0	The position offset in \$AA_OFF is deselected with RESET
1	The position offset in \$AA_OFF is retained after RESET

JOG mode

Execution of an overlaid movement because of \$AA_OFF can also be enabled for JOG mode:

MD36750 \$MA_AA_OFF_MODE, bit 2 = <value>

<value>	Meaning
0	JOG mode: Overlaid movement because of \$AA_OFF disabled
1	JOG mode: Overlaid movement because of \$AA_OFF enabled

A mode change to JOG mode is only possible when the current position offset has been traversed. Otherwise the following alarm is displayed:

Alarm "16907 Action ... only possible in stop state"

Boundary conditions

- **Interrupt routines and ASUB**
When an interrupt routine is activated, modal motion-synchronous actions are retained and are also effective in the ASUB. If the subprogram return is not made with `REPOS`, the modal synchronous actions changed in the asynchronous subprogram continue to be effective in the main program.
 - **REPOS**
In the remainder of the block, the synchronous actions are treated in the same way as in an interruption block. Modifications to modal synchronous actions in the ASUB are not effective in the interrupted program. Polynomial coefficients programmed with `FCTDEF` are not affected by ASUB and `REPOS`.
The polynomial coefficients from the calling program are active in the ASUB. The polynomial coefficients from the ASUB continue to be active in the calling program.
 - **End of program**
Polynomial coefficients programmed with `FCTDEF` remain active after the end of program.
 - **Block search: Collection of the polynomial coefficients**
During block search with calculation, the polynomial coefficients are collected in the system variables.
 - **Block search: Deselection of active overlaid movements**
During block search, the `CORROF` and `DRFOF` commands are collected and output in an action block. All the deselected DRF offsets are collected in the last block that contains `CORROF` or `DRFOF`.
The commands for the deselection of overlaid movements `CORROF(<axis>, "AA_OFF")` are not collected during a block search. If a user wishes to continue to use this block search, this is possible by means of block search via "SERUPRO" program testing.
- Further information**
Function Manual Basic Functions
- **Deselection of the position offset in case of synchronous actions**
Alarm 21660 is displayed if a synchronous action is active when the position offset is deselected via the `CORROF(<axis>, "AA_OFF")` command. `$AA_OFF` is deselected simultaneously and not set again. If the synchronous action becomes active later in the block after `CORROF`, `$AA_OFF` remains set and a position offset is interpolated.

Note

The coordinate system (BCS or WCS) in which a main run variable is defined determines whether frames will or will not be included.

Distances are always calculated in the set basic system (metric or inch). A change with `G70` or `G71` has no effect.

DRF offsets, zero offsets external, etc., are only taken into consideration in the case of main run variables that are defined in the MCS.

4.3.18 Online tool length compensation (\$AA_TOFF)

Function

In conjunction with an active orientation transformation or an active tool carrier, tool length compensations can be applied during processing/machining in real time. Changing the effective tool length using online tool length compensation produces changes in the compensatory movements of the axes involved in the transformation in the event of changes in orientation. The resulting velocities can be higher or lower depending on machine kinematics and the current axis position.

Velocity and acceleration, with which specified tool length compensations can be traversed via the system variable \$AA_TOFF, can be specified via the following machine data:

- MD21194 \$MC_TOFF_VELO (velocity, online offset in tool direction)
- MD21196 \$MC_TOFF_ACCEL (acceleration, online offset in tool direction)

For further information regarding the activation of the function, see:

More information

Programming Manual NC Programming

Applications in synchronized actions

In synchronized actions, tool length compensations can be applied in all three dimensions via the system variable \$AA_TOFF. The three geometry axis names X, Y, Z are used as index. All three offset directions can be active at the same time.

For an active orientation transformation or for an active tool carrier that can be oriented, the offsets are effective in the respective tool axes. An overlaid movement must be switched off with TOFFOF () before switching a transformation on or off.

After deselection of the tool length compensation in one dimension, the value of the system variable \$AA_TOFF in this dimension is equal to 0.

Mode of operation of the offset in the tool direction

The tool length compensations do not change the tool parameters, but are taken into account within the transformation or the tool carrier that can be orientated, so that offsets are obtained in the tool coordinate system.

For each dimension, it is possible to define whether the tool length compensation specified in \$AA_TOFF should be calculated as an absolute or incremental (integrating) value via the following machine data:

MD21190 \$MC_TOFF_MODE (operation of tool offset in tool direction)

The current value of the tool length compensation can be read via the system variable \$AA_TOFF_VAL.

Note

An evaluation of the variables \$AA_TOFF_VAL is only useful in conjunction with an active orientation transformation or an active tool carrier.

Examples

Selecting the online tool length compensation

Machine data for online tool length compensation:

- MD21190 \$MC_TOFF_MODE = 1
- MD21194 \$MC_TOFF_VEL[0] = 10000
- MD21194 \$MC_TOFF_VEL[1] = 10000
- MD21194 \$MC_TOFF_VEL[2] = 10000
- MD21196 \$MC_TOFF_ACC[0] = 1
- MD21196 \$MC_TOFF_ACC[1] = 1
- MD21196 \$MC_TOFF_ACC[2] = 1

Activate online tool length compensation in the part program:

Program code

```
N5 DEF REAL XOFFSET
; Activate orientation transformation
N10 TRAORI
; Activate tool length compensation in the Z direction
N20 TOFFON(Z)
; Tool length compensation in the Z direction: 10 mm
N30 WHEN TRUE DO $AA_TOFF[Z] = 10
G4 F5
...
; Static synchronized action: Tool length compensation in the X direction
; corresponds to the position of the X2 axis in the WCS
N50 ID=1 DO $AA_TOFF[X] = $AA_IW[X2]
G4 F5
...
; Note: Current total tool length compensation in the X direction
N100 XOFFSET = $AA_TOFF_VAL[X]
; Retract the tool length compensation in the X direction to 0
N120 TOFFON(X, -XOFFSET)
G4 F5
```

4.3 System variables for synchronized actions

Deselecting the online tool length compensation

Program code

```
; Activate orientation transformation
N10 TRAORI

; Activate tool length compensation in the X direction
N20 TOFFON(X)

; Tool length compensation in the X direction: 10 mm
N30 WHEN TRUE DO $AA_TOFF[X] = 10
G4 F5

...

; Delete tool length compensation in the X direction
; No axis is traversed. To the current position in the WCS
; the position offset in accordance with the current orientation
; is added.
N80 TOFFOF(X)
N90 TRAFOOF
```

Activating and deactivating in the part program

The online tool length compensation is activated in the part program with `TOFFON` and deactivated with `TOFFOF`. When activating for the respective offset direction, an offset value can be specified, e.g. `TOFFON(Z, 25)`, which is then immediately traversed. The status of the online tool length compensation is activated at the NC/PLC interface via the following signals:

- `<Chan>.basic.in.toffonActive` (TOFF active)
- `<Chan>.basic.in.toffonMotionActive` (TOFF motion active)

Note

The online tool length compensation remains inactive until it is reselected using via `TOFFON` in the part program.

Behavior at reset and power on

The behavior at reset can be set via the machine data:

MD21190 \$MC_TOFF_MODE, bit 0 = <value> (operation of tool offset in tool direction)

Value	Meaning
0	The tool length offset \$AA_TOFF is deselected at reset
1	The tool length offset \$AA_TOFF is retained at reset

This is always necessary in case of synchronized actions `IDS=<number> DO $AA_TOFF[n]=<value>`, as otherwise there would be an immediate tool length compensation.

Similarly, a transformation or a tool carrier that can be oriented, can be deselected **after** reset via the following machine data:

MD20110 \$MC_RESET_MODE_MASK (initial setting after reset)

The tool length compensation must also be deleted in this case.

If a tool length compensation is to remain active extending beyond a reset, and a transformation change or a change of the tool carrier that can be oriented takes place, alarm 21665 "Channel %1 \$AA_TOFF[] reset" is output. The tool length compensation is set to 0.

After power on, all tool length offsets are set to 0.

The function is deactivated after POWER ON.

Behavior at change of operating mode

The tool length compensation remains active after a change of operating mode. The offset is executed in all operating modes except JOG and REF.

If a tool length compensation is traversed because of \$AA_TOFF[] at a change of operating mode, the operating mode changeover is only carried out after the traversal of the tool length compensation. Alarm 16907 "Channel %1 action %2 <ALNX> possible only in stop state" is displayed.

Behavior with REPOS

The tool length compensation is active in REPOS mode.

Constraints

With an existing tool length offset, the following constraints must be taken into account:

- A transformation must be switched off with TRAFOFF.
- Before activating a transformation in the part program, an active tool length offset must be deleted with TOFFOF.
- A transformation is switched off when changing over from CP to PTP. A tool length offset must be deleted **before** the changeover. If a tool length compensation is active when you change to axis-specific manual travel in JOG mode, the change to PTP is not performed. CP remains active until the tool length compensation has been deleted via TOFFOF.
- Before a geometry axis interchange, an active tool length offset in the direction of the geometry axis must be deleted via TOFFOF.
- Before a change of plane, an active tool length offset must be deleted via TOFFOF.
- The TOFFON and TOFFOF are not collected during a block search and not output in the action block.

PLC signals

NC → PLC

Basic Program Plus	Basic Program	
<Chan>.basic.in.toffonActive	LBP_Chan*.E_TOFF	DB21,DBX318.2
<Chan>.basic.in.toffonMotionActive	LBP_Chan*.E_TOFFmovem	DB21,DBX318.3

4.3.19 Current block in the interpolator (\$AC_BLOCKTYPE, \$AC_BLOCKTYPEINFO, \$AC_SPLITBLOCK)

Information on the block currently being processed in the main run can be read in synchronized actions via the following system variables.

\$AC_BLOCKTYPE and \$AC_BLOCKTYPEINFO

The system variable \$AC_BLOCKTYPE contains the block type or the ID for the function that generated the block.

The system variable \$AC_BLOCKTYPEINFO contains, in addition to the block type (thousands position), the function-specific cause for the generation of the intermediate block.

\$AC_BLOCKTYPE		\$AC_BLOCKTYPEINFO	
Value	Meaning: Current block has been generated because of ...	Value	Meaning
0	Programmed block!	-	-
1	NC as intermediate block	1000	Contains no further information
2	Chamfer/rounding	2001	Straight line
		2002	Circle
3	Smooth approach/retraction (SAR)	3001	Approach with straight line
		3002	Approach with quadrant
		3003	Approach with semicircle
4	Tool offset	4001	Approach block after STOPRE
		4002	Connection blocks if intersection point not found
		4003	Point-type circle on inner corners (on TRACYL only)
		4004	Bypass circle (or conical cut) at outer corners
		4005	Approach blocks for offset suppression
		4006	Approach blocks on repeated TRC activation
		4007	Block split due to excessive curvature
		4008	Compensation blocks for 3D front milling (tool vector parallel to plane vector)
5	Corner rounding	5001	Rounding contour through G641
		5002	Rounding contour through G642
		5003	Rounding contour through G643
		5004	Rounding contour through G644
6	Tangential tracking (TLIFT)	6001	Linear movement of the tangential axis without lift movement
		6002	Non-linear movement of the tangential axis (polynomial) without lift movement
		6003	Lift movement: Tangential axis and lift movement start simultaneously
		6004	Lift movement: Tangential axis does not start until a certain lift position is reached

\$AC_BLOCKTYPE		\$AC_BLOCKTYPEINFO	
Value	Meaning: Current block has been generated because of ...	Value	Meaning
7	Path segmentation	7001	Programmed path segmentation is active without punching or nibbling
		7002	Programmed path segmentation with active punching or nibbling
		7003	Automatically, internally generated path segmentation
8	Compile cycle	x	x: ID of the compile cycle application that generated the block
9	Path-relative orientation interpolation (ORIPATH/ORIOTC)	9000	Interpolation of the tool orientation with ORIPATH
		9001	Interpolation of the rotation of the tool with ORIOTC
10	Pole handling with orientation transformation	10000	Look-ahead positioning of the pole axis
		10001	Traversal of the pole taper

\$AC_SPLITBLOCK

The system variable \$AC_SPLITBLOCK can be used to determine whether an internally generated block or a programmed block shortened by the NC is present.

\$AC_SPLITBLOCK	
Value	Meaning:
0	Programmed block. A block generated by the compressor is also treated as a programmed block.
1	Internally generated block or a shortened original block
3	Last block in a chain of internally generated blocks or shortened original blocks

Example

Synchronized actions for counting smoothing blocks.

The query of the system variable \$AC_TIMEC == 0 (interpolation cycles since start of the block) ensures that the block type is determined only once at the start of the block.

Program code	Comment
\$AC_MARKER[0]=0	; Counter for all smoothing blocks
\$AC_MARKER[1]=0	; Counter for G641 smoothing blocks
\$AC_MARKER[2]=0	; Counter for G642 smoothing blocks
...	
; Synchronized action for counting all smoothing blocks	
ID=1 WHENEVER (\$AC_TIMEC==0) AND (\$AC_BLOCKTYPE==5) DO	
\$AC_MARKER[0] = \$AC_MARKER[0] + 1	
...	
; Synchronized action for counting the G641 smoothing blocks	
ID=2 WHENEVER (\$AC_TIMEC==0) AND (\$AC_BLOCKTYPEINFO==5001) DO	
\$AC_MARKER[1] = \$AC_MARKER[1]+1	
...	
; Synchronized action for counting the G642 smoothing blocks	

4.3 System variables for synchronized actions

Program code	Comment
ID=3 WHENEVER (\$AC_TIMEC==0) AND (\$AC_BLOCKTYPEINFO==5002) DO	
\$AC_MARKER[2] = \$AC_MARKER[2] + 1	
...	

4.3.20 Initialization of array variables (SET, REP)

Function

Array variables that can be written in synchronous actions can also be initialized using the SET and REP commands in synchronous actions.

Example

Value assignment to INTEGER user variables for synchronous actions in the data block SGUD (SYG_IS)

Precondition: MD18661 \$MM_NUM_SYNACT_GUD_INT[<x>] ≥ 5

Program code	
PROC MAIN	
N10 DEF INT SYG_IS[4]	
...	
WHEN TRUE DO SYG_IS[0]=REP(1,3)	; If condition fulfilled: 1st to 3rd INTEGER user variable for synchronous actions in the SGUD block to be set to the value "1".
WHEN TRUE DO SYG_IS[3]=SET(2,4)	; If condition fulfilled: 4th and 5th INTEGER user variable for synchronous actions in the SGUD block to be set to the value "2" or "4".
...	

More information

Detailed information on SET and REP: → Programming Manual NC Programming

4.3.21 Grinding-specific system variables (\$AC_IN_KEY_G...)

When grinding, input signals asynchronous with the machine operation must be identified and the appropriate actions must be integrated in the program sequence. The following system variables and NC/PLC interface signals are available:

System variable	NC/PLC interface DB21, ...	Description
NC-internal communication		
\$AC_IN_KEY_G_ENABLE[1 ... 8] ¹	---	Input signal enable on the NC side
Communication NC → PLC ²⁾		

System variable	NC/PLC interface DB21, ...	Description
\$AC_IN_KEY_G_ISENABLE[1 ... 8] ¹⁾	<Chan>.grinding.in.signalEnabled	Input signal enable
\$AC_IN_KEY_G_RUN_OUT[1 ... 8]	<Chan>.grinding.in.requested	Enable request for the action on the NC side (optional)
Communication PLC → NC ³⁾		
\$AC_IN_KEY_G[1 ... 8]	<Chan>.grinding.out.set	Input signal
---	<Chan>.grinding.out.disableSignal ¹⁾	Input signal inhibit on the PLC side
\$AC_IN_KEY_G_RUN_IN[1 ... 8]	<Chan>.grinding.out.ackFuncExecution	Enable request for the action on the PLC side (optional)
1) As a result of the AND logic operation of the NC enable signal on the NC side in \$AC_IN_KEY_G_ENABLE and PLC enable signal NOT(<Chan>.grinding.out.disableSignal), the enable signal is formed in \$AC_IN_KEY_G_ISENABLE and <Chan>.grinding.in.signalEnabled. 2) The content of the system variable is transferred in the NC/PLC interface 3) The content of the NC/PLC interface is transferred in the system variable		

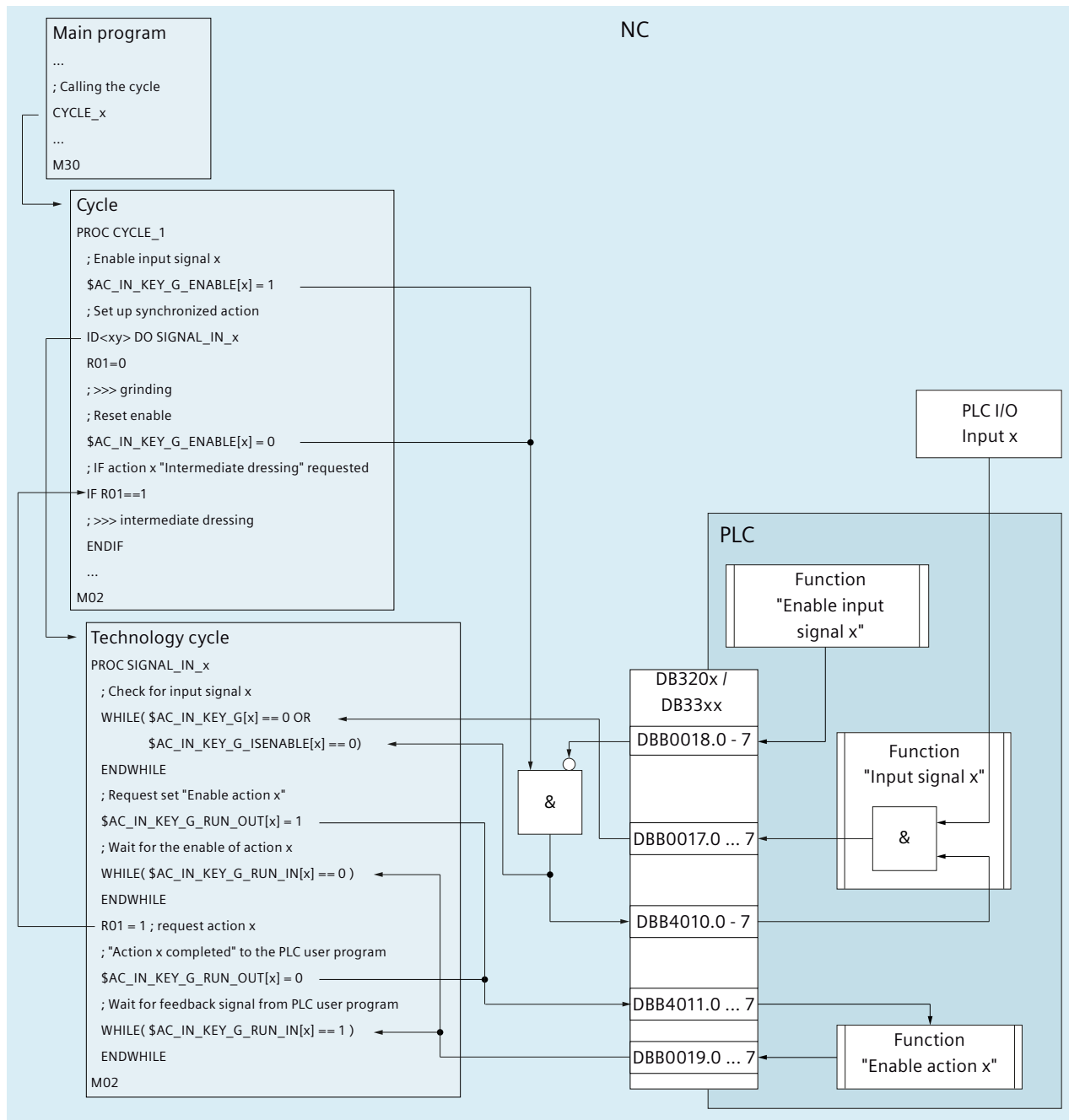
Example

Inputs

- The main program starts a cycle (ZYKLUS_1), in which grinding is executed as well as the intermediate dressing.
- An operator can request an "intermediate dressing" action asynchronous to the machining operation itself using an input signal of the PLC I/O.
- Identifying the input signal and requesting the action is realized in a technology cycle. Technology cycle ("SIGNAL_IN_x") is called in the action part of the synchronized action (ID <xy>) set up in the cycle.

The call schematic, the relevant commands and the signal flow are shown in the following diagram.

4.3 System variables for synchronized actions



- ① <Chan>.grinding.out.disableSignal
- ② <Chan>.grinding.out.set
- ③ <Chan>.grinding.in.signalEnabled
- ④ <Chan>.grinding.in.requested
- ⑤ <Chan>.grinding.out.ackFuncExecution

The PLC user program must provide the functions on the PLC side, for example "enable input signal x".

Sequence

- Main program
 - Call cycle "ZYKLUS_1"
- Cycle "ZYKLUS_1"
 - Set the enable for input signal x (\$AC_IN_KEY_G_ENABLE)
 - Set up the synchronized action with technology cycle "SIGNAL_IN_x"
 - Initialize the trigger for action x "intermediate dressing" (R01)
 - Grind the component
- Technology cycle "SIGNAL_IN_x" (in parallel with the cycle)
 - Identify the input signal (\$AC_IN_KEY_G) with the enable active (\$AC_IN_KEY_G_IENABLE)
 - Request the enable of action x from the PLC (\$AC_IN_KEY_G_RUN_OUT)
 - Wait for the enable of action x from the PLC (\$AC_IN_KEY_G_RUN_IN)
 - Request action x in cycle (R01)
 - Feedback signal of the request to the PLC (\$AC_IN_KEY_G_RUN_OUT)
 - Wait for acknowledgment from the PLC (\$AC_IN_KEY_G_RUN_IN)
- Cycle "ZYKLUS_1" (after completing grinding)
 - Reset the enable for input signal x (\$AC_IN_KEY_G_ENABLE)
 - If action x is requested ⇒ execute intermediate dressing
 - ... (possibly re-execute grinding/intermediate dressing operations etc.)
- PLC user program
 - Function "Enable input signal x":
Setting the interface signal <Chan>.grinding.out.disableSignal
 - Function "Enable input signal x":
Linking (AND) of the input signal from the PLC I/O with the enable of input signal (<Chan>.grinding.in.signalEnabled) and setting the result in the interface (<Chan>.grinding.out.set)
 - Function "Enable action x":
Checking/setting the interface signals <Chan>.grinding.in.requested / <Chan>.grinding.out.ackFuncExecution

PLC signals**NC → PLC**

Basic Program Plus	Basic Program	
<Chan>.grinding.in.signalEnabled	LBP_Chan*.E_InKeyGIsEn1-8	DB21,DBX390.0..7
<Chan>.grinding.in.requested	LBP_AxisX*.E_InKeyGRunOut1-8	DB21,DBX391.0..7
<Chan>.grinding.out.set	LBP_Chan*.A_InKeyG<n>	DB21,DBX385.0..7

4.3 System variables for synchronized actions

Basic Program Plus	Basic Program	
<Chan>.grinding.out.disableSignal	LBP_Chan*.A_InKeyGEn<n>	DB21,DBX386.0..7
<Chan>.grinding.out.ackFuncExecution	LBP_Chan*.A_InKeyGRunIn<n>	DB21,DBX387.0..7

4.3.22 Status Synchronized action disabled (\$AC_SYNA_STATE)

System variable \$AC_SYNA_STATE outputs in bit code whether a synchronized action is disabled via the PLC (Page 127) or a synchronized action (Page 126).

These bits have the following meaning:

Bit	Value	Meaning
0	0	Not disabled
	1	Disabled via PLC or synchronized action
1	0	Not disabled via PLC
	1	Disabled via PLC
2	0	Not disabled via synchronized action
	1	Disabled via synchronized action

Disabling via PLC or synchronized action have different levels of priority. The following hierarchy of priorities applies:

- Priority 1 (highest priority): Disabled via PLC across all channels (⇒ all synchronized actions in the channel are inhibited)
- Priority 2: Disabled via synchronized action
- Priority 3: Individual disabling via PLC (⇒ a single synchronized action in the channel is disabled)

\$AC_SYNA_STATE only outputs the value of the disable with the highest priority, even if disabling is simultaneously active via PLC **and** Synchronized action:

Status	Highest priority	\$AC_SYNA_STATE			
		Bit 2	Bit 1	Bit 0	Bit-coded value
Channel-wide disable via PLC is active. In addition, disable via synchronized action can be active.	Channel-wide disable via PLC	0	1	1	3
Channel-wide disable via PLC is not active. Disable via synchronized action is active. Additionally, a single disable can be active via PLC.	Disabled via synchronized action	1	0	1	5
Channel-wide disable via PLC is not active. Disable via synchronized action is not active. Single disable via PLC is active.	Single disable via PLC	0	1	1	3
No disable is active.	-	0	0	0	0

4.4 User-defined variables for synchronized actions

GUD variables capable of synchronized actions

As well as specific system variables, predefined global synchronized-action user variables (synchronized action GUD) can also be used in synchronized actions. The number of synchronized action GUD items available to the user is parameterized for each specific data type and access using the following machine data:

- MD18660 \$MM_NUM_SYNACT_GUD_REAL[<x>] = <number>
- MD18661 \$MM_NUM_SYNACT_GUD_INT[<x>] = <number>
- MD18662 \$MM_NUM_SYNACT_GUD_BOOL[<x>] = <number>
- MD18663 \$MM_NUM_SYNACT_GUD_AXIS[<x>] = <number>
- MD18664 \$MM_NUM_SYNACT_GUD_CHAR[<x>] = <number>
- MD18665 \$MM_NUM_SYNACT_GUD_STRING[<x>] = <number>

The index <x> is used to specify the data block (access rights) and the value <number> to specify the number of synchronized-action GUDs for each data type (REAL, INT, etc.). A 1-dimensional array variable with the following naming scheme is then created in the relevant data block for each data type.: SYG_<data type><access right>[<index>]:

Index <x>		Data type (MD18660 to MD18665)					
	Block	REAL	INT	BOOL	AXIS	CHAR	STRING
0	SGUD	SYG_RS[i]	SYG_IS[i]	SYG_BS[i]	SYG_AS[i]	SYG_CS[i]	SYG_SS[i]
1	MGUD	SYG_RM[i]	SYG_IM[i]	SYG_BM[i]	SYG_AM[i]	SYG_CM[i]	SYG_SM[i]
2	UGUD	SYG_RU[i]	SYG_IU[i]	SYG_BU[i]	SYG_AU[i]	SYG_CU[i]	SYG_SU[i]
3	GUD4	SYG_R4[i]	SYG_I4[i]	SYG_B4[i]	SYG_A4[i]	SYG_C4[i]	SYG_S4[i]
4	GUD5	SYG_R5[i]	SYG_I5[i]	SYG_B5[i]	SYG_A5[i]	SYG_C5[i]	SYG_S5[i]
5	GUD6	SYG_R6[i]	SYG_I6[i]	SYG_B6[i]	SYG_A6[i]	SYG_C6[i]	SYG_S6[i]
6	GUD7	SYG_R7[i]	SYG_I7[i]	SYG_B7[i]	SYG_A7[i]	SYG_C7[i]	SYG_S7[i]
7	GUD8	SYG_R8[i]	SYG_I8[i]	SYG_B8[i]	SYG_A8[i]	SYG_C8[i]	SYG_S8[i]
8	GUD9	SYG_R9[i]	SYG_I9[i]	SYG_B9[i]	SYG_A9[i]	SYG_C9[i]	SYG_S9[i]

Where i = 0 to (<number> - 1)

Block: _N_DEF_DIR/_N ... _DEF, e.g. for SGUD ⇒ _N_DEF_DIR/_N_SGUD_DEF

Properties

Synchronized action GUD have the following properties:

- Synchronized action GUD can be read and written in synchronized actions and part programs/cycles.
- Synchronized action GUD can be accessed via the OPL.
- Synchronized action GUD is displayed on the HMI user interface in the "Parameters" operating area.

4.5 Language elements for synchronized actions and technology cycles

- Synchronized action GUD can be used on the HMI in the Wizard, in the variables view and in the variables log.
- The array size for STRING type synchronized action GUD is set to a fixed value of 32 (31 characters + \0).
- Even if no definition files have been created manually for global net data (GUD), synchronized action GUD defined using machine data can be read in the corresponding GUD block from the HMI.

Note

User variables (GUD, PUD, LUD) can only be defined with the same name as synchronized action GUD (DEF . . . SYG_xy) if no synchronized-action GUD has been parameterized with the same name (MD18660 - MD18665). These user-defined items of GUD **cannot** be used in synchronized actions.

Access rights

The access rights defined in a GUD definition file remain valid and refer only to the GUD variables defined in this GUD definition file.

Deletion behavior

If the content of a particular GUD definition file is reactivated, the old GUD data block in the active file system is deleted first. The configured synchronized action GUD is also reset at this point. This process is also possible using the HMI in the operating area "Services" > "Define and activated net data (GUD)".

4.5 Language elements for synchronized actions and technology cycles

The following language elements can be used in synchronized actions and technology cycles:

Fixed addresses	
L	Subprogram number
F	Feed
S ^{1) 2)}	Spindle
M ^{1) 2)}	M function
H ¹⁾	H function
1) Chapter: "Output of M, S and H auxiliary functions to the PLC (Page 70)"	
2) Chapter: "Traversing spindles (M, S, SPOS) (Page 95)"	

Fixed addresses with axis extension: Miscellaneous	
POS	Traversing axes to position (Page 82)
POSA	Modal positioning axis
SPOS	Spindle positioning (Page 95)

4.5 Language elements for synchronized actions and technology cycles

Fixed addresses with axis extension: Miscellaneous	
MOV ¹⁾	Traversing axes, endless (Page 88)
FA	Axial feedrate (Page 89)
OVRA	Axial override
ACC	Axial acceleration
MEASA	Axial measurement with deletion of distance-to-go
MEAWA	Axial measurement without delete distance-to-go (Page 113)
MEAC	Cyclic measuring (Page 113)
SCPARA	Parameter set changeover
VELOLIMA	Axial velocity/speed limitation
ACCLIMA	Axial acceleration limitation
JERKLIMA	Axial jerk limitation
1) Not permitted in technology cycles	

Settable addresses: Travel to fixed stop ¹⁾	
FXS	Activate travel to fixed stop
FXST	Torque limit for travel to fixed stop
FXSW	Monitoring window for travel to fixed stop
FOC	Non-modal torque/force limitation
FOCON	Activate travel with limited torque/force
FOCOF	Deactivate travel with limited torque/force
1) Chapter: "Travel to fixed stop (FXS, FXST, FXSW, FOCON, FOCOF, FOC) (Page 116)"	

Settable addresses: Couplings > Generic coupling ¹⁾	
CPBC	Block change criterion with active coupling
CPDEF	Create coupling module
CPDEL	Delete coupling module
CPFMOF	Behavior of the following axis when switching off the coupling
CPFMON	Behavior of the following axis when switching on the coupling
CPFMSON	Synchronization mode during coupling
CPFPOS	Synchronized position of the following axis when switching on
CPFRS	Reference system for the coupling module of the following axis
CPLA ²⁾	Assigning an axis as leading axis to a following axis
CPLCTID	Number of the curve table for the coupling of the following axis
CPLDEF	Definition of the reference: Leading axis to following axis
CPLDEL	Cancellation of the reference: Leading axis to following axis
CPLDEN	Coupling factor: Numerator
CPLNUM	Coupling factor: Denominator
CPLDYPRIO	Priority of the leading axis for the dynamic limitation
CPLDYVLL	Limitation of the overlaid motion of the leading axis: Lower limit
CPLDYVLU	Limitation of the overlaid motion of the leading axis: Upper limit
CPLINSC	Scaling factor for the input value of the leading axis

4.5 Language elements for synchronized actions and technology cycles

Settable addresses: Couplings > Generic coupling ¹⁾	
CPLINTR	Offset value for the input value of the leading axis
CPLOF	Coupling of leading axis to following axis: Switch off
CPLON	Coupling of leading axis to following axis: Switch on
CPLOUTSC	Scaling of the output value
CPLOUTTR	Offset of the output value
CPLPOS	Synchronized position of the leading axis when switching on
CPLSETVAL	Coupling type of the following axis to the leading axis
CPMALARM	Define alarm behavior
CPMBRAKE ²⁾	Defining the response to a stop signal and commands
CPMPRT	Define start behavior for program test
CPMRESET	Define reset behavior
CPMSTART	Define start behavior
CPMVDI	Define behavior regarding NC/PLC interface signals
CPOF	Deactivation of the coupling to all defined leading axes
CPON	Activation of the coupling to all defined leading axes
CPRES ²⁾	Activates the coupling parameter parameterized in the machine data
CPSETTYPE	Define basic coupling properties
CPSYNCOF	Position synchronism "coarse"
CPSYNCOF2	Position synchronism 2 "coarse"
CPSYNFIP	Position synchronism "fine"
CPSYNFIP2	Position synchronism 2 "fine"
CPSYNCOV	Velocity synchronism "coarse"
CPSYNFIV	Velocity synchronism "fine"
1) Chapter: "Couplings (CP..., LEAD..., TRAIL..., CTAB...) (Page 109)"	
2) Currently not available in synchronized actions	

G functions: Set measuring system ¹⁾	
G70	Inch measuring system
G71	Metric measuring system
G700	Inch measuring system
G710	Metric measuring system
1) Chapter: "Setting the measuring system (G70, G71, G700, G710) (Page 86)"	

Predefined subprograms: Miscellaneous	
POLFA	Axial retraction position for single axis
POLFC	Axial retraction position for channel axes
STOPREOF	Revoke preprocessing stop (Page 80)
RDISABLE	Programmed read-in disable (Page 79)
DELDTG	Delete distance-to-go (Page 80)
LOCK	Lock synchronized action
UNLOCK	Unlock synchronized action

4.5 Language elements for synchronized actions and technology cycles

Predefined subprograms: Miscellaneous	
ICYCON	Technology cycle: One block per interpolator clock cycle
ICYCOF	Technology cycle: All blocks in one interpolator clock cycle
SYNFCT	Polynomial evaluation (Page 72)
FTOC	Tool fine compensation (Page 77)
SOFTENDSA	Software limit switch
PROTA	Change status of a protection zone
SETM	Set marker of the channel coordination (Page 118)
CLEARM	Delete marker of the channel coordination (Page 118)
RET	Subprogram return
GET	Request axis (Page 90)
RELEASE	Release axis (Page 90)
AXTOCHAN	Transfer axis to another channel (Page 90)
SETAL	User-specific error responses (Page 119)
IPOBRKA	Block change criterion: Deceleration ramp
ADISPOSA	Tolerance window for end-of-motion criterion

Predefined subprograms: Coupling > Coupled motion ¹⁾	
TRAILON	Coupled motion on
TRAILOF	Coupled motion off
1) Section: "Couplings (CP..., LEAD..., TRAIL..., CTAB...) (Page 109)"	

Predefined subprograms: Couplings > Master value coupling ¹⁾	
LEADON	Master value coupling on
LEADOF	Master value coupling off
1) Section: "Couplings (CP..., LEAD..., TRAIL..., CTAB...) (Page 109)"	

Predefined subprograms: Couplings > Torque coupling (main/sub-coupling)	
MASLON	Coupling on
MASLOF	Coupling off
MASLDEF	Define coupling
MASLDEL	Delete coupling
MASLOFS	Coupling with sub-spindle off

Predefined functions: Coupling > Curve tables ¹⁾	
CTAB	Calculates the following axis position based on the leading axis position using the curve table
CTABINV	Calculates the leading axis position based on the following axis position using the curve table
CTABID	Determines the table number of the curve table
CTABLOCK	Disable curve table

4.5 Language elements for synchronized actions and technology cycles

Predefined functions: Coupling > Curve tables ¹⁾	
CTABUNLOCK	Enable curve table
CTABISLOCK	Determines the lock status of the curve table
CTABEXISTS	Checks whether the curve table exists
CTABMEMTYP	Determines the storage location of the curve table (static/dynamic memory)
CTABPERIOD	Determines the periodicity of the curve table
CTABNO	Determines the number of curve tables
CTABNOMEM	Determines the number of existing curve tables in a specific storage location
CTABSEG	Determines the number of already used curve segments in a specific storage location
CTABSEGID	Determines the number of already used curve segments in a specific table
CTABFSEG	Determines the number of curve segments that are still possible in a specific table
CTABMSEG	Determines the maximum possible number of curve segments in a specific storage location
CTABPOL	Determines the number of already used polynomials in a specific storage location
CTABPOLID	Determines the number of already used polynomials in a specific table
CTABFPOL	Determines the number of polynomials that are still possible in a specific table
CTABMPOL	Determines the maximum possible number of polynomials in a specific storage location
CTABTSV	Determines the following value at the start of the table
CTABTEV	Determines the following value at the end of the table
CTABTSP	Determines the leading value at the start of the table
CTABTEP	Determines the leading value at the end of the table
CTABTMIN	Determines the minimum following value of the table
CTABTMAX	Determines the minimum following value of the table
CTABFNO	Determines the number of curve tables that are still possible in a specific storage location
CTABSSV	Determines the starting value of a table segment for the following axes
CTABSEV	Determines the end value of a table segment for the following axes
1) Section: "Couplings (CP..., LEAD..., TRAIL..., CTAB...) (Page 109)"	

Predefined functions: Arithmetic	
SIN	Sine
ASIN	Arc sine
COS	Cosine
ACOS	Arc cosine
TAN	Tangent
ATAN2	Arc tangent 2
SQRT	Square root

4.5 Language elements for synchronized actions and technology cycles

Predefined functions: Arithmetic	
POT	2nd power (square)
TRUNC	Integer component
ROUND	Round to next integer
ROUNDUP	Rounding up of an input value to the next integer
ABS	Absolute value
LN	Natural logarithm
EXP	Exponential function
MINVAL	Smaller of two values
MAXVAL	Larger of two values
BOUND	Check for defined value range

Predefined functions: Current machine data values	
GETMDACT	Determines the current value of the machine data
GETMDPEAK	Determines the maximum value that has occurred in the machine data since the last RESETPEAK
GETMDLIM	Determines the maximum or minimum limit value of the machine data
RESETPEAK	Resets the maximum value again for GETMDPEAK

Predefined functions: Format conversions	
ITOR	INT → REAL
RTOI	REAL → INT
RTOB	REAL → BOOL
BTOR	BOOL → REAL
ITOB	INT → BOOL
BTOI	BOOL → INT

Predefined functions: Miscellaneous	
POSRANGE	Position in specified reference range (Page 87)
PRESETON	Actual value setting with loss of the referencing status (Page 97)
PRESETONS	Actual value setting without loss of the referencing status (Page 103)

Predefined procedures: Miscellaneous	
CANCELSUB	Cancels the current subprogram level (Page 119)

More information

A detailed description of all NC language elements is provided in the Programming Manual NC Programming.

4.6 Language elements for technology cycles only

The following language elements may only be used in technology cycles:

Jump statements	
IF	Branch
GOTO	Jump to label, search direction forward, then backward
GOTOF	Jump to label, search direction forward
GOTOB	Jump to label, search direction backward

End of program	
M02	End of program
M17	End of program
M30	End of program
RET	End of program

4.7 Actions in synchronized actions

4.7.1 Output of M, S and H auxiliary functions to the PLC

Output timing

Auxiliary functions of the M, S and H type can be output from synchronized actions. The output to the PLC is immediate, i.e. directly in the interpolator clock cycle in which the action is executed.

Any output times set via the machine data for auxiliary functions have no effect when output from synchronized actions:

- MD11110 \$MN_AUXFU_GROUP_SPEC (auxiliary function group specification)
- MD22200 \$MC_AUXFU_M_SYNC_TYPE (output time of M functions)
- MD22210 \$MC_AUXFU_S_SYNC_TYPE (output time of the S functions)
- MD22230 \$MC_AUXFU_H_SYNC_TYPE (output time of the H functions)

Maximum number

General

A maximum of 10 auxiliary functions can be output simultaneously from the part program and the active synchronized actions of a channel, i.e. in one OB40 cycle of the PLC.

Synchronized-action-specific

The maximum permissible number of auxiliary functions in the action part of a synchronized action is:

- M functions: 5
- S functions: 3
- H functions: 3

Non-modal synchronized actions

In non-modal synchronized actions (without specification of `ID` or `IDS`), auxiliary functions can only be output in conjunction with the scanning frequency `WHEN` or `EVERY`.

Predefined M functions

Predefined M functions generally must not be output in synchronized actions.

Exceptions: M3, M4, M5, M40, M41, M42, M43, M44, M45, M70 and M17

See also

Frequency (`WHENEVER`, `FROM`, `WHEN`, `EVERY`) (Page 21)

4.7.2 Reading and writing of system variables

The system variables of the NC are listed in the "System Variables" Parameter Manual with their respective properties. System variables that can be read or written in the action part of synchronous actions are marked with an "X" in the corresponding line (Read or Write) of the "SA" (synchronous action) column.

Note

System variables used in synchronous actions are implicitly read and written synchronous to the main run.

Further information

List Manual System Variables

4.7.3 Polynomial evaluation (SYNFCT)

Application

A variable that is evaluated via a polynomial can be read with the SYNFCT function in the main run and the result can be written to another variable. Application examples:

- Feedrate as a function of drive load
- Position as a function of a sensor signal
- Laser power as a function of path velocity

Syntax

SYNFCT(<Poly_No>,<SysVar_Out>,<SysVar_In>)

Meaning

Parameter	Meaning
<Poly_No>:	Number of the polynomial defined with FCTDEF: $f(x) = a_0 + a_1 \cdot x + a_2 \cdot x^2 + a_3 \cdot x^3$
<SysVar_Out>:	System variable, output: $\text{<SysVar_Out>} = f(x)$
<SysVar_In>:	System variable, input: $x = \text{<SysVar_In>}$
For information on FCTDEF, see Chapter "Polynomial coefficients, parameters (\$AC_FCT ...) (Page 47)"	

Example: Additive override of the path feedrate

An override value is added to the programmed feedrate (F word):

$$F_{\text{active}} = F_{\text{programmed}} + F_{\text{AC}}$$

<SysVar_Out>	Meaning
\$AC_VC	additive path feedrate override
\$AA_VC[axis]	additive axial feedrate override

Input value is the actual current value \$AA_CURR of the X axis.

The operating point is set to 5 A.

The feedrate may be altered by ± 100 mm/min and the axial current deviation may be ± 1 A.

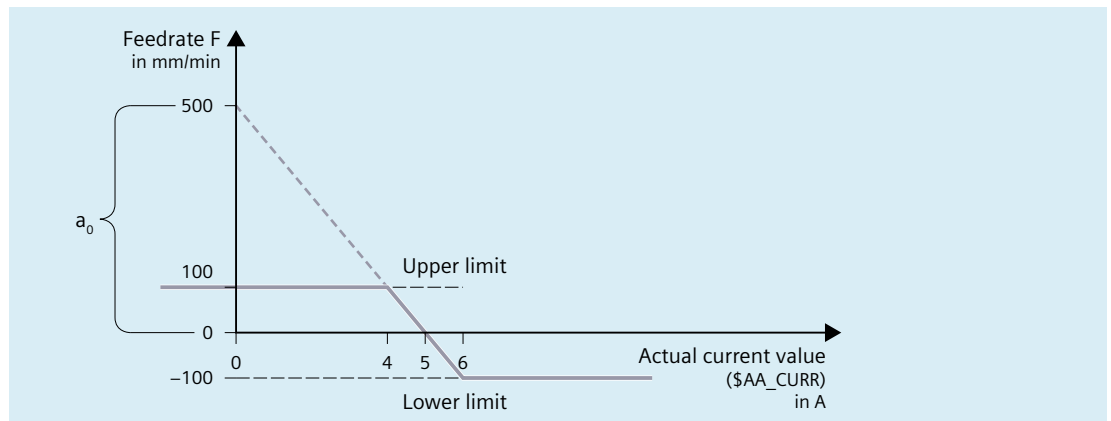


Figure 4-3 Example: Additive control of path feed

Determining the parameters of the FCTDEF function:

FCTDEF(<Poly_No>, <Lo_Limit>, <Up_Limit>, a_0 , a_1 , a_2 , a_3)

<Poly_No>: = 1 (example)

<Lo_Limit>: = -100

<Up_Limit>: = 100

Polynomial: $f(x) = a_0 + a_1x + a_2x^2 + a_3x^3$

a_0 : $1 / 100 = 5 / a_0 \Rightarrow a_0 = 500$

a_1 : = 100 mm/min / -1 A = -100 [mm/min / A]

a_2 : = 0 (not a square component)

a_3 : = 0 (not a cubic component)

Calculation of the override value:

SYNFCT(<Poly_No>, <SysVar_Out>, <SysVar_In>)

<Poly_No>: = 1

<SysVar_Out>: \$AC_VC (additive path feedrate override)

<SysVar_In>: \$AA_CURR (drive actual current value)

Programming:

Program code

```
N100 FCTDEF(1, -100, 100, 500, -100)
N110 ID=1 DO SYNFCT(1, $AC_VC[X], $AA_CURR[X])
```

Example: Multiplicative override of the path feedrate

The programmed feedrate is multiplied by a percentage factor (additional override):

$$F_{\text{active}} = F_{\text{programmed}} * \text{Factor}_{AC}$$

<SysVar_Out>	Meaning
\$AC_OVR	Path override can be specified via synchronous action

Input value is the percentage drive load \$AA_LOAD of the X axis.

4.7 Actions in synchronized actions

The operating point is set to 100% at 30% drive load.

The axis must stop at 80% load.

An excessive velocity corresponding to the programmed value +20% is permissible.

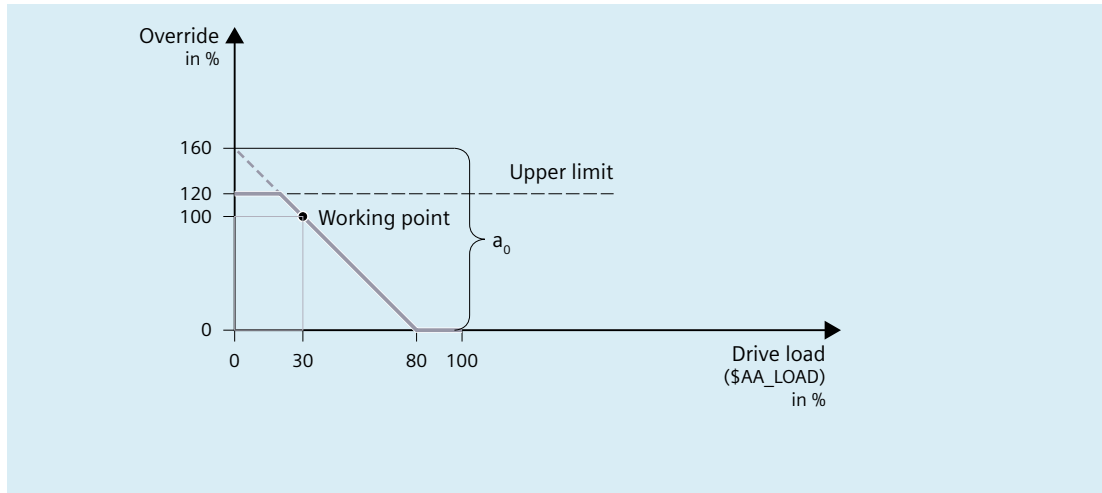


Figure 4-4 Example: Multiplicative control

Determining the parameters of the FCTDEF function:

FCTDEF(<Poly_No>, <Lo_Limit>, <Up_Limit>, a_0 , a_1 , a_2 , a_3)

<Poly_No>: = 2 (example)

<Lo_Limit>: = 0

<Up_Limit>: = 120

Polynomial: $f(x) = a_0 + a_1x + a_2x^2 + a_3x^3$

a_0 : $50 / 100 = 80 / a_0 \Rightarrow a_0 = 160$

a_1 : $= 100\% / -50\% = -2$

a_2 : = 0 (not a square component)

a_3 : = 0 (not a cubic component)

Calculation of the override value:

SYNFCT(<Poly_No>, <SysVar_Out>, <SysVar_In>)

<Poly_No>: = 2

<SysVar_Out>: \$AC_OVR (path override can be specified via synchronous action)

<SysVar_In>: \$AA_LOAD (drive load)

Programming:

Program code

N100 FCTDEF(2, 0, 120, 160, -2)

N110 ID=1 DO SYNFCT(2, \$AC_OVR[X], \$AA_LOAD[X])

Example: Clearance control

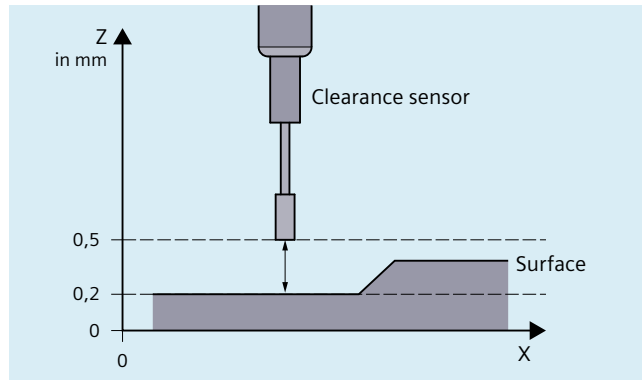


Figure 4-5 Clearance control: Principle

The clearance control of the infeed axis Z is performed via the `FCTDEF` and `SYNFCT` functions as well as by the system variables `$AA_OFF` and `$A_INA`.

Constraints:

- The analog voltage of the clearance sensor is connected via the analog input `$A_INA[3]`.
- The position deviations are summated in `$AA_OFF` (integrated):
MD36750 `$MA_AA_OFF_MODE`, bit 0 = 1
- If the upper limit of the Z axis is exceeded by 1 mm, the X axis is stopped:
SD43350 `$SA_AA_OFF_LIMIT[Z] = 1`
See also Chapter "Overlaid movements (`$AA_OFF`) (Page 49)".

Note

`$AA_OFF` is effective in the basic coordinate system (BCS)

The offset is effective before the kinematic transformation in the basic coordinate system (BCS). The example therefore **cannot** be used for a clearance control in the orientation direction of the tool (workpiece coordinate system WCS).

For clearance control system with high dynamic response or 3D clearance control, see:

More information

Function Manual Technologies

User-specific responses

When the limit value SD43350 `$SA_AA_OFF_LIMIT` is reached, customized responses can be triggered, for example:

- Section "Override (`$A...OVR`) (Page 40)"
- Section "User-specific error reactions (`SETAL`) (Page 119)"

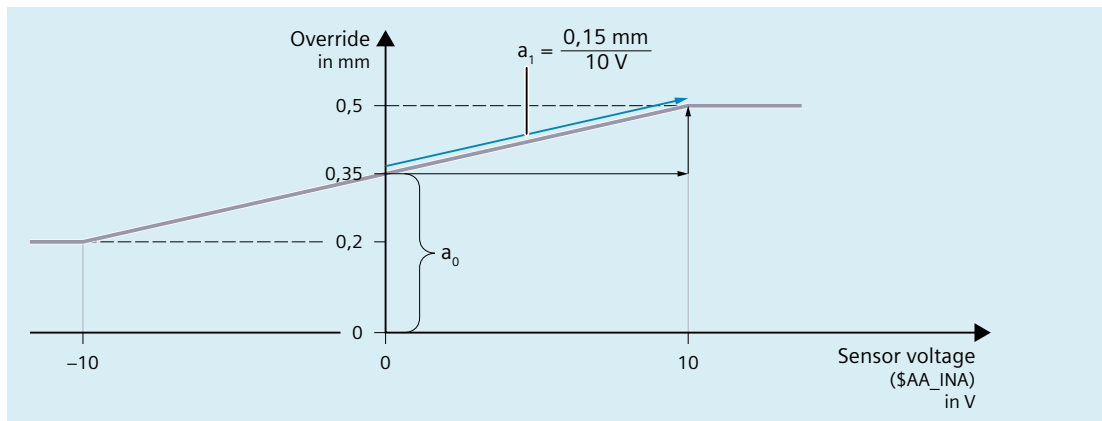


Figure 4-6 Clearance control

Determining the parameters of the FCTDEF function:

FCTDEF(<Poly_No>, <Lo_Limit>, <Up_Limit>, a_0 , a_1 , a_2 , a_3)

<Poly_No>: = 1 (example)

<Lo_Limit>: = 0.2

<Up_Limit>: = 0.5

Polynomial: $f(x) = a_0 + a_1x + a_2x^2 + a_3x^3$

a_0 : $10 / x = 20 / 0.3 \Rightarrow a_0 = x + 0.2 = 0.15 + 0.2 = 0.35$

a_1 : $= 0.15 \text{ mm} / 10 \text{ V} = 1.5 \cdot 10^{-2} \text{ mm/V}$

a_2 : = 0 (not a square component)

a_3 : = 0 (not a cubic component)

Calculation of the override value:

SYNFCT(<Poly_No>, <SysVar_Out>, <SysVar_In>)

<Poly_No>: = 1

<SysVar_Out>: \$AA_OFF (overlaid movement of an axis)

<SysVar_In>: \$A_INA (analog input)

Programming:

Program code: %_N_AON_SPF	Comment
PROC AON	; Clearance control "ON"
FCTDEF(1, 0.2, 0.5, 0.35, 1.5 EX-2)	; Polynomial definition
ID=1 DO SYNFCT(1, \$AA_OFF[Z], \$A_INA[3])	; Clearance control
ID=2 WHENEVER \$AA_OFF_LIMIT[Z] <> 0 DO \$AA_OVR[X] = 0	; Limit value test
RET	
ENDPROC	

Program code: %_N_AOFF_SPF	Comment
PROC AOFF	; Clearance control "OFF"
CANCEL(1)	; Delete clearance control
CANCEL(2)	; Delete limit value check

Program code: %_N_AOFF_SPF	Comment
RET	
ENDPROC	

Program code: %_N_MAIN_MPF	Comment
N100 \$SA_AA_OFF_LIMIT[Z]=1	
N110 AON	; Clearance control "ON"
...	
N200 G1 X100 F1000	
N210 AOFF	; Clearance control "OFF"
M30	

See also

Online tool offset (FTOC) (Page 77)

4.7.4 Online tool offset (FTOC)

The FTOC function enables the overlaid movement of a geometry axis for the online tool offset, depending on a reference value, e.g. the actual value of an arbitrary axis. The offset value is calculated on the basis of a polynomial defined with `FCTDEF` (see Section "Polynomial coefficients, parameters (\$AC_FCT ...) (Page 47)"). The coefficient a_0 specified in the polynomial definition is also evaluated by `FTOC`.

Example: Machining and dressing in the "Grinding" technology

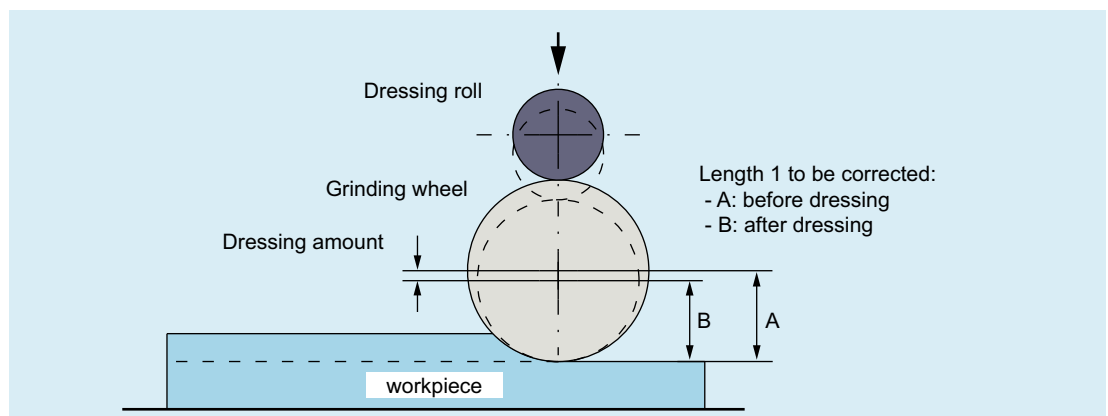


Figure 4-7 Dressing during machining using a dressing roll

More information

Function Manual for tools

Syntax

`FTOC(<Poly_No>,<systemvar>,<wear>[,<channel_No>,<spindle_No>])`

Meaning

Parameter	Meaning
<Poly_No>:	Number of the polynomial defined with FCTDEF
<Systemvar>:	Arbitrary system variable of the REAL type that can be used in synchronous actions.
<wear>:	Wear parameter (length 1, 2 or 3) in which the offset value is added .
<Channel_No>:	Target channel in which the offset must be applied. This enables simultaneous dressing from a parallel channel. In the target channel of the offset, the online offset must be switched on with FTOCON. If no channel number is programmed, the offset acts in the active channel.
<Spindle_No>:	The spindle number is programmed if a non-active grinding wheel needs to be dressed. Requirement: One of the following functions is active "Constant grinding wheel peripheral speed" "Tool monitoring" If no spindle number is programmed, the active tool is compensated.

Example

Compensate length of an active grinding wheel

Program code	Comment
FCTDEF(1, -1000, 1000, -\$AA_IW[V], 1)	
; FTOC:	
; Polynomial no.: 1	
; System variable: \$AA_IW[V] (axial actual value of the V axis)	
; Wear parameter: Length 3	
; Target channel: Channel 1	
ID=1 DO FTOC(1, \$AA_IW[V], 3, 1)	
WAITM (1,1,2)	; Synchronization with the machining channel
G1 V-0.05 F0.01 G91	; Traversing motion of the V axis
...	
CANCEL(1)	; Deselect online offset
...	

Note

Because no frequency and no condition has been specified in the synchronous action, the action part is executed in every interpolator clock cycle.

4.7.5 Programmed read-in disable (RDISABLE)

Function

The `RDISABLE` command in the active section causes block processing to be stopped when the relevant condition is fulfilled. Processing of programmed motion-synchronous actions still continues. The read-in disable is canceled again as soon as the condition for the `RDISABLE` is no longer fulfilled.

An exact stop is initiated at the end of the block containing `RDISABLE` irrespective of whether or not the read-in disable is still active. The exact stop is also triggered if the control is in the continuous-path mode (G64, G641 ... G645).

`RDISABLE` can be programmed with reference to the block or also modal (`ID=`, `IDS=`)!

Application

Using `RDISABLE`, for example, the program can be started in the interpolator clock cycle as a function of external inputs.

Example

Program code	Comment
WHENEVER \$A_INA[2]<7000	; Program processing is stopped if the voltage at input
DO RDISABLE	2 drops to below 7 V (assuming that the value 1000 corresponds to 1 V).
...	
N10 G01 X10	; RDISABLE acts at the end of N10, if the condition is fulfilled during its processing.
N20 Y20	
...	

Supplementary conditions

Read-in disable `RDISABLE` in conjunction with axis exchange

Acts via the synchronized actions `RDISABLE` read-in disable and axis exchange (e.g. path axis → positioning axes) together in one block, `RDISABLE` does not act on the action block, but the re-approach block `REPOSA` implicitly generated as a result of the axis exchange:

Program code	Comment
N100 G0 G60 X300 Y300	
N105 WHEN TRUE DO POS[X]=20 FA[X]=20000	; Synchronized action → REORG → REPOSA
N110 WHENEVER \$AA_IM[X]<>20 DO RDISABLE	; RDISABLE acts on REPOSA
N115 G0 Y20	; 1. X-axis, 2nd Y axis
N120 Y-20	
N125 M30	

Path axis X becomes a positioning axis as a result of the synchronized action in the block N105. REORG is therefore executed in the channel with REPOSA. Therefore, `RDISABLE`

4.7 Actions in synchronized actions

in N110 does not act on block N115 – but instead on the internal `REPOSA` block. As a consequence, to start, positioning axis X is traversed to its programmed position and then in block N115, the Y axis to its programmed position.

An explicit release of path axis X before traversing as positioning axis (synchronized action in N105) with `RELEASE (X)` avoids the `REORG` operation, and the X and Y axes traverse together in block N115.

Program code	Comment
N100 G0 G60 X300 Y300	
N101 RELEASE (X)	; Explicit release
N105 WHEN TRUE DO POS[X]=20 FA[X]=20000	
...	

4.7.6 Cancel preprocessing stop (STOPREOF)

With the `STOPREOF` command, an existing preprocessing stop can be cancelled from a synchronized action.

Note

The `STOPREOF` command can only be programmed in non-modal synchronized actions (without specification of ID or IDS) and only in conjunction with the scanning frequency `WHEN`.

Example

- N10: Non-modal synchronized action.
If the path distance-to-go `$AC_DTEB` is less than 5 mm, the existing preprocessing stop due to the reading of the analog input `$A_INA` is cancelled.
- N20: Traversing block whose path distance-to-go is evaluated via `$AC_DTEB`.
- N30: Branch that triggers the preprocessing stop due to the reading of `$A_INA`.

Due to the synchronized action, input `$A_INA` is not evaluated at the end of the N20 block, but already 5 mm before the end of the block. If the voltage is then greater than 5 V at input `$A_INA`, there is a branch to "MARKE_1".

Program code
N10 WHEN <code>\$AC_DTEB</code> < 5 DO <code>STOPREOF</code>
N20 G01 X100
N30 IF <code>\$A_INA[7]</code> > 5000 GOTOF MARKE_1

4.7.7 Delete distance-to-go (DELDTG)

The path distance-to-go can be deleted with the `DELDTG` command and axial distances-to-go can be deleted with the `DELDTG ( . . . )` function in synchronized actions.

After deletion of the distance-to-go, the value of the deleted distance-to-go can be read via a system variable:

- Path distance-to-go: \$AC_DELT
- Axial distance-to-go: \$AA_DELT

Syntax

```
DELDTG
DELDTG(<axis 1>[,<axis 2>, ... ])
```

Meaning

Parameter	Meaning
DELDTG	Deletion of the path distance-to-go
DELDTG(...)	Deletion of the axial distances-to-go of the specified channel axes
<Axis n>:	Channel axis

Supplementary conditions

Path-specific and axial delete distance-to-go

Path-specific and axial delete distance-to-go can only be executed in a **non-modal** synchronized action (without specification of ID or IDS).

Path-specific delete distance-to-go

- The deletion of the path distance-to-go can only be executed in a non-modal synchronized action (without specification of ID or IDS).
- The deletion of the path distance-to-go must **not** be used with active tool radius compensation.

Axial delete distance-to-go

Delete distance-to-go for indexing axes:

- **Without** Hirth tooth system: The axis is braked immediately
- **With** Hirth tooth system: The axis traverses to the next indexing position

Examples

Delete path distance-to-go

If the input \$A_IN is set during the traversing block N20, the path distance-to-go is deleted.

```
Program code
N10 WHEN $A_IN[1]==1 DO DELDTG
N20 G01 X100 Y100 F1000
```

4.7 Actions in synchronized actions

Delete axial distances-to-go

N10: If input 1 is set at any time within the part program, the V axis is started as a positioning axis in the positive traversing direction.

N100: Non-modal synchronized action to delete distance-to-go of the V axis, depending on digital input 2.

N110: Non-modal synchronized action to delete distance-to-go of the X1 axis, depending on digital input 3.

N120: The X1 axis is positioned modally. The Y and Z axes are traversed as path axes. The non-modal synchronized actions from N100 and N110 are executed together with N120. The non-modal synchronized actions are also terminated with the end of block N120.

For this reason, the distances-to-go of the X1 and V axes can only be deleted as long as N120 is active.

Program code

```
N10 ID=1 WHEN $A_IN[1]==1 DO MOV[V]=1 FA[V]=700
...
N100 WHEN $A_IN[2]==1 DO DELDTG(V)
N110 WHEN $A_IN[3]==1 DO DELDTG(X1)
N120 POSA[X1]=100 FA[X1]=10 G1 Y100 Z100 F1000
```

4.7.8 Traversing axes, to position (POS)

With the **POS** command, an axis can be traversed using a synchronized action. The axis is then called the command axis. It is possible to traverse the axis alternating via the part program and the synchronized action. If a command axis traversed via synchronized actions is subsequently traversed via the part program, a preprocessing stop with reorganization (**STOPRE**) is executed in the channel of the part program.

Example 1: Alternate traversing via part program and synchronized action

Program code**Comment**

N10 G01 X100 Y200 F1000	; Traversing via part program
...	
; Traversing via static synchronized action when input 1 is set	
N20 ID=1 WHEN \$A_IN[1]==1 DO POS[X]=150 FA[X]=200	
...	
CANCEL(1)	; Deselect synchronized action
...	
; Traversing again via part program => implicit preprocessing stop	
; with reorganization, if the X axis in the meantime has been	
; traversed via synchronized action	
N100 G01 X240 Y200 F1000	

Example 2: Alternate traversing of the X-axis via two synchronized actions

If the traversing motion of one synchronized action is still active when the traversing motion of the other synchronized action is started, the second traversing motion replaces the first.

Program code

```
; 1st traversing motion
ID=1 EVERY $A_IN[1]>=1 DO POS[V]=100 FA[V]=560
; 2nd traversing motion
ID=2 EVERY $A_IN[2]>=1 DO POS[V]=$AA_IM[V] FA[V]=790
```

Dimensions: Absolute/incremental

The commands G90/G91 to specify the dimensions (absolute/incremental) cannot be programmed in synchronized actions. Therefore by default, the dimensions that were active in the part program at the time of execution of the synchronized action are also effective in the synchronized action.

The following commands can be programmed in the action part to specify the dimensions within a synchronized action:

Command	Meaning
IC (...)	Incremental
AC (...)	Absolute
DC (...)	Direct (position rotary axis via shortest route)
ACN (...)	Position modulo rotary axis absolutely in negative direction of motion
ACP (...)	Position modulo rotary axis absolutely in positive direction of motion
CAC (...)	Traverse axis to coded position absolutely
CIC (...)	Traverse axis to coded position incrementally
CDC (...)	Traverse rotary axis to coded position via shortest route
CACN (...)	Traverse modulo rotary axis to coded position in negative direction
CACP (...)	Traverse modulo rotary axis to coded position in positive direction

Examples:

Program code

```
; Incremental traversing by 10 mm
ID=1 EVERY G710 $AA_IM[B]>75 DO POS[X]=IC(10)
...
; Absolute traversing
ID=1 EVERY G710 $AA_IM[B]>75 DO POS[X]=AC($AA_MW[V]-$AA_IM[W]+13.5)
```

Behavior with active axis-specific frames

Whether programmable and settable axis-specific frames and tool length compensations are included in synchronized actions, depends on the following MD setting:

4.7 Actions in synchronized actions

MD32074 \$MA_FRAME_OR_CORRPOS_NOTALLOWED

Bit	Value	Meaning
9	0 (default)	The active axis-specific frame and/or tool length compensation which is active in the part program at the time of execution takes effect in the synchronized action which is executed parallel to the part program.
	1	Axis-specific frames and tool length compensation are not considered for command axes.

Example 1: Traversing with **active** frames / tool length compensations (bit 9 == 0):

Program code	Comment
N100 TRANS X20	; Work offset in X: 20 mm.
; Synchronized action: The X axis traverses to position 60 mm	
IDS=1 EVERY G710 \$A_IN==1 DO POS[X]=40	
...	
; Work offset in X: -10 mm. =>	
; Synchronized action: The X axis now traverses to position 30 mm	
N130 TRANS X-10	
...	

Example 2: Traversing with **deactivated** frames / tool length compensations (bit 9 == 1):

Program code	Comment
N100 TRANS X=0.001	; Work offset in X: 0.001 degrees
N120 POS[X]=270	; X traverses to position 270.001 degrees
...	
; With \$A_IN=1, X traverses to position 180.000 degrees.	
IDS=1 EVERY G710 \$A_IN==1 DO POS[X]=180	
...	
; X traverses to position 90.001 degrees	
N130 POS[X]=90	
...	
; Coded position 1 = 100 degrees => X traverses to 100.001 degrees	
N140 POS[X]=CAC(1)	
...	
; Coded position 2 = 200 degrees => X traverses to 200.000 degrees	
N150 POS[X]=CIC(1)	

Note

If a command axis travels to indexing positions incrementally, the axis-specific frames have **no** effect on this command axis.

Non-modal suppression of the active frame with G153

If MD32074 is set so that active axis-specific frames and tool offsets always have to be taken into account for command axes (bit 9 == 0), the active frame can be suppressed in a non-modal synchronized action, if necessary, with G153. For this purpose, the block must be

turned into an executable block with the G153 command via program code G4 F0.1. In this case, it must be ensured that the dwell time is at least 0.1. This is the only way to ensure that the processing time in the interpolator is sufficient.

Examples:

```
...
WHEN TRUE DO POS[Z]=401 FA[Z]=$MA_MAX_AX_VELO[Z]
G153 G4 F0.1
...
```

```
...
WHILE $AA_IM[Z]<400
ENDWHILE
WHEN TRUE DO POS[Z]=400
G153 G4 F0.1
...
```

Takeover of the control of a command axis by the PLC

The control of a command axis that has been started via a static synchronized action (IDS) is taken over by the PLC irrespective of the status of the part program containing the synchronized action:

<Axis>.plcCtrl.out.requestPlcCtrl (request control by the PLC for autonomous single-axis operations)

References

Function Manual Axes and Spindles

Parameterizable axis status

The behavior with regard to the axis status after the end of the part program and NC Reset can be parameterized via the following machine data:

MD30450 \$MA_IS_CONCURRENT_POS_AX[<axis>] = <value>

<value>	Axis status before PP end / NC RESET ¹⁾	Axis status after PP end / NC RESET ¹⁾
0	Channel axis	Channel axis
0	Command axis	Channel axis
1	Channel axis	Command axis
1	Command axis	Command axis

¹⁾ PP end: Part program end

PLC signals

Basic Program Plus	Basic Program	
<Axis>.plcCtrl.out.requestPlcCtrl	LBP_Axis*.A_PLCAxis	DB31,DBX28.7

See also

Technology cycles (Page 120)

4.7.9 Setting the measuring system (G70, G71, G700, G710)

If a specific system of units (inch/metric) is not explicitly defined in a synchronized action with G70, G71, G700, G710, the measuring system active in the part program at the time the synchronized action is executed takes effect:

- G70/G71 active in the part program:
 - All the **programmed** position values are interpreted in the **programmed** system of units.
 - All the **read** position data is interpreted in the **parameterized basic system**.
- G700/G710 active in the part program:
 - All the **programmed** position values are interpreted in the **programmed** system of units.
 - All the **read** position data is interpreted in the **parameterized basic system**.

The following rules apply when defining the system of units in the synchronized action:

- If a system of units is programmed in the condition part, this also takes effect in the action part if a measuring system has not been specifically programmed there.
- If there is only a system of units programmed in the action part, the system which is currently activated in the part program takes effect in the condition part.
- Different systems of units can be programmed in the condition and action parts.
- The system of units programmed in the synchronized action has no effect on the part program.

Example

Program code	Comment		
N10 ID=1 EVERY \$AA_IM[Z]>200 DO POS[Z2]=10	;	\$AA_IM:	#
	;	200:	#
	;	10:	#
N20 ID=2 EVERY \$AA_IM[Z]>200 DO G70 POS[Z2]=10	;	\$AA_IM:	#
	;	200:	#
	;	10:	inch
N30 ID=3 EVERY G71 \$AA_IM[Z]>200 DO POS[Z2]=10	;	\$AA_IM:	#
	;	200:	mm
	;	10:	mm
N40 ID=4 EVERY G71 \$AA_IM[Z]>200 DO G70 POS[Z2]=10	;	\$AA_IM:	#
	;	200:	mm
	;	10:	inch

Program code	Comment		
N50 ID=5 EVERY \$AA_IM[Z]>200 DO G700 POS[Z2]=10	;	\$AA_IM:	#
	;	200:	#
	;	10:	inch
N60 ID=6 EVERY G710 \$AA_IM[Z]>200 DO POS[Z2]=10	;	\$AA_IM:	mm
	;	200:	mm
	;	10:	mm
N70 ID=7 EVERY G710 \$AA_IM[Z]>200 DO G700 POS[Z2]=10	;	\$AA_IM:	mm
	;	200:	mm
	;	10:	inch
#: The unit depends on the parameterized basic system (MD10240 \$MN_SCALING_SYSTEM_IS_METRIC) and the system of units programmed in the part program			

Note**System of units and technology cycles**

If a technology cycle is being used, the system of units can also be programmed in the technology cycle instead of the system of units having to be assigned in the action part of the synchronized action.

4.7.10 Position in specified reference range (POSRANGE)

Function

The POSRANGE function can be used to determine whether the current position of an axis is within the tolerance range around a specified reference position.

Note

With modulo axes, the modulo offset is taken into account.

Syntax

```
<Status> POSRANGE(<axis>, <RefPos>, <tolerance>, [<CoordSys>] )
```

Meaning

<status>	Function return value Type: BOOL TRUE: The current position of the axis is within the tolerance range. FALSE: The current position of the axis is not within the tolerance range.
<axis>	Name of the channel axis Type: AXIS
<RefPos>	Reference position Type: REAL
<Tolerance>	Permissible tolerance around the reference position Type: REAL The tolerance is specified as an absolute value. The tolerance range results from: Reference position +/- tolerance
<CoordSys>	Optional: Coordinate system Type: INT Range of values: 0 = MCS (machine coordinate system) 1 = BCS (basic coordinate system) 2 = SZS (settable zero system) 3 = WCS (workpiece coordinate system)

4.7.11 Traversing axes, endless (MOV)**Function**

An axis can be traversed endlessly, i.e. without specifying an end position, in a specific direction via the **MOV** command. The axis traverses so long in the specified direction until it is stopped or another traversing direction is specified by a **MOV** command.

Application example: Endlessly rotating rotary axes

Syntax

```
MOV[<axis>] = <direction>
```

Meaning

MOV	Traversing command for a command axis
<axis>	Channel axis name Type: AXIS

<Direction>	Traversing direction
	Type: INT
	Range of values:
	> 0: Positive traversing direction (default: +1)
	< 0: Negative traversing direction (default: -1)
	= 0: Stop

Note**Indexing axis**

If an indexing axis is stopped with `MOV[<indexing axis>] = 0`, it stops at the next indexing position.

Technology cycle

The `MOV` command must **not** be used in technology cycles.

See also

Axial feedrate (FA) (Page 89)

4.7.12 Axial feedrate (FA)

An axial feedrate can be specified in a synchronized action via the `FA` command. The axial feedrate is modal.

Examples

Constant feedrate value:

Program code

```
ID=1 EVERY $AA_IM[B] > 75 DO POS[U]=100 FA[U]=990
```

Variable feedrate value:

Program code

```
ID=1 EVERY $AA_IM[B] > 75 DO POS[U]=100 FA[U]=$AA_VACTM[W]+100
IDS=2 WHENEVER $A_IN[1] == 1 DO POS[X]=100 FA[X]=$R1
```

Remarks

- The default value for the feedrate of positioning axes is set via axial machine data: MD32060 \$MA_POS_AX_VELO (initial setting for positioning axis velocity)
- The axial feedrate can be specified as a linear or revolutionary feedrate.
The feedrate type can be set via the setting data: SD43300 \$SA_ASSIGN_FEED_PER_REV_SOURCE (revolutional feedrate for positioning axes/spindles)
- The feedrate type can be switched synchronous to the part program via the `FPRAON` and `FPRAOF` commands. See also:
Further information
Function Manual Axes and Spindles

Note

So that technology cycles executed in parallel do not obstruct each other, the axial feedrate from synchronized actions is not output as an auxiliary function to the NC/PLC interface.

See also

Traversing axes, endless (MOV) (Page 88)

4.7.13 Axis replacement (GET, RELEASE, AXTOCHAN)

Command axes can be interchanged between channels via the `GET`, `RELEASE` and `AXTOCHAN` commands.

Requirement

The command axis that is interchanged between the channels must be known and parameterized as channel axis in the respective channel.

Programming**Syntax**

```
GET(<axis 1> [{, <axis n>}])
RELEASE((<axis 1> [{, <axis n> }])
AXTOCHAN(<axis 1>, <channel number 1> [{, <axis n>, <channel number n> }])
```

Meaning

<code>GET:</code>	Request to replace an axis in own channel
<code>RELEASE:</code>	Release of an axis for an axis interchange
<code>AXTOCHAN:</code>	Request for an axis for replacement in the specified channel
<code><Axis n>:</code>	Machine axis name
Type:	AXIS
Value range:	Machine axis names defined in the channel

<Channel number n>:	Channel number
Type:	INTEGER
Value range:	1 ... maximum channel number

Axis type and axis status regarding axis interchange

The axis type and axis status currently valid at the time of the synchronized action activation can be queried via the \$AA_AXCHANGE_TYP or \$AA_AXCHANGE_STAT system variable. Depending on the channel that has the current interpolation authorization for this axis and depending on the status for the permissible axis interchange, a different sequence results from the synchronized action.

An axis can be requested with `GET` from a synchronized action, if

- Another channel has the write or interpolation authorization for the axis
- The requested axis is already assigned to the requested channel
- The axis in the neutral axis state is controlled by the PLC
- The axis is a command axis, oscillating axis, or concurrent PLC axis
- The axis is already assigned to the NC program of the channel

Note

Constraint: An "axis controlled exclusively by the PLC" or a "permanently assigned PLC axis" cannot be assigned to the NC program.

An axis can be released from a synchronized action with `RELEASE`, if the axis:

- Was previously assigned to the NC program of the channel
- Is already in the neutral axis state
- Already has another channel that has the interpolation authorization of this axis

Request axis from another channel

If, when the `GET` action is activated, **another channel** has the interpolation authorization for the axis `$AA_AXCHANGE_TYP[axis] == 2`, axis interchange is used to fetch the axis from this channel `$AA_AXCHANGE_TYP[axis] == 6` and assign it to the requesting channel as soon as possible. The axis then becomes the **neutral axis** `$AA_AXCHANGE_TYP[axis] == 3`.

The status change to a neutral axis does **not** result in reorganization in the requesting channel.

Requested axis was already requested as neutral axis:

4.7 Actions in synchronized actions

\$AA_AXCHANGE_TYP[<axis>]==6, the axis is required for the NC program
\$AA_AXCHANGE_TYP[axis] == 5 and assigned as soon as possible to the NC program of
the channel \$AA_AXCHANGE_TYP[axis] == 0.

Note

This assignment **results in a** reorganization.

Axis is already assigned to the requested channel

If the requested axis has already been assigned **to this channel** at the point of activation, and its status is that of a neutral axis not controlled by the PLC \$AA_AXCHANGE_TYP[axis]==3, it is assigned to the NC program \$AA_AXCHANGE_TYP[axis]==0.

This **results in a** reorganization procedure.

Axis in the state of the neutral axis is controlled from the PLC

If the axis is in neutral axis status controlled **by the PLC** \$AA_AXCHANGE_TYP[axis]==4, the axis is requested as a neutral axis \$AA_AXCHANGE_TYP[axis] == 8. This disables the axis for automatic axis interchange between channels (Bit 0 == 0) in accordance with the value of bit 0 in machine data:

MD10722 \$MN_AXCHANGE_MASK (parameterization of the axis interchange behavior)

This corresponds to \$AA_AXCHANGE_STAT[axis] == 1.

Axis is active as command axis / assigned to the PLC

If the axis is active as a command axis or oscillating axis or a concurrent positioning axis (PLC axis) (\$AA_AXCHANGE_TYP[<axis>] == 1), the axis is requested as a neutral axis (\$AA_AXCHANGE_TYP[<axis>] == 8). Depending on the setting in the following machine data, the axis is blocked for an automatic axis interchange between channels:

MD10722 \$MN_AXCHANGE_MASK (parameterization of the axis interchange behavior)

This corresponds to \$AA_AXCHANGE_STAT[<axis>] == 1.

With a further GET request, the axis is then requested for the NC program ⇒
\$AA_AXCHANGE_TYP[axis] == 7.

Axis already assigned to the NC program of the channel

If the axis is already assigned to the NC program of the channel (\$AA_AXCHANGE_TYP[<axis>] == 0) or if this assignment is requested, e.g. axis interchange triggered by the NC program (\$AA_AXCHANGE_TYP[<axis>] == 5 or \$AA_AXCHANGE_TYP[<axis>] == 7), there is **no** state change.

Release axis for axis interchange

If the axis is assigned to the NC program at the time of release ($\$AA_AXCHANGE_TYP[<axis>] == 0$), it is transferred to the neutral axis state ($\$AA_AXCHANGE_TYP[<axis>] == 3$) and if required, released for axis interchange in another channel.

This **results in a** reorganization procedure.

Axis to be released is already a neutral axis:

If the axis is already in the neutral axis state ($\$AA_AXCHANGE_TYP[<axis>] == 3$) or active as command or oscillating axis or assigned to the PLC as concurrent positioning axis ($\$AA_AXCHANGE_TYP[<axis>] == 1$), the axis is released for an automatic axis interchange between channels.

$\$AA_AXCHANGE_STAT[<axis>]$ is reset from 1 to 0 if there is no other reason to link the axis to the channel. Such a link of the axis is present, for example, with:

- Active axis coupling
- Active fast retraction
- Active transformation
- JOG request
- Rotating frame with PLC, command or oscillating axis motion

Another channel already has the interpolation authorization

If another channel already has the interpolation authorization ($\$AA_AXCHANGE_TYP[<axis>] == 2$), there is no state change. This also means that waiting for an axis, triggered by NC program ($\$AA_AXCHANGE_TYP[<axis>] == 5$) or a previous **GET** request from a synchronized action ($\$AA_AXCHANGE_TYP[<axis>] == 6$) cannot be aborted by a **RELEASE** from a synchronized action.

Constraints

- If several **GET** requests and **RELEASE** enables are programmed for the same axis in the action part of a synchronized action or in a block of a technology cycle, they may mutually cancel each other under certain circumstances and only the last respective task is performed.

Example:

Programming: `GET (X, Y)   RELEASE (Y, Z)   GET (Z)`

Version: `GET(X) RELEASE(Y) GET(Z)`

- If further commands are programmed in the action part of a synchronized action in addition to **GET** / **RELEASE**, there is no waiting period until the **GET** / **RELEASE** request is completed before these commands are executed. This can lead to an error if, for example, an axis requested for the positioning motion with **GET** is not yet available:

`GET[<axis>]   POS[<axis>]`

Example 1: **GET** and **RELEASE** as action in synchronized actions in two channels

Requirement: The Z axis must be channel axis in the 1st and 2nd channel

4.7 Actions in synchronized actions

1. Program sequence in the 1st channel:

Program code	Comment
WHEN TRUE DO RELEASE(Z)	; Z axis becomes neutral
; Read-in disable as long as Z axis is program axis	
WHENEVER \$AA_TYP[Z] == 1 DO RDISABLE	
N110 G4 F0.1	
...	
; Z axis become NC program axis again	
WHEN TRUE DO GET(Z)	
; Read-in disable until Z axis is program axis	
WHENEVER \$AA_TYP[Z] <> 1 DO RDISABLE	
N120 G4 F0.1	
...	
WHEN TRUE DO RELEASE(Z)	; Z axis becomes neutral
; Read-in disable as long as Z axis is program axis	
WHENEVER \$AA_TYP[Z] == 1 DO RDISABLE	
N130 G4 F0.1	
...	
N140 START(2)	; Start 2nd channel
N150 ; See below: "3. Continuation: Program sequence in the 1st channel"	

2. Program sequence in the 2nd channel:

Program code	Comment
WHEN TRUE DO GET(Z)	; Move Z axis to 2nd channel (neutral)
; Read-in disable as long as Z axis is in other channel	
WHENEVER \$AA_TYP[Z] == 0 DO RDISABLE	
N210 G4 F0.1	
...	
WHEN TRUE DO GET(Z)	; Z axis becomes NC program axis
; Read-in disable until Z axis is program axis	
WHENEVER \$AA_TYP[Z] <> 1 DO RDISABLE	
N220 G4 F0.1	
...	
WHEN TRUE DO RELEASE(Z)	; Z axis in 2nd channel neutral
; Read-in disable as long as Z axis is program axis	
WHENEVER \$AA_TYP[Z] == 1 DO RDISABLE	
N230 G4 F0.1	
...	
N250 WAITM(10,1,2)	; Synchronize with channel 1
N999 M30	

3. Continuation: Program sequence in the 1st channel:

Program code	Comment
N150 WAITM(10,1,2)	; Synchronize with channel 2

Program code	Comment
...	
WHEN TRUE DO GET(Z)	; Move Z axis to this channel
; Read-in disable as long as Z axis is in other channel	
WHENEVER \$AA_TYP[Z] == 0 DO RDISABLE	
N160 G4 F0.1	
...	
N199 WAITE(2)	; Wait for end of program in Channel 2
N999 M30	

Transfer axis to another channel (AXTOCHAN)

An axis can be requested for an arbitrary channel from a synchronized action with the AXTOCHAN command.

If the axis is already assigned to the NC program of the channel (\$AA_AXCHANGE_TYP[<axis>] == 0), there is **no** state change.

If an axis is requested for the same channel from a synchronized action, AXTOCHAN is mapped on the GET command.

- With the **first** request for the same channel, the axis becomes a neutral axis.
- With the **second** request, the axis is assigned to the NC program.

Constraint

A "PLC-controlled axis" corresponds to a "concurrent positioning axis" where special constraints must be carefully observed. For further details, see:

More information

Function Manual Axes and Spindles

Note

A PLC axis cannot replace the channel.

An axis controlled exclusively by the PLC cannot be assigned to the NC program.

4.7.14 Traversing spindles (M, S, SPOS)

Spindles can be started, positioned and stopped via synchronized actions. The programming is performed in the action part of the synchronized action with the same syntax as in the part program. Without numeric extension the commands for the master spindle apply. By specifying a numeric extension, it is possible to program each spindle individually:

Program code	Comment
ID = 1 EVERY \$A_IN[1]==1 DO M3 S1000	;Master spindle
ID = 2 EVERY \$A_IN[2]==1 DO SPOS=270	;Master spindle
ID = 1 EVERY \$A_IN[1]==1 DO M1=3 S1=1000 SPOS[2]=90	

4.7 Actions in synchronized actions

If concurrent commands are specified for a spindle through synchronized actions that are active in parallel, the chronological sequence decides the activation.

User-specific spindle enable

The start of spindle motions at defined times can be achieved via synchronized actions by blocking the motion programmed in the part program.

Example:

The spindle is programmed within a part program and should not start at the beginning of the block, but only when input 1 is set. The synchronized action holds the spindle override at 0% until the enable via input 1. See Chapter "Override (\$A...OVR) (Page 40)".

Program code

```
; As long as input 1 is not set => spindle override = 0%
ID=1 WHENEVER $A_IN[1]==0 DO $AA_OVR[S1]=0
...
; The start of the spindle is triggered
; The spindle is enabled when input 1 is set
G01 X100 F1000 M3 S1=1000
```

Transition between command axis and spindle

Since several synchronized actions can be active simultaneously, the situation may arise where a spindle motion is started when the spindle is already active. In this case, the most recently activated motion is applicable. At a reversal in the direction of motion, the spindle is first braked and then traversed in the opposite direction.

Direction of rotation, speed and position can also be changed during the motion.

Examples

Program code	Comment
ID=1 EVERY \$AC_TIMER[1] >= 5 DO M3 S300	;Direction of rotation and speed
ID=2 EVERY \$AC_TIMER[1] >= 7 DO M4 S500	;Direction of rotation and speed
ID=3 EVERY \$A_IN[1]==1 DO S1000	;Speed
ID=4 EVERY (\$A_IN[4]==1) AND (\$A_IN[1]==0) DO SPOS=0	;Spindle positioning

Transitions between axis and spindle

In state ↓	To →	POS	MOV<>0	MOV=0	SPOS	M3/M4	M5	LEADON	TRAIL ON
during traversing									
Axis		x	x	x	x	x	x	x	x
Position-controlled spindle		x	x	x	x	x	x	-	-
Speed-controlled spindle		-	-	-	x	x	x	-	-
In motion									

In state ↓	To →	POS	MOV<>0	MOV=0	SPOS	M3/M4	M5	LEADON	TRAIL ON
Axis		x	x	x	-	-	-	x	x
Position-controlled spindle		-	-	-	-	-	-	-	-
Speed-controlled spindle		-	-	-	x	x	x	-	-
Transitions marked with x are permitted: The transitions marked with - are rejected with an alarm.									

4.7.15 Actual value setting with loss of the referencing status (PRESETON)

Function

The `PRESETON()` procedure sets new actual values in the machine coordinate system (MCS) from synchronized actions for **one** axis. This corresponds to work offset of the axis MCS. The axis is not traversed.

From synchronized actions, `PRESETON` **must only be used on command axes**, i.e. on axes that have been started from a synchronized action. The axis must also be assigned to the channel, i.e. this channel must have the interpolation right for this axis. The axis is **not** requested from another channel via axis interchange.

Referencing status

By setting a new actual value in the machine coordinate system, the referencing status of the machine axis is reset:

```
<Axis>.basic.in.enc1Synchronized / <Axis>.basic.in.enc2Synchronized = 0
```

(referenced/synchronized measuring system 1/2)

It is recommended that `PRESETON` only be used for axes that do not require a reference point.

To restore the original machine coordinate system, the measuring system of the machine axis must be referenced again, e.g. through active reference point approach from the part program (G74).



CAUTION

Loss of the referencing status

By setting a new actual value in the machine coordinate system with `PRESETON`, the referencing status of the machine axis is reset to "not referenced/synchronized".

Programming

Syntax

```
WHEN | EVERY ... DO PRESETON(<axis>, <value>)
```

Meaning

WHEN, EVERY:	Only WHEN and EVERY must be used as frequency (Page 21).
PRESETON:	Actual value setting with loss of the referencing status
<Axis>:	Machine axis name
	Type: AXIS
	Value range: Machine axis names defined in the channel
<Value>:	New actual value of the machine axis in the machine coordinate system (MCS)
	The input is made in the currently valid system of units (inch/metric)
	An active diameter programming (DIAMON) is taken into account
	Type: REAL

System variable**\$AC_PRESET**

The axis-specific system variable \$AC_PRESET provides the vector from the zero point of the currently offset MCS' to the zero point of the original MCS₀ after the referencing of the machine axis.

$\$AC_PRESET<axis> = \$AC_PRESET<axis> + \text{"current actual position of the axis in the MCS"} - \text{"PRESETON actual position"}$

The work offset can be undone with the system variables:

`PRESETON(<axis>, $VA_IM + $AC_PRESET[<axis>]) ; "current actual position of the axis in the MCS'" + "offsets"`

Example**Program code**

```

N10 G1 X=10 F5000
; Traverse the X axis as command axis to position 200
N20 WHEN TRUE DO G71 POS[X]=200
; IF set position of the X axis in the MCS ($AA_IM[X]) >= 80
; THEN "actual position of the X axis in the MCS" = "set position of the X
axis in the MCS" + "offset"
;                                     = 80 + 70 = 150
;   "progr. end position of the X axis" = "progr. end position of the X
axis" + "offset"
;                                     = 200 + 70 = 270
;   $AC_PRESET = $AC_PRESET - 70
N30 WHEN G71 $AA_IM[X] >= 80 DO PRESETON(X, $AA_IM[X]+70)
N40 G4 F3

```

Constraints

Axes for which **PRESETON** must not be used

- Traversing command axes in spindle mode
- Traversing concurrent positioning axes (`SINU_TraversePosAxis` (Basic Program FC18))
- Axes involved in a transformation
- Traversing path axes
- Oscillating axes
- Axes on which one or more of the following safety functions (Safety Integrated) are active
 - Enable "Safe software limit switch"
MD36901 \$MA_SAFE_FUNCTION_ENABLE[<safe axis>], bit 1 = 1
 - Enable "Safe software cam", pair 1 ... 4, cam +/-
MD36901 \$MA_SAFE_FUNCTION_ENABLE[<safe axis>], bits 8 ... 15 = 1
or
Enable "Safe Cam Track", cam 1 ... 30
MD36903 \$MA_SAFE_CAM_ENABLE[<safe axis>], bits 0 ... 29 = 1
- Hirth axes
- Synchronous axes of a gantry grouping
- Axes for which the reference point approach from the part program (G74) is active
- Sub-axis of a speed/torque coupling (main/sub-coupling)

4.7 Actions in synchronized actions

Geometry axes

- PRESETON can be used on a stationary geometry axis when a further geometry axis is not being traversed in the channel at the same time.
- PRESETON can be used on a stationary geometry axis even when a further geometry axis is being traversed in the channel at the same time, but this axis is in the "neutral axis" state or traversing as a command axis.

Example: A further geometry (X) is traversing at the same time in the "**neutral axis**" state

Program code	Comment
N10 G0 X0 Y0	; X, Y: Geometry axes
N15 RELEASE(Y) ¹⁾	; Neutral axis
N20 ID=1 WHEN 20.0 < \$AA_IM[X] DO PRESETON(Y,20)	; \$AA_IM: Set position in the MCS
N30 G0 X40	; Geometry axis X traverses
N40 M30	
1) Note The release of an axis in the action part of a synchronized action does not ensure that the release is on time. N20 ID=1 WHEN 20.0 < \$AA_IM[X] DO RELEASE(Y) PRESETON(Y,20) ; NOT recommended!	

Example: A further geometry (X) is traversing at the same time as **command axis**

Program code	Comment
N10 G0 X0 Y0	; X, Y: Geometry axes
N20 ID=1 WHEN TRUE DO POS[X]=40 FA[X]=1000	; X command axis
N30 ID=2 WHEN 20.0 < \$AA_IM[X] DO PRESETON(Y,20)	; \$AA_IM: Set position in the MCS
N40 M30	

PLC-controlled axes

- PRESETON can be used on PLC-controlled axes according to their current type.

Spindle states

The following table shows the reactions that occur when PRESETON is used on a spindle in a synchronized action:

PRESETON in synchronized action			
Spindle mode	Traversing status	Assigned to the NC program	Main axis
Speed control mode	In motion	Alarm 17601	Alarm 17601
	Stationary	+	+
Positioning mode SPOS	In motion	Alarm 17601	Alarm 17601
	Stationary	+	+
Positioning across block boundaries SPOSA	In motion	Alarm 17601	-

PRESETON in synchronized action			
Spindle mode	Traversing status	Assigned to the NC program	Main axis
Axis mode	In motion	Alarm 17601	+
	Stationary	+	+
+: Possible -: Not possible			

PRESETON in the NC program			
Spindle mode	Traversing status	Assigned to the NC program	Main axis
Speed control mode	In motion	Alarm 22324	Alarm 22324
	Stationary	+	+
Positioning mode SPOS	In motion	-	+
	Stationary	+	+
Positioning across block boundaries SPOSA	In motion	Alarm 10610	-
Axis mode	In motion	-	+
	Stationary	+	+
+: Possible -: Not possible			

Axis couplings

- Leading axes: The sudden change of the leading axis position caused by PRESETON is not traversed in the following axes. The coupling is not changed.
- Following axes: Only the overlaid position component of the following axis is affected by PRESETON.

Gantry grouping

- If PRESETON is used on the guide axis of a gantry grouping, the work offset is also performed in all synchronous axes of the gantry grouping.

Indexing axes

- PRESETON can be used on indexing axes.

Software limit switches, operating range limit, protection areas

- If the axis position is outside the specified limits after a work offset by PRESETON, an alarm is not displayed until an attempt is made to traverse the axis.

Block search with calculation

PRESETON commands are collected during the block search and executed with the NC start to continue the NC program.

Position-dependent NC/PLC interface signals

- The status of the position-dependent NC/PLC interface signals is redetermined based on the new actual position.

Example: Fixed point positions

- Parameterized fixed point positions: MD30600 \$MA_FIX_POINT_POS[0...3] = <fixed point position 1...4>
- NC/PLC interface signals: <Axis>.basic.in.jogFixedPointReachedNumber (approach fixed point in JOG: reached)

If the axis is at a fixed point position with the exact stop tolerance, the associated NC/PLC interface signal is set. The NC/PLC interface signal is reset when the actual value is set by PRESETON to a different value outside the exact stop tolerance around the fixed point position.

DRF offset

- A DRF offset of the axis is deleted by PRESETON.

Overlaid movement \$AA_OFF

- An overlaid movement (\$AA_OFF) (Page 49) is not affected by PRESETON.

Online tool offset FTOC

- An active online tool offset (FTOC) (Page 77) remains active even after PRESETON.

Axis-specific compensations

Axis-specific compensations remain active after PRESETON.

JOG mode

- PRESETON must only be used on a stationary axis.

JOG mode, REF machine function

- PRESETON must **not** be used.

PLC signals**NC → PLC**

Basic Program Plus	Basic Program	
<Axis>.basic.in.enc1Synchronized	LBP_Axis*.E_RefSyn1	DB31,DBX60.4
<Axis>.basic.in.enc2Synchronized	LBP_AxisX*.E_RefSyn2	DB31,DBX60.5
<Axis>.basic.in.jogFixedPointReachedNumber	LBP_Axis*.E_JogFixPPos0 ... 2	DB31,DBX75.3..5

See also

On-the-fly parting (Page 149)

4.7.16 Actual value setting without loss of the referencing status (PRESETONS)

Function

The `PRESETONS ( )` procedure sets new actual values in the machine coordinate system (MCS) from synchronized actions for **one** axis. This corresponds to work offset of the axis MCS. The axis is not traversed.

From synchronized actions, `PRESETONS` **must only be used on command axes**, i.e. on axes that have been started from a synchronized action. The axis must also be assigned to the channel, i.e. this channel must have the interpolation right for this axis. The axis is **not** requested from another channel via axis interchange.

Referencing status

By setting a new actual value in the machine coordinate system (MCS) with `PRESETONS`, the referencing status of the machine axis is **not** changed.

Requirements

- **Encoder type**

`PRESETONS` is only possible with the following encoder types of the active measuring system:

- MD30240 \$MA_ENC_TYPE[<measuring system>] = 0 (simulated encoder)
- MD30240 \$MA_ENC_TYPE[<measuring system>] = 1 (raw signal encoder)

- **Referencing mode**

`PRESETONS` is only possible with the following referencing mode of the active measuring system:

- MD34200 \$MA_ENC_REFP_MODE[<measuring system>] = 0 (no reference point approach possible)
- MD34200 \$MA_ENC_REFP_MODE[<measuring system>] = 1 (referencing of incremental, rotary or linear measuring systems: zero pulse on the encoder track)

Startup

Axis-specific machine data

Actual value setting without loss of the referencing status (`PRESETONS`) must be set axis-specifically:

MD30460 \$MA_BASE_FUNCTION_MASK, bit 9 = 1

Note

PRESETON deactivated

Activation of the "Actual value setting without loss of the referencing status `PRESETONS`" function deactivates the "Actual value setting with loss of the referencing status `PRESETON`" function. The options mutually exclude each other.

Programming

Syntax

<frequency> ... DO PRESETONS(<axis>, <value>)

Meaning

<frequency>: Only WHEN and EVERY must be used as frequency (Page 21).

PRESETONS: Actual value setting without loss of the referencing status

<Axis>: Machine axis name
 Type: AXIS
 Value range: Machine axis names defined in the channel

<Value>: New current actual value of the machine axis in the machine coordinate system (MCS)
 The input is made in the active system of units (inch/metric)
 An active diameter programming (DIAMON) is taken into account
 Type: REAL

System variable

\$AC_PRESET

The axis-specific system variable \$AC_PRESET provides the vector from the zero point of the currently offset MCS' to the zero point of the original MCS₀ after the referencing of the machine axis.

$\$AC_PRESET\langle axis \rangle = \$AC_PRESET\langle axis \rangle + \text{"current actual position of the axis in the MCS"} - \text{"PRESETONS actual position"}$

The work offset can be undone with the system variables:

```
PRESETONS(<axis>, $VA_IM + $AC_PRESET[<axis>]) ; "current actual position of the axis in the MCS'" + "offsets"
```

Example

Work offset of the X axis MCS by 70 units.

The programmed end position of the X axis (command axis) is transformed to the new MCS with PRESETONS.

Program code

```
N10 G1 X=10 F5000
; Traverse the X axis as command axis to position 200
N20 WHEN TRUE DO G71 POS[X]=200
```


Program code

```

; IF set position of the X axis in the MCS ($AA_IM[X]) >= 80
; THEN "actual position of the X axis in the MCS" = "set position of the X
axis in the MCS" + "offset"
;                                     = 80 + 70 = 150
;     "progr. end position of the X axis" = "progr. end position of the X
axis" + "offset"
;                                     = 200 + 70 = 270
;     $AC_PRESET = $AC_PRESET - 70
N30 WHEN G71 $AA_IM[X] >= 80 DO PRESETONS(X, $AA_IM[X]+70)
N40 G4 F3

```

Constraints**Axes for which PRESETONS must not be used**

- Traversing command axes in spindle mode
- Traversing concurrent positioning axes (SINU_TraversePosAxis (Basic Program FC18))
- Axes involved in a transformation
- Traversing path axes
- Oscillating axes
- Axes on which one or more of the following safety functions (Safety Integrated) are active
 - Enable "Safe software limit switch"
MD36901 \$MA_SAFE_FUNCTION_ENABLE[<safe axis>], bit 1 = 1
 - Enable "Safe software cam", pair 1 ... 4, cam +/-
MD36901 \$MA_SAFE_FUNCTION_ENABLE[<safe axis>], bits 8 ... 15 = 1
or
Enable "Safe Cam Track", cam 1 ... 30
MD36903 \$MA_SAFE_CAM_ENABLE[<safe axis>], bits 0 ... 29 = 1
- Hirth axes
- Synchronous axes of a gantry grouping
- Axes for which the reference point approach from the part program (G74) is active
- Sub-axis of a speed/torque coupling (main/sub-coupling)

4.7 Actions in synchronized actions

Geometry axes

- PRESETONS can be used on a stationary geometry axis when a further geometry axis is not being traversed in the channel at the same time.
- PRESETONS can be used on a stationary geometry axis even when a further geometry axis is being traversed in the channel at the same time, but this axis is in the "neutral axis" state or traversing as a command axis.

Example: A further geometry (X) is traversing at the same time in the "**neutral axis**" state

Program code	Comment
N10 G0 X0 Y0	; X, Y: Geometry axes
N15 RELEASE(Y) ¹⁾	; Neutral axis
N20 ID=1 WHEN 20.0 < \$AA_IM[X] DO PRESETONS(Y,20)	; \$AA_IM: Set position in the MCS
N30 G0 X40	; Geometry axis X traverses
N40 M30	
1) Note The release of an axis in the action part of a synchronized action does not ensure that the release is on time. N20 ID=1 WHEN 20.0 < \$AA_IM[X] DO RELEASE(Y) PRESETONS(Y,20) ; NOT recommended!	

Example: A further geometry (X) is traversing at the same time as **command axis**

Program code	Comment
N10 G0 X0 Y0	; X, Y: Geometry axes
N20 ID=1 WHEN TRUE DO POS[X]=40 FA[X]=1000	; X command axis
N30 ID=2 WHEN 20.0 < \$AA_IM[X] DO PRESETONS(Y,20)	; \$AA_IM: Set position in the MCS
N40 M30	

PLC-controlled axes

- PRESETONS can be used on PLC-controlled axes according to their current type.

Spindle states

The following table shows the reactions that occur when PRESETONS is used on a spindle in a synchronized action:

PRESETONS in synchronized action			
Spindle mode	Traversing status	Assigned to the NC program	Main axis
Speed control mode	In motion	Alarm 17601	Alarm 17601
	Stationary	+	+
Positioning mode SPOS	In motion	Alarm 17601	Alarm 17601
	Stationary	+	+
Positioning across block boundaries SPOSA	In motion	Alarm 17601	-

PRESETONS in synchronized action			
Spindle mode	Traversing status	Assigned to the NC program	Main axis
Axis mode	In motion	Alarm 17601	+
	Stationary	+	+
+: Possible -: Not possible			

PRESETONS in the NC program			
Spindle mode	Traversing status	Assigned to the NC program	Main axis
Speed control mode	In motion	Alarm 22324	Alarm 22324
	Stationary	+	+
Positioning mode SPOS	In motion	-	+
	Stationary	+	+
Positioning across block boundaries SPOSA	In motion	Alarm 10610	-
Axis mode	In motion	-	+
	Stationary	+	+
+: Possible -: Not possible			

Axis couplings

- Leading axes: The sudden change of the leading axis position caused by PRESETONS is not traversed in the following axes. The coupling is not changed.
- Following axes: Only the overlaid position component of the following axis is affected by PRESETONS.

Gantry grouping

- If PRESETONS is used on the guide axis of a gantry grouping, the work offset is also performed in all synchronous axes of the gantry grouping.

Indexing axes

- PRESETONS can be used on indexing axes.

Software limit switches, operating range limit, protection areas

- If the axis position is outside the specified limits after a work offset by PRESETONS, an alarm is not displayed until an attempt is made to traverse the axis.

Block search with calculation

PRESETONS commands are collected during the block search and executed with the NC start to continue the NC program.

Position-dependent NC/PLC interface signals

- The status of the position-dependent NC/PLC interface signals is redetermined based on the new actual position.

Example: Fixed point positions

- Parameterized fixed point positions: MD30600 \$MA_FIX_POINT_POS[0...3] = <fixed point position 1...4>
- NC/PLC interface signals: <Axis>.basic.in.jogFixedPointReachedNumber (approach fixed point in JOG: reached)

If the axis is at a fixed point position with the exact stop tolerance, the associated NC/PLC interface signal is set. The NC/PLC interface signal is reset when the actual value is set by PRESETONS to a different value outside the exact stop tolerance around the fixed point position.

DRF offset

- A DRF offset of the axis is deleted by PRESETONS.

Overlaid movement \$AA_OFF

- An overlaid movement (\$AA_OFF) (Page 49) is not affected by PRESETONS.

Online tool offset FTOC

- An active online tool offset (FTOC) (Page 77) remains active even after PRESETONS.

Axis-specific compensations

Axis-specific compensations remain active after PRESETONS.

JOG mode

- PRESETONS must only be used on a stationary axis.

JOG mode, REF machine function

- PRESETONS must **not** be used.

PLC signals

Basic Program Plus	Basic Program	
<Axis>.basic.in.jogFixedPointReachedNumber	LBP_Axis*.E_JogFixPPos0 ... 2	DB31,DBX75.3..5

4.7.17 Couplings (CP..., LEAD..., TRAIL..., CTAB...)

The commands listed in Section "Language elements for synchronized actions and technology cycles (Page 64)" can be programmed in synchronous actions for the functions coupled motion (TRAIL...), curve tables (CTAB...), master value coupling (LEAD...) and generic coupling (CP...).

Note

Generic coupling

Note that the "generic coupling" CP... commands are always executed in synchronous actions in the sequence of the programming from left to right. This means that in contrast to the programming in the part program, the effect of the various commands depends on the sequence in the synchronous action.

Curve tables

The CTAB and CTABINV commands can be used in the condition and in the action.

More information

Detailed information on coupling commands can be found in:

- Coupled motion, curve tables, master value coupling
Programming Manual NC Programming
- Generic coupling
Function Description Axes and Spindles

Coupled motion

When the coupling is activated from the synchronous action, the leading axis can be in motion. In this case the following axis is accelerated up to the set velocity. The position of the leading axis at the time of synchronization of the velocity is the starting position for coupled-axis motion.

Master value coupling

Syntax

```
... DO LEADON(<FA>, <LA>, <NO>, <OVW>)
```

Meaning

<FA>:	Name of the following axis Type: AXIS
<LA>:	Name of the leading axis Type: AXIS

4.7 Actions in synchronized actions

<NO>:	Number of the curve table Type: INT
<OVW>:	Status of the overwrite permission Type: BOOL 0: Overwriting of the table is not permitted 1: Overwriting of the table is permitted

- Synchronous actions can be used to change the basic curve table without a resynchronization even during an active master value coupling. The following axis attempts as fast as possible to follow the position values specified by the new curve table.
- In order to be able to program an axis to be coupled via synchronous actions, the axis must first be released with the `RELEASE` command.
Example:

Program code

```
...
N60 RELEASE(X)
N50 ID=1 EVERY SR1==1 DO LEADON(C, X, 1)
```

Example: On-the-fly parting

An extruded material which passes continuously through the operating area of a cutting tool must be cut into parts of equal length.

- X axis: Axis in which the extruded material moves (WCS)
- X1 axis: Machine axis of the extruded material (MCS)
- Y axis: Axis in which the cutting tool "tracks" the extruded material

It is assumed that the infeed and control of the cutting tool are controlled via the PLC user program. The signals at the PLC interface can be evaluated to determine whether the extruded material and cutting tool are synchronized.

Program code

```
N100 R3=1500      ;Length of a part to be cut off
N200 R2=100000 R13=R2/300
N300 R4=100000
N400 R6=30        ;Start position Y axis
N500 R1=1         ;Start condition for conveyor axis
N600 LEADOF(Y,X)  ;Delete coupling
N700 CTABDEF(Y,X,1,0) ;Table definition
N800 X=30 Y=30    ;Value pairs
N900 X=R13 Y=R13
N1000 X=2*R13 Y=30
N1100 CTABEND      ;End of table definition
N1200 PRESETON(X1,0) ;PRESET at beginning
N1300 Y=R6 G0      ;Start position Y axis, axis is linear
; PRESET after length R3, new start after parting
```

Program code

```

N1400 ID=1 WHENEVER $AA_IW[X]>$R3 DO PESETON(X1,0)
N1500 RELEASE(Y)
; Couple Y to X via table 1, for X < 10
N1800 ID=6 EVERY $AA_IM[X]<10 DO LEADON(Y,X,1)
; > 30 before traversed parting distance, deactivate coupling
N1900 ID=10 EVERY $AA_IM[X]>$R3-30 DO LEADOF(Y,X)
N2000 WAITP(X)
; Set extruded material axis continuously in motion
N2100 ID=7 WHEN $R1==1 DO MOV[X]=1 FA[X]=$R4
N2200 M30

```

Generic coupling

- When a coupling module is activated in a synchronous action, the following axis **must** already be active in the channel and be in the state "neutral axis" or "axis already assigned to the part program of the channel". The corresponding axis state can be generated, if necessary, in the synchronous action by programming GET[<following axis>].
- The commands of the generic coupling CP . . . are processed directly in synchronous actions by the coupling module. The command therefore takes effect immediately.
- With the programming of a coupling factor (CPLNUM, CPLDEN) or table number (CPLCTID), a previously activated non-linear coupling relationship, e.g. a curve table, is deactivated.

Generic coupling: Using the TRAIL, LEAD, EG or COUP coupling type.

If in the framework of the generic coupling, a behavior corresponding to one of the known coupling types "Coupled motion", "Master value coupling", "Electronic gear" or "Synchronous spindle" is required, the command CPSETTYPE is also possible in synchronous actions when creating or defining the coupling module:

```
CPSETTYPE[FAx] = <coupling type>
```

<Coupling type>	Meaning
CP	Freely programmable
TRAIL	"Coupled motion" coupling type
LEAD	"Master value coupling" coupling type
EG	"Electronic gearbox" coupling type
COUP	"Synchronous spindle" coupling type

Constraints**Synchronism status of a following axis**

The system variable \$AA_SYNC[<axis>] can be used to read the synchronism status of a following axis in the part program or synchronous action.

Axis replacement with cross-channel coupling

For axis replacement, the following and leading axes must be known to the calling channel. Axis replacement of leading axes can be performed independently of the state of the coupling. A defined or active coupling does not produce any other supplementary conditions.

Note

With the activation of the coupling, the following axis becomes the main run axis and is not available for an axis replacement. The following axis is thus logged out of the channel. With this type of coupling, an overlaid movement is therefore not possible.

See also Section "Axis replacement (GET, RELEASE, AXTOCHAN) (Page 90)"

Conflict prevention when changing from following axis to channel axis

In order to be able to traverse a following axis traversed via synchronous actions as a channel axis again, you must ensure that the coupling is deactivated before the channel requests the relevant axis.

The following example shows an **error case**:

Program code

```
...
N50 WHEN TRUE DO TRAILOF(Y, X)
N60 Y100
```

The Y axis is not released early enough in N50 because TRAILOF only becomes active with N60 through the non-modal synchronous action.

Corrected example:

Program code

```
...
N50 WHEN TRUE DO TRAILOF(Y, X)
N55 WAITP(Y)          ; Wait for end of travel of the positioning axis
N60 Y100
```

Examples

Define coupling: Y = following axis, X = leading axis

Program code

```
... DO CPLDEF[Y]=X CPLNUM[Y,X]=1.5
```

Activate coupling and define coupling relationship.

- N10 with the correct sequence: First CPLON then CPLNUM
- N20 with **incorrect** sequence: First CPLNUM then CPLON

Program code

```
N10 ... DO CPLON[Y]=X CPLNUM[X,Y]=1.5
```


Program code

```
N20 ... DO CPLNUM[X,Y]=2 CPLON[Y]=X ; Error
```

Activate coupling, deactivation/activation with implicit resynchronization

Program code

```
N10 ... DO CPLON[X]=Y CPLNUM[X,Y]=3
N20 Y100 F100
N30 ... DO CPLOF=X CPLON[X]=Y CPLNUM[X,Y]=3
```

Activate coupling, deactivate and traverse as a command axis

Program code

```
N10 ... DO CPLON[X]=Y CPLNUM[X,Y]=3
N20 Y100 F100
N30 ... DO CPLOF=X MOV[X]=10
```

4.7.18 Measurement (MEAWA, MEAC)

The following commands can be used in synchronized actions for measurement:

- MEAWA (measurement without delete distance-to-go)
- MEAC (continuous measurement without delete distance-to-go)

While the measuring function in the part program is limited to one motion block, the measuring function can be switched on and off any number of times from synchronized actions.

Note

Measurement can also be performed in JOG mode via static synchronized actions `IDS ...`

More information

Detailed information on measuring commands can be found in:

- Coupled motion, curve tables, master value coupling
Programming Manual NC Programming
- Generic coupling
Function Description Axes and Spindles

Measurement tasks and state changes

When a measurement task has been executed from a synchronized action, the control system responds in the following way:

State	Response
Operating mode change	A measurement task activated by a modal synchronized action is not affected by a change in operating mode. It remains active beyond block limits.
Reset	The measurement task is aborted.
Block search	Measurement tasks are collected, but not activated until the programmed condition is fulfilled.
REPOS	Activated measurement tasks are not affected.
End of program	Measurement tasks started from static synchronized actions remain active.

Remarks

System variable

The following system variables can be used in conjunction with synchronized actions:

- \$AA_MEAACT (axial measuring active)
- \$A_PROBE (probe state)
- \$AA_MM1 ... 4 (probe position 1st to 4th trigger (machine coordinate system))

The following system variable **cannot** be used in conjunction with synchronized actions:

- \$AC_MEA (probe has responded)

Measurement job

Only one measurement job at a time may be active for an axis.

Priority with more than one measurement

A new measurement task for the same axis has the effect that the trigger events are reactivated and the measurement results reset.

Measurement jobs started from the part program cannot be influenced from synchronized actions. If a measurement task is started from a synchronized action for an axis for which a measurement task is already active from the part program, an alarm is displayed.

If a measurement task is already active from a synchronized action, measurement can no longer be started from the part program.

Saving measurement results

A FIFO memory is set up in the \$AC_FIFO system variables to save the measurement results. See Chapter "FIFO variables (\$AC_FIFO) (Page 35)".

Examples

In the following examples, two FIFO memories are set up via machine data:

- MD28050 \$MC_MM_NUM_R_PARAM = 300
- MD28258 \$MC_MM_NUM_AC_TIMER = 1
- MD28260 \$MC_NUM_AC_FIFO = 1 (set up FIFO memory)
- MD28262 \$MC_START_AC_FIFO = 100 (FIFO memory starts from R100)
- MD28264 \$MC_LEN_AC_FIFO = 28 (22 variables + 6 management data)
- MD28266 \$MC_MODE_AC_FIFO = 0 (no summation)

Example 1

All rising edges of probe 1 are to be recorded between 0 and 100 mm for the X axis. It is assumed that no more than 22 measuring edges occur.

Program code	Comment
DEF INT NUMBER	; number of current measured values
DEF INT INDEX_R	; loop index
N10 G0 X0	; approach starting point for the measurement
;Measurement: Mode = 1 (simultaneously), FIFO memory = 1,	
; trigger event = 1 (rising edge of probe 1)	
N20 MEAC[X]=(1, 1, 1) POS[X]=100	
N30 STOPRE	; stop preprocessing
N40 MEAC[X]=(0)	; cancel measurement
N50 ANZAHL=\$AC_FIFO1[4]	; number of saved measured values
N60 ANZAHL = ANZAHL - 1	
N70 FOR INDEX_R=0 TO ANZAHL	
N80 R[INDEX_R]=\$AC_FIFO1[0]	; save measured value in R parameter
N90 ENDFOR	

Example 2

All rising and falling edges of probe 1 are to be recorded between 0 and 100 mm for the X axis. The number of measurements is not known. Therefore, the measured values must be fetched parallel to the measurement and stored in ascending order as of \$R1. The number of stored measured values is entered in \$R0.

Program code	Comment
\$AC_MARKER[1]=1	; initialize index for R-parameter index
N10 G0 X0	; approach starting point for the measurement
; If measured values are available in the FIFO memory, the oldest value is read and	
; stored in the current R parameter[\$AC_MARKER[1]].	
; The R parameter index is then incremented.	
N20 ID=1 WHENEVER \$AC_FIFO1[4] >= 1 DO \$R[\$AC_MARKER[1]] = \$AC_FIFO1[0]	
	\$AC_MARKER[1] = \$AC_MARKER[1] + 1

4.7 Actions in synchronized actions

Program code	Comment
; Continuous measurement: Mode = 1 (simultaneously), FIFO memory = 1,	
; trigger event 1 = 1 (rising edge of probe 1),	
; trigger event 2 = -1 (falling edge of probe 1)	
N30 MEAC[X]=(1, 1, 1, -1) POS[X]=100	
N40 MEAC[X]=(0)	; turn off measurement
N50 STOPRE	; stop preprocessing
N60 R0 = \$AC_MARKER[1]	; number of recorded measured values

Example 3

Rising and falling edges of probe 1 are to be recorded between 0 and 500 mm for the X axis. The number of measurements is limited to 10.

The distance-to-go of the X axis is then deleted.

Program code	Comment
N10 G0 X0	; approach starting point for the measurement
; Abort condition: Deselect continuous measurement after 10 or more measurements	
; and perform "delete distance-to-go"	
N10 WHEN \$AC_FIFO1[4] >= 10 DO MEAC[X]=(0) DELDTG(X)	
; Continuous measurement: Mode = 1 (simultaneously), FIFO memory = 1,	
; trigger event 1 = 1 (rising edge of probe 1),	
; trigger event 2 = -1 (falling edge of probe 1)	
N20 MEAC[X]=(1, 1, 1, -1) G01 X100 F500	
N30 MEAC [X]=(0)	; turn off measurement
N40 R0 = \$AC_FIFO1[4]	; number of recorded measured values

4.7.19 Travel to fixed stop (FXS, FXST, FXSW, FOCON, FOCOF, FOC)

Function**Travel to fixed stop**

The function "Travel to fixed stop" can be controlled via synchronized actions with the FXS, FXST and FXSW commands.

The activation can also be performed without traversing motion of the relevant axis. The torque is immediately limited. The fixed stop is monitored as soon as the axis is traversed.

Travel with limited torque/force

Travel with limited torque/force can be controlled via synchronized actions with the FOCON, FOCOF and FOC commands.

Syntax

FXS [<axis>]=<request>

```

FXST[<axis>]=<clamping torque>
FXSW[<axis>] = <window width>
FOCON[<axis>]
FOCOF[<axis>]
FOC[<axis>]

```

Meaning

Parameter	Meaning
FXS:	Travel to fixed stop
<Request>:	Request to the "Travel to fixed stop" function: 0 = switch off 1 = switch on
FXST:	Set clamping torque
<Clamping torque>:	Clamping torque as % of the maximum drive torque
FXSW:	Set monitoring window
<Window width>:	Width of the tolerance window around the fixed stop Unit: mm, inch or degrees
FOCON:	Switch on modal torque/force limitation
FOCOF:	Switch off modal torque/force limitation
FOC:	Non-modal torque/force limitation
<axis>:	Name of the channel axis on which the command will be applied

Remarks

Avoidance of multiple selection

The "Travel to fixed stop" function must only be switched on once per axis. In the event of an error, alarm 20092 is displayed and the corresponding alarm response takes effect.

To avoid multiple selections, it is recommended that a selection marker be used in the synchronized action.

Example:

Program code	Comment
N10 R1=0	; Initialize selection marker
...	
N20 IDS=1 WHENEVER (\$R1==0 AND \$AA_IW[AX3] > 7) DO \$R1=1 FXS[AX1]=1	

Switching on during the approach motion

"Travel to fixed stop" can also be switched on during the approach motion through a non-modal synchronized action.

Example:

Program code	Comment
N10 G0 G90 X0 Y0	; Approach initial setting
...	

4.7 Actions in synchronized actions

Program code	Comment
<pre> ; "Travel to fixed stop" is switched on for the X axis, ; as soon as the position setpoint in the WCS is > 20 mm ; Execution of the non-modal synchronized action: With N30 N20 WHEN G71 \$AA_IW[X] > 20 DO FXS[X]=1 N30 G1 F200 X100 </pre>	<pre> ; Traversing block of the X axis </pre>

Example: Travel to fixed stop completely via synchronized actions

Program code	Comment
<pre> ; IF selection request \$R1==1 AND state of the Y axis == "not to fixed stop" ; THEN: For the Y axis: ; - Switch on FXS ; - Traverse to position 150 mm ; - Reduce drive torque to 10% IDS=1 WHENEVER G71 ((\$R1==1) AND \$AA_FXS[Y]==0)) DO \$R1=0 FXS[Y]=1 FXST[Y]=10 FA[Y]=200 POS[Y]=150 ... ; IF state of the Y-Axis == "Fixed stop has been detected" ; THEN: Increase drive torque to 30% IDS=2 WHENEVER (\$AA_FXS[Y]==4) DO FXST[Y]=30 ... ; IF state of the Y axis == "Successful travel to fixed stop" ; THEN: Set drive torque in accordance with setting \$R0 IDS=3 WHENEVER (\$AA_FXS[Y]==1) DO FXST[Y]=\$R0 ... ; Deselection depending on R3 and retract. IDS=4 WHENEVER ((\$R3==1) AND \$AA_FXS[Y]==1)) DO FXS[Y]=0 FA[Y]=1000 POS[Y]=0 ... N10 R1=0 FXS[Y]=0 G0 G90 Y0 N30 RELEASE(Y) N50 ... N60 GET(Y) </pre>	<pre> ; Initialization ; Enable Y axis for traversing in synchronized actions ; Include Y axis in the path group again </pre>

4.7.20 Channel synchronization (SETM, CLEARM)

Synchronization markers can be set and deleted in the channel in which the synchronized action runs with the SETM and CLEARM commands.

Syntax

```

SETM(<No_marker 1> [, <No_marker 2> {, ... <No_marker n>} ] )
CLEARM(<No_marker 1> [, <No_marker 2> {, ... <No_marker n>} ] )

```

Meaning

A detailed description of the `SETM` and `CLEARM` commands can be found in:

More information

Programming Manual NC Programming

4.7.21 User-specific error reactions (SETAL)

Synchronous actions can be used to react user-specifically to application-specific error states. Possible reactions are:

- Axis with stop via override = 0%
- Display user-specific alarm
- Set digital output

Display alarm**Syntax**

```
SETAL(<Alarm_no>[, "Alarm text"])
```

Meaning

Parameter	Meaning
<Alarm_no>:	Alarm number from the range: 65000 - 69999

A complete description of the configuration of user alarms can be found in:

Further information

Commissioning Manual Basesoftware

Examples

```
; If the distance between axes X1 and X2 is less than 5 mm =>
; stop axis X2
ID=1 WHENEVER G71 ($AA_IM[X1]-$AA_IM[X2])<5.0 DO $AA_OVR[X2]=0

; If the distance between axes X1 and X2 is less than 5 mm =>
; display alarm 65000
ID=1 WHENEVER G71 ($AA_IM[X1]-$AA_IM[X2])<5.0 DO SETAL(65000)
```

4.7.22 Cancel the actual subprogram level (CANCELSUB)

Using `CANCELSUB`, in the channel in which the synchronized action is executed, the NC program active in the current subprogram level is canceled and in the calling program, the next higher program level is selected. There, program execution is continued normally.

Properties

- For each call, only the current subprogram level is canceled.
- After a cancellation, the next higher program level can only be canceled if a return jump is made from the canceled subprogram level.
- The main program level cannot be canceled.

Syntax

CANCELSUB

Meaning

CANCELSUB:	Cancels the current subprogram level
------------	--------------------------------------

See also

The cancellation of the current subprogram level can be realized using the channel-specific NC/PLC interface signal, also from the PLC user program, which is functionally identical:

<Chan>.basic.out.abortProgLevel (program level abort)

PLC signals

PLC → NC

Basic Program Plus	Basic Program	
<Chan>.basic.out.abortProgLevel	LBP_Chan*.A_ProgAbort	DB21,DBX6.4

4.8 Technology cycles

4.8.1 General

Definition

A technology cycle is an NC program that is called in the action part of a synchronous action. All language elements and system variables that are also used in the action part of a synchronous action can be used in a technology cycle. In addition, there are also the following language elements that may only be used within a technology cycle:

- Chapter "User-defined variables for synchronized actions (Page 63)"
- Chapter "Language elements for synchronized actions and technology cycles (Page 64)"
- Chapter "Language elements for technology cycles only (Page 70)"

- Chapter "System variables for synchronized actions (Page 25)"
- Chapter "Actions in synchronized actions (Page 70)"

End of program

The following commands are permitted as end of program: M02, M17, M30, RET

Search path

When calling a technology cycle, the same search path is used as for subprograms and cycles.

Further information

Programming Manual NC Programming

Multiple calls

If a condition is fulfilled again while the technology cycle is being executed, the technology cycle is **not** restarted.

If a technology cycle is started because of a fulfilled **WHENEVER** condition and the condition is still fulfilled after completion of the technology cycle, then the technology cycle is started again.

Behavior with non-modal synchronous actions

A non-modal synchronous action is always linked to the next main run block. If the execution time of the technology cycle is longer than the processing time of the associated main run block, the technology cycle is aborted with the block change.

Execution sequence of technology cycles

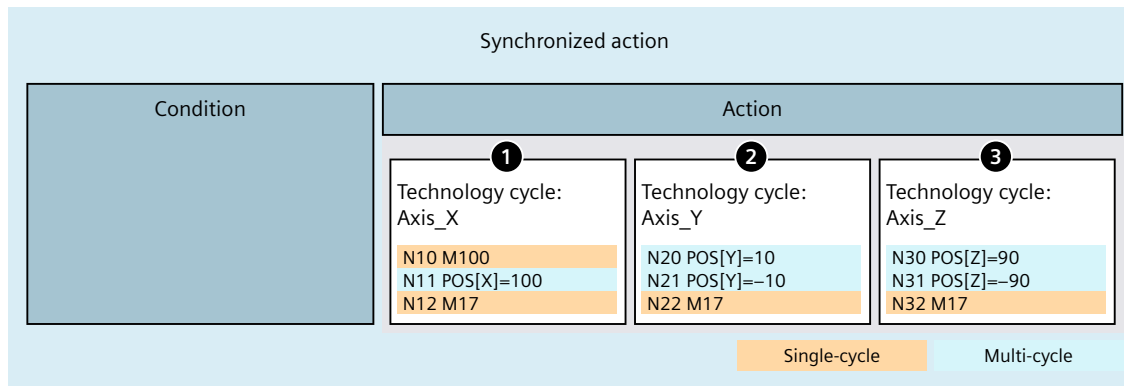
If several technology cycles are programmed in the action part of a synchronous action, they are executed in the programmed sequence from left to right.

Example

Calling three technology cycles in the action part of a synchronous action.

Program code

```
ID=1 <condition part> DO AXIS_X AXIS_Y AXIS_Z
```



Execution sequence of the technology cycle blocks: N10, N11, N12, N20, N21, N22, N30, N31, N32

Note

Constraints

- A maximum of eight technology cycles may be called in the action part of a synchronous action.
- Except for the call of further technology cycles, no other action may be programmed in the action part of a synchronous action in which a technology cycle is called.

4.8.2 Processing mode (ICYCON, ICYCOF)

Function

The **ICYCOF** and **ICYCON** commands can be used to control the processing mode of the actions within technology cycles.

Per default, the processing mode **ICYCON** is active.

Processing mode: ICYCON

A non-modal technology cycle is executed in the **ICYCON** processing mode. The execution of all actions programmed in a block is initiated in the same interpolator clock cycle. As soon as all initiated actions are completed, the next block is processed in the following interpolator clock cycle.

A distinction is made between single-cycle and multi-cycle actions. Examples are:

- Single-cycle actions: Auxiliary function output, value assignments
- Multi-cycle actions: Traversing motions of axes and spindles

Each block of a technology cycle requires at least one interpolator clock cycle.

Processing mode: ICYCOF

All actions of all blocks of a technology cycle are initiated in parallel in the **ICYCOF** processing mode.

NC program as a part program

If an NC program is executed as a part program, the ICYCOF and ICYCON commands have no effect.

Syntax**In the action part of a synchronized action**

```
ID=1 <condition part> DO [ICYCOF] <technology cycle 1> [ICYCOF | ICYCON] <technology cycle 2> ...
```

As property of an NC program

```
PROC <name> [ICYCOF | ICYCON]
```

Within an NC program

```
PROC <name>
  N10 ...
  N20 [ICYCOF | ICYCON]
  N90 ...
  N100 [ICYCOF | ICYCON]
  N110 ...
RET
```

Example

Program code	Effective processing mode	Interpolator cycle
PROC TECHNOCYC	ICYCON	
\$R1=1	ICYCON	1
POS[X]=100	ICYCON	2 ... 25
ICYCOF	ICYCOF	26
\$R1=2	ICYCOF	26
\$R2=\$R1+1	ICYCOF	26
POS[X]=110	ICYCOF	26
\$R3=3	ICYCOF	26
RET	ICYCOF	26

4.8.3 Definitions (DEF, DEFINE)

If an NC program is used as technology cycle, that contains commands for variable (DEF) and/or macro definition (DEFINE) then these have **no effect** when executing the technology cycle.

Although variables and macro definitions have no effect within a technology cycle, they must nevertheless have the correct syntax. In the event of an error, the execution of the technology cycle is aborted and an alarm displayed.

As the variables and macros are not available in the technology cycle, special measures may have to be taken in the program code. See Chapter "Context variable (\$P_TECCYCLE) (Page 124)".

4.8.4 Parameter transfer

Only the Call-by-Value parameter transfer is possible in a subprogram being applied as a technology cycle.

The application of Call-by-Reference parameters is **not** permissible and will trigger a corresponding alarm.

Further information

A detailed description of the parameter transfer and parameter definition in subprograms can be found in:

Programming Manual NC Programming

4.8.5 Context variable (\$P_TECCYCLE)

Function

If an NC program is used as part program as well as also technology cycle, then context-specific program sections can be defined using system variable \$P_TECCYCLE:

- \$P_TECCYCLE == TRUE ⇒ The NC program is currently being executed as technology cycle
- \$P_TECCYCLE == FALSE ⇒ The NC program is currently being executed as part program

Application

The (DEF) variables and (DEFINE) macro definitions have no effect in technology cycles. If an NC program is executed as a technology cycle that contains the appropriate definitions, a context-specific case distinction has to be made in the program code as the variables and macros are then no longer available.

Example

Travel parameters via user variables in the part program and R parameters in the technology cycle

Program code	Comment: Use in
PROC UP_1	
DEF REAL POS_X=100.0	Part program
DEF REAL F_X=250.0	Part program
IF \$P_TECCYCLE==TRUE	
\$R1=100.0	Technology cycle
\$R2=250.0	Technology cycle
ENDIF	
IF \$P_TECCYCLE==TRUE	
N100 POS[X]=\$R1 FA[X]=\$R2	Technology cycle
ELSE	
N200 POS[X]=POS_X FA[X]=F_X	Part program
ENDIF	

Program code	Comment: Use in
RET	

See also

Definitions (DEF, DEFINE) (Page 123)

4.9 Protected synchronized actions

Each synchronized action is clearly identified via its ID.

The following machine data can be used to define an NC global or channel-specific range of identification numbers with which a synchronized action can be protected against overwriting, deletion (`CANCEL (ID)`) and locking (`LOCK (ID)`):

- NC global:
MD11500 \$MN_PREVENT_SYNACT_LOCK (protected synchronized actions)
- Channel-specific:
MD21240 \$MN_PREVENT_SYNACT_LOCK (protected synchronized actions)

Behavior is the same in both cases.

Protected synchronized actions cannot be locked via the NC/PLC interface or are displayed as non-lockable:

- `<Chan>.synActs[1].out.lock` (lock synchronous actions)
- `<Chan>.synActs[1].in.lockable` (synchronous actions lockable)

Application

The synchronized actions defined by the machine manufacturer to react to certain machine states should not be changed after commissioning.

Note

It is recommended that the protection of synchronized actions should not be activated during the commissioning phase as otherwise a Power on reset is required at each change to the synchronized action.

Example

In a system with two channels, the synchronized actions of the following identification number areas should be protected:

Channel 1: 20 ... 30

Channel 2: 25 ... 35

4.10 Coordination via part program and synchronized action (LOCK, UNLOCK, CANCEL)

Machine data configuration

NC-global protection area:

- MD11500 \$MN_PREVENT_SYNACT_LOCK[0] = 25
- MD11500 \$MN_PREVENT_SYNACT_LOCK[1] = 35

Channel-specific protection area for channel 1:

- MD21240 \$MC_PREVENT_SYNACT_LOCK_CHAN[0] = 20
- MD21240 \$MC_PREVENT_SYNACT_LOCK_CHAN[1] = 30

Channel-specific protection area for channel 2:

- MD21241 \$MC_PREVENT_SYNACT_LOCK_CHAN[0] = -1
- MD21241 \$MC_PREVENT_SYNACT_LOCK_CHAN[1] = -1

A separate protection was not defined in channel 2 and therefore the NC-global protection area applies.

PLC signals**PLC → NC**

Basic Program Plus	Basic Program	
<Chan>.synActs[1].out.lock	LBP_Chan*.A_SynDisabl[1] - [64]	DB21,DBX300.0..3 07.7

NC → PLC

Basic Program Plus	Basic Program	
<Chan>.synActs[1].in.lockable	LBP_Chan*.E_SynDisabled[1] - [64]	DB21,DBX308.0..3 15.7

4.10 Coordination via part program and synchronized action (LOCK, UNLOCK, CANCEL)

Each modal and static synchronized action must be assigned a unique identification number during the definition:

Program code

ID=<number> condition part DO action part

IDS=<number> condition part DO action part

By specifying the identification number, synchronized actions from part programs and from synchronized actions can be coordinated via the following commands:

Keyword	Meaning	TP ¹⁾	SA ²⁾
LOCK (<number>):	Lock synchronized action An active positioning action is interrupted.	-	x
UNLOCK (<number>):	Continue interrupted synchronized action An interrupted positioning operation is continued.	-	x
CANCEL (<number>):	Delete synchronized action An active positioning action is terminated.	x	-
¹⁾ Can be programmed in the part program			
²⁾ Can be programmed in a synchronized action / technology cycle			

4.11 Coordination via PLC

With regard to their execution by the NC, synchronized actions that are not protected can be locked. Either all synchronized actions in the channel can be locked together or individually in the ID/IDS 1 - 64 area.

All, channel-specific

Lock all synchronized actions in the channel:

`<Chan>.basic.out.lockAllSynActs = 1` (lock all synchronous actions)

Individually, channel-specific

Synchronized actions that can be locked

The synchronized actions ID/IDS that can be inhibited are displayed using:

`<Chan>.synActs[1].in.lockable == 1` (synchronous actions ID/IDS lockable)

Lock synchronized actions

For each synchronized action ID/IDS, which is to be inhibited in the channel, the PLC user program must set the associated inhibit signal:

`<Chan>.synActs[1].out.lock = 1` (lock synchronous action ID/IDS)

PLC signals

PLC → NC

Basic Program Plus	Basic Program	
<code><Chan>.basic.out.lockAllSynActs</code>	<code>LBP_Chan*.A_SynOff</code>	<code>DB21,DBX1.2</code>
<code><Chan>.synActs[1].out.lock</code>	<code>LBP_Chan*.A_SynDisabl[1] - [64]</code>	<code>DB21,DBX300.0..307.7</code>

NC → PLC

Basic Program Plus	Basic Program	
<Chan>.synActs[1].in.lockable	LBP_Chan*.E_SynDisabled[1] - [64]	DB21,DBX308..315 .7

4.12 Configuration

Number of synchronized action elements

The number of synchronized action elements that can be provided per channel is set via the machine data:

MD28250 \$MC_MM_NUM_SYNC_ELEMENTS (number of elements for expressions in synchronized actions)

At least four synchronized action elements are required per synchronized action. Further synchronized action elements are required for:

Operation	Number of required elements
Operator in the condition	1
Action	≥ 1
Assignment	2
Further operands in complex expressions	1

The number of programmable synchronized actions therefore depends on the number of available synchronized action elements and the complexity of the synchronized actions.

Memory utilization

The status display for synchronized actions can be used to track the memory utilization of the synchronized action memory (see chapter "Diagnostics (Page 134)").

The number of free synchronized action elements can also be read via the system variable \$AC_SYNA_MEM.

If more synchronized action elements are required during operation than are available, alarm "14751 Resources for motion synchronous actions not sufficient" is displayed.

Number of FCTDEF elements

The number of FCTDEF elements per channel is set via the machine data:

MD28252 \$MC_MM_NUM_FCTDEF_ELEMENTS (number of FCTDEF elements)

Synchronized actions and interpolator clock cycle

If there are a large number of simultaneously active synchronized actions, the interpolator clock cycle may have to be increased:

MD10070 \$MN_IPO_SYSCLOCK_TIME_RATIO

Table 4-1 Time required by individual operations

Synchronized action commands	Time required ¹⁾	
	Total	Text in bold print
Basic load for a synchronized action if condition is not fulfilled: WHENEVER FALSE DO \$AC_MARKER[0]=0	10 µs	10 µs
Read variable: WHENEVER \$AA_IM[Y]>10 DO \$AC_MARKER[0]=1	11 µs	1 µs
Write variable: DO \$R2=1	11-12 µs	1-2 µs
Read/write setting data: DO \$SN_SW_CAM_MINUS_POS_TAB_1[0]=20	24 µs	14 µs
Basic arithmetic operations, e.g. multiplication: DO \$R2= \$R2*2	22 µs	12 µs
Trigonometric functions (e.g. cos): DO \$R2= COS(\$R2)	23 µs	13 µs
Start positioning axis: WHEN TRUE DO POS[z]=10	83 µs	73 µs
1) Measured with SINUMERIK 840D sl with NCU 7x0.3 PN		

4.13 Control behavior in specific operating states

4.13.1 Power On

No synchronized actions are active during ramp-up of the NC (Power On).

Synchronized actions that are to be active immediately after the ramp-up of the NC (Power On), must be event-driven as static synchronized actions within an ASUB or activated via the PLC user program.

Further information

Detailed information on the activation of synchronized actions after ramp-up of the NC (Power On) can be found in:

PLC user program

PLC Function Manual

Event-driven

Basic Functions Function Manual

4.13 Control behavior in specific operating states

4.13.2 NC reset

State after NC reset:

From:	Modal and non-modal synchronized action (ID)	Static synchronized action (IDS)
Synchronized action	Aborted or inactive	Active
Traversing motion	The traversing motions are aborted	
Speed-controlled spindle	MD35040 \$MA_SPIND_ACTIVE_AFTER_RESET = <value> TRUE ⇒ The spindle remains active FALSE ⇒ The spindle is stopped	
Master value coupling	MD20110 \$MC_RESET_MODE_MASK, bit 13 = <value> 1 ⇒ The coupling remains active 0 ⇒ The coupling is released	
Measuring	Aborted	

4.13.3 NC stop

Non-modal and modal synchronized actions (ID)

Traversing motions from non-modal and modal synchronized actions are stopped by NC stop.

While the channel is in the "interrupted" state (`<Chan>.basic.in.stateInterrupted == 1`), a block-by-block or modal synchronous action remains active. If the condition is fulfilled during this time, all actions are executed with the exception of traversing motions.

Stopped traversing motions are continued with NC start.

Static synchronized actions (IDS)

Whether traversing motions from static synchronized actions are stopped by NC stop depends on which is controlling the command axis:

- **NC** controls axis (`<Chan>.basic.in.stateInterrupted == 0`):
Traversing motions from static synchronized actions are stopped by NC stop.
- **PLC** controls axis (`<Chan>.basic.in.stateInterrupted == 1`):
Traversing motions from static synchronized actions are **not** stopped by NC stop.

PLC signals

PLC → NC

Basic Program Plus	Basic Program	
<code><Axis>.plcCtrl.out.requestPlcCtrl</code>	<code>LBP_Axis*.A_PLCAxis</code>	<code>DB31,DBX28.7</code>

NC → PLC

Basic Program Plus	Basic Program	
<code><Chan>.basic.in.stateInterrupted</code>	<code>LBP_Chan*.E_ChanInterrupt</code>	<code>DB21,DBX35.6</code>

4.13.4 Operating mode change

Status after operating mode change:

From:	Modal and non-modal synchronized action (ID)	Static synchronized action (IDS)
Synchronized action	Aborted or inactive ¹⁾	Active
Traversing motion	Aborted ²⁾	Active
Speed-controlled spindle	Active	Active
Master value coupling	MD20110 \$MC_RESET_MODE_MASK, bit 13 = <value> 1 ⇒ The coupling remains active 0 ⇒ The coupling is released	Active
Measuring	Aborted	Active
1) After the change back to AUTOMATIC mode, whether the synchronized actions become active again depends on the commands in the synchronized action (see example). 2) End of program M30 is delayed until the axis is at standstill.		

Example

Case 1:

```
ID=1 Do POS[X]=100 FA[X]=100
```

```
G4 F999
```

ID=1 Do means: The synchronized action is started in every interpolation cycle of the following block, even if it is already waiting at pos. "100".

Case 2:

Changing the synchronized action of Case 1 to:

```
ID=1 When true do POS[X]=100 FA[X]=100
```

```
G4 F999
```

ID=1 When true do means: Synact is started "once" when the block is loaded. The axis moves with this command.

Performing an operator action

After an NC stop and override "0", in both cases the axis stops and is not in the end position.

In case 2, the synchronized action comes to an end after the first interpolation cycle. In this case, the axis is no longer started on NC start.

The axis is not in the end position.

4.13 Control behavior in specific operating states

4.13.5 End of program

State after end of program:

From:	Modal and non-modal synchronized action (ID)	Static synchronized action (IDS)
Synchronized action	Aborted or inactive	Active
Traversing motion	Aborted 1)	Active
Speed-controlled spindle	MD35040 \$MA_SPIND_ACTIVE_AFTER_RESET = <value> TRUE ⇒ The spindle remains active FALSE ⇒ The spindle is stopped	Active
Master value coupling	MD20110 \$MC_RESET_MODE_MASK, bit 13 = <value> 1 ⇒ The coupling remains active 0 ⇒ The coupling is released	Active
Measuring	Aborted	Active
1) End of program M30 is delayed until the axis is at standstill.		

4.13.6 Block search

Synchronized actions are collected during the block search but not activated. I.e. the conditions are not evaluated, the actions are not executed.

The synchronized actions only become active with NC start. I.e. the conditions are evaluated and the actions executed if necessary.

Static synchronized actions (IDS)

Static synchronized actions that are already active remain effective during the block search. However, as these are collected additionally, this may have the result that the number of synchronized action elements required during the block search exceeds the number of elements available for the channel according to the parameter setting in MD28250 \$MC_MM_NUM_SYNC_ELEMENTS. This would lead to Alarm 14751 "Insufficient number of resources motion-synchronous actions " (see "Configuration (Page 128)").

To prevent a possible memory overload during the block search, we recommend that you proceed as follows:

1. Activate automatic ASUB start after block search:
MD11450 \$MN_SEARCH_RUN_MODE Bit1 = 1
⇒ Following NC Start and when the last action block is loaded, the block search. ASUB stated in MD11620 \$MN_PROG_EVENT_NAME is activated.
2. Activate static synchronized actions in the block search ASUB within the query IF (\$P_PROG_EVENT==5) (ASUB has been implicitly activated after output of the last action block following block search run) (see example).
3. In the part program, bracket static synchronized actions with IF (\$P_SEARCH==0) (block search not active), to prevent this section from being performed during block search.

Example:

Subprogram _N_PROG_EVENT_SPF (section)

```
DEF INT II, SIDS
```

```

IF ($P_PROG_EVENT==5)

FOR II=SIDS TO SIDS+8
CANCEL(II)
ENDFOR
STOPRE
IDS=SIDS DO G710 $AC_MARKER[SACM+2]=(( $AC_MARKER[SACM+2]+1) MOD 2)*(( $A_DBW[72]
B_AND 16)/16)
SIDS=SIDS+1
IDS=SIDS WHENEVER (( $AC_PARAM[SACP+6])>=ZPOS_CO[1])
DO $AC_PARAM[SACP+4]=SIN((( $AC_PARAM[SACP+6]-ZPOS_CO[0]) / (ZPD_CO))*90)

... ; Further synchronized actions

ENDIF

```

4.13.7 Program interruption by ASUB

Non-modal and modal synchronized actions (ID)

Active modal synchronized actions also remain active during the ASUB.

Traversing motions started from non-modal and modal synchronized actions are interrupted. If at the end of the ASUB, positioning is at the interruption point of the part program (REPOS), then the interrupted traversing motions are continued.

Static synchronized actions (IDS)

Static synchronized actions also remain active during the ASUB.

Traversing motions started from static synchronized actions are not interrupted by the ASUB.

Synchronized actions of the ASUB

If the ASUB is `not` continued with REPOS, the modal and static synchronized actions from the ASUB remain effective in the part program.

4.13.8 REPOS

In the remainder of the block, the synchronized actions are treated in the same way as in an interruption block.

Modifications to modal synchronized actions in the asynchronous subprogram are not effective in the interrupted program.

Polynomial coefficients programmed with `FCTDEF` are not affected by ASUB and REPOS.

The coefficients from the calling program are applied in the asynchronous subprogram. The coefficients from the asynchronous subprogram continue to be applied in the calling program.

If positioning motions started from synchronized actions are interrupted by the operating mode change or start of the interrupt routine, then they are continued with `REPOS`.

4.13.9 Response to alarms

- If an action of a synchronized action triggers an alarm, this action will be aborted. Other actions of the synchronized action are processed.
- If a modal synchronized action triggers an alarm, it will be inactive after the interrupt time.
- If a technology cycle generates an alarm with motion stop, it will then be aborted and no longer processed.
- If an alarm is triggered with motion stop, all axis/spindle motions, which were started by synchronized actions, will be stopped. Actions without traversing motion are still executed.
- If an alarm is triggered with interpreter stop, it will only have an effect on synchronized actions after complete execution of the predecoded blocks.

4.14 Diagnostics

4.14.1 Displaying the status of synchronized actions

The following information is displayed for diagnosing synchronized actions in the "Synchronized Actions" window of the "Machine" operating area:

- List of programmed synchronized actions
- Validity and identification number (only for modal synchronized actions)
See Section "Validity, identification number (ID, IDS) (Page 20)"
- Status of synchronized actions

Status	Meaning
Waiting	The condition is being checked in the interpolation cycle.
Active	The action part of the synchronized action is being executed. If the action consists of a technology cycle, the cycle displays the current block number.
Disabled	The synchronized action is locked. See Section: • Coordination via part program and synchronized action (LOCK, UNLOCK, CANCEL) (Page 126) • Coordination via PLC (Page 127)

Note

Non-modal synchronized actions can only be identified by their status display. They are only displayed during execution.

Examples

5.1 Examples of conditions in synchronized actions

Condition	Programming
Path distance-to-go ≤ 10 mm (WCS)	... WHEN \$AC_DTEW ≤ 10 DO ...
Distance-to-go of the X axis ≤ 10 mm (WCS)	... WHEN \$AA_DTEW[X] ≤ 10 DO ...
Path distance to start of block ≥ 20 mm (BCS)	...WHEN \$AC_PLTBB ≥ 20 DO ...
Actual value of the Y axis (MCS) $> 10 * \sin(R10)$	... WHEN \$AA_IM[y] $> 10 * \sin(R10)$ DO...
Input 1 changes from 0 to 1	... EVERY \$A_IN[1]==1 DO ...
Input 1 == 1	... WHENEVER \$A_IN[1]==0 DO ...

5.2 Reading and writing of SD/MD from synchronized actions

Infeed and oscillation for grinding operations

Setting data, whose values remain unchanged during machining, are addressed by name as in the part program.

Example: Oscillation from synchronized actions

Program code
<pre> N610 ID=1 WHENEVER \$AA_IM[Z] > \$SA_OSCILL_REVERSE_POS1[Z] DO \$AC_MARKER[1]=0 ... ; ALWAYS WHEN current position of the oscillating axis in the MCS < start of reversal area 2, ; THEN override of the infeed axis = 0% N620 ID=2 WHENEVER \$AA_IM[Z] < \$SA_OSCILL_REVERSE_POS2[Z] - 6 DO \$AA_OVR[X]=0 \$AC_MARKER[0]=0 ... ; ALWAYS WHEN current position of the oscillating axis in the MCS == reversal position 1, ; THEN override of the oscillation axis = 0%, override of the infeed axis = 100% ; This cancels the previous synchronized action! N630 ID=3 WHENEVER \$AA_IM[Z] == \$SA_OSCILL_REVERSE_POS1[Z] DO \$AA_OVR[Z]=0 \$AA_OVR[X]=100 ... ; ALWAYS WHEN distance-to-go of the partial infeed == 0, ; THEN override of the oscillation axis = 100% ; This cancels the previous synchronized action! N640 ID=4 WHENEVER \$AA_DTEPW[X]==0 DO \$AA_OVR[Z]=100 \$AC_MARKER[0]=1 \$AC_MARKER[1]=1 N650 ID=5 WHENEVER \$AC_MARKER[0]==1 DO \$AA_OVR[X]=0 N660 ID=6 WHENEVER \$AC_MARKER[1]==1 DO \$AA_OVR[X]=0 </pre>

5.2 Reading and writing of SD/MD from synchronized actions

Program code

```

...
; WHEN current position of the oscillating axis in the WCS == reversal position 1,
; THEN override of the oscillation axis = 100%, override of the infeed axis = 0%
; This cancels the second synchronized action once!
N670 ID=7 WHEN $AA_IM[Z] == $$SA_OSCILL_REVERSE_POS1[Z] DO $AA_OVR[Z]=100 $AA_OVR[X]=0
...
; Setting data whose value changes during machining (e.g. by means of
; operator input or synchronized action) must be programmed with $$$...:
; Example: Oscillation from synchronized actions with change of the oscillation
; position via the user interface
N610 ID=1 WHENEVER $AA_IM[Z] > $$SA_OSCILL_REVERSE_POS1[Z] DO $AC_MARKER[1]=0
...
; ALWAYS WHEN current position of the oscillating axis in the MCS < start of reversal area 2,
; THEN override of the infeed axis = 0%
N620 ID=2 WHENEVER $AA_IM[Z] < $$SA_OSCILL_REVERSE_POS2[Z]-6 DO
    $AA_OVR[X]=0 $AC_MARKER[0]=0
...
; ALWAYS WHEN current position of the oscillating axis in the MCS == reversal position 1,
; THEN override of the oscillation axis = 0%, override of the infeed axis = 100%
; This cancels the previous synchronized action!
N630 ID=3 WHENEVER $AA_IM[Z]==$$SA_OSCILL_REVERSE_POS1[Z] DO
    $AA_OVR[Z]=0 $AA_OVR[X]=100
...
; ALWAYS WHEN distance-to-go of the partial infeed == 0,
; THEN override of the oscillation axis = 100%
; This cancels the previous synchronized action!
N640 ID=4 WHENEVER $AA_DTEPW[X]==0 DO $AA_OVR[Z]=100 $AC_MARKER[0]=1 $AC_MARKER[1]=1
N650 ID=5 WHENEVER $AC_MARKER[0]==1 DO $AA_OVR[X]=0
N660 ID=6 WHENEVER $AC_MARKER[1]==1 DO $AA_OVR[X]=0
...
; WHEN current position of the oscillating axis in the WCS == reversal position 1,
; THEN override of the oscillation axis = 100%, override of the infeed axis = 0%
; This cancels the second synchronized action once!
N670 ID=7 WHEN $AA_IM[Z]==$$SA_OSCILL_REVERSE_POS1[Z]
DO $AA_OVR[Z]=100 $AA_OVR[X]=0

```


5.3 Examples of adaptive control

General procedure

The following examples use the polynomial evaluation function `SYNFCT()`.

1. Representation of relationship between input value and output value (main run variables in each case)
2. Definition of this relationship as polynomial with limitations
3. With position offset: Setting the MD and SD
 - MD36750 `$MA_AA_OFF_MODE` (Effect of value assignment for axial override in case of synchronized actions)
 - SD43350 `$SA_AA_OFF_LIMIT` (optional) (Upper limit of the offset value `$AA_OFF` in case of clearance control)
4. Activation of the control in a synchronized action

5.3.1 Clearance control with variable upper limit

Example of polynomial with dynamic upper limit

For the purpose of clearance control, the upper limit of the output (`$AA_OFF`, override value in axis V) is varied as a function of the spindle override (analog input 1). The upper limit for polynomial 1 is varied dynamically as a function of analog input 2.

Polynomial 1 is defined directly via system variables:

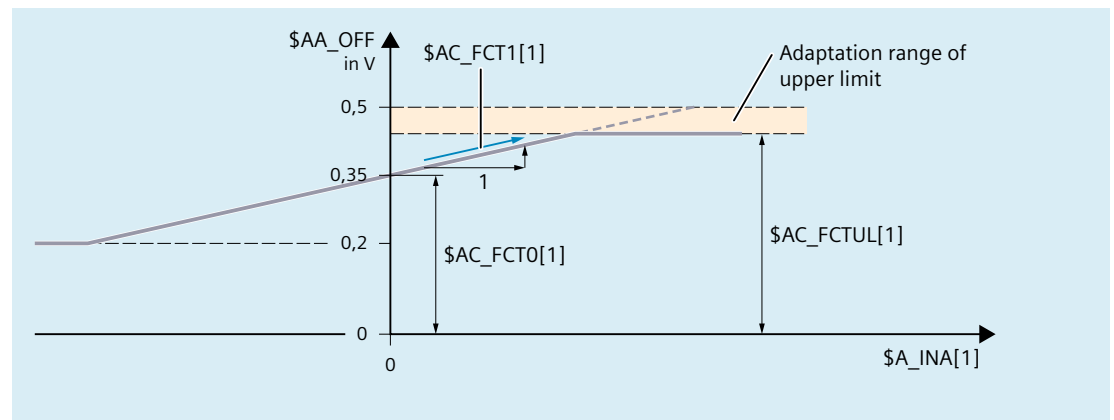


Figure 5-1 Clearance control with variable upper limit

```

$AC_FCTL[1]=0.2           ; Lower limit
$AC_FCTUL[1]=0.5         ; Request Value of upper limit
$AC_FCT0[1]=0.35        ; Zero passage a0
$AC_FCT1[1]=1.5 EX-5    ; Pitch a1
STOPRE                  ; see following note

```

5.3 Examples of adaptive control

```

...
STOPRE                                     ; see following note
ID=1 DO $AC_FCTUL[1]=$A_INA[2]*0.1+0.35    ; Adjust upper limit dynamically via
                                           ; analog input 2,
                                           ; no condition
                                           ;
ID=2 DO SYNFACT(1, $AA_OFF[V], $A_INA[1])  ; Clearance control by override of no
                                           ; condition
                                           ;
...

```

Note

When system variables are used in the part program, `STOPRE` must be programmed to ensure block-synchronous writing. The following is an equivalent notation for polynomial definition:

```
FCTDEF(1,0.2, 0.5, 0.35, 1.5EX-5).
```

5.3.2 Feedrate control

Example of adaptive control with an analog input voltage

A process quantity (measured via `$A_INA[1]`) is to be controlled at 2 V using an additive control factor implemented by a path (or axial) feedrate override. Feedrate override is to be performed within the range of +100 [mm/min].

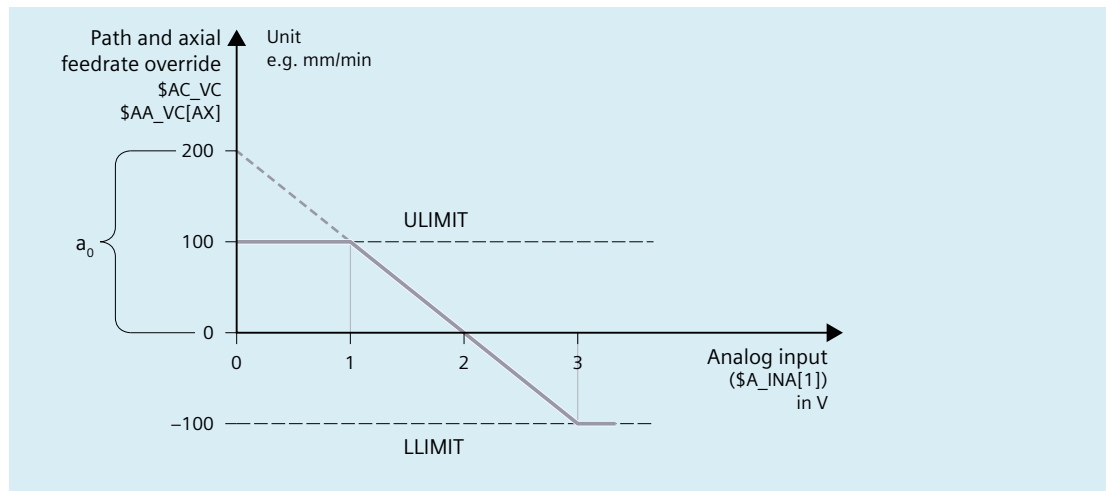


Figure 5-2 Diagram illustrating adaptive control

Determining the coefficients:

$$y = f(x) = a_0 + a_1x + a_2x^2 + a_3x^3$$

$a_1 = -100\text{mm} / (1\text{min} * 1\text{V})$

$a_1 = -100\%$ control constant, gradient

$a_0 = -(-100) * 2 = 200$

$a_2 = 0$ (not a square component)

$a_3 = 0$ (not a cubic component)

upper limit = 100

Lower limit = -100

```
FCTDEF (      Polynomial
           LLIMIT,
           ULIMIT,
           a0,           ; y for x = 0
           a1,           ; gradient
           a2,           ; square component
           a3 )          ; cubic component
```

With the values determined above, the polynomial is defined as follows:

```
FCTDEF(1, -100, 100, 200, -100, 0, 0)
```

The following synchronized actions can be used to activate the adaptive control function:

for the axis feedrate:

```
ID = 1 DO SYNFACT (1, $AA_VC[X], $A_INA[1])
```

or for the path feedrate:

```
ID = 2 DO SYNFACT(1, $AC_VC, $A_INA[1])
```

5.3.3 Control velocity as a function of normalized path

Multiplicative adaptation

The normalized path is applied as an input quantity: $\$AC_PATHN$.

0: At block start

1: at block end

Variation quantity $\$AC_OVR$ must be controlled as a function of $\$AC_PATHN$ according to a 3rd order polynomial. The override must be reduced from 100 to 1% during the motion.

5.4 Monitoring a safety clearance between two axes

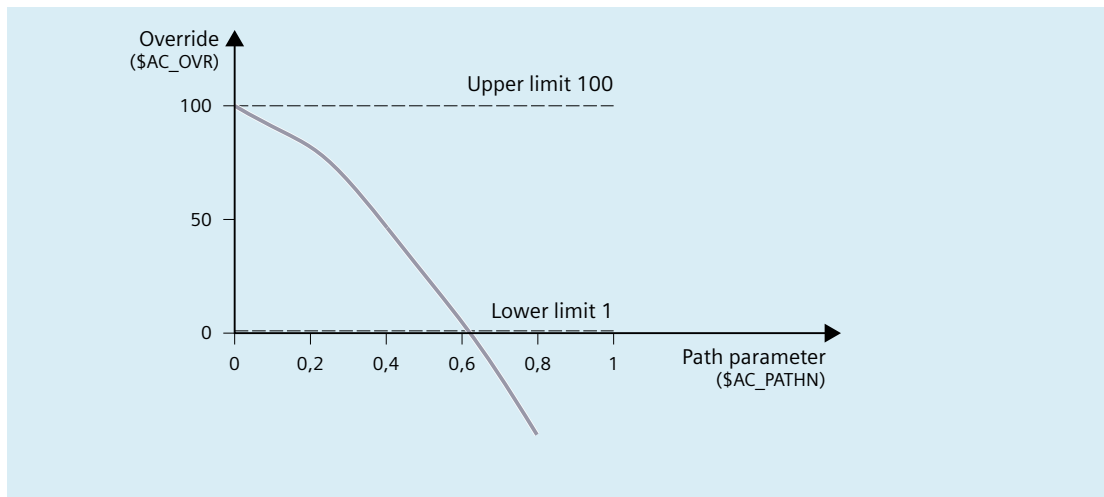


Figure 5-3 Regulate velocity continuously

Polynomial 2:

Lower limit: 1

Hi limit: 100

a_0 : 100

a_1 : -100

a_2 : -100

a_3 : not used

With these values, the polynomial definition is as follows:

```
FCTDEF(2, 1, 100, 100, -100, -100)
```

; Activation of the variable override as a function of the path:

```
ID= 1 DO SYNFACT (2, $AC_OVR, $AC_PATHN)
```

```
G01 X100 Y100 F1000
```

5.4 Monitoring a safety clearance between two axes

Task

The axes X1 and X2 operate two independently controlled transport devices used to load and unload workpieces.

To prevent the axes from colliding, a safety clearance must be maintained between them.

If the safety clearance is violated, then axis X2 is decelerated. This interlock is applied until axis X1 leaves the safety clearance area again.

If axis X1 continues to move towards axis X2, thereby crossing a closer safety barrier, then it is traversed into a safe position.

NC language	Comment
ID=1 WHENEVER \$AA_IM[X2] - \$AA_IM[X1] < 30 DO \$AA_OVR[X2]=0	; Safety barrier
ID=2 EVERY \$AA_IM[X2] - \$AA_IM[X1] < 15 DO POS[X1]=0	; Safe position

5.5 Store execution times in R parameters

Task

Store the execution time for part program blocks starting at R parameter 10.

Program	Comment
	; The example is
	; as follows without symbolic programming:
IDS=1 EVERY \$AC_TIMEC==0 DO	; Advance R parameter
\$AC_MARKER[0] = \$AC_MARKER[0] + 1	; pointer on block change
IDS=2 DO \$R[10+\$AC_MARKER[0]] =	; Write current time
\$AC_TIME	; of block start in each case to R parameter
	; The example is
	; as follows with symbolic programming:
DEFINE INDEX AS \$AC_MARKER[0]	; Agreements for symbolic
	; programming
IDS=1 EVERY \$AC_TIMEC==0 DO INDEX =	; Advance R parameter
INDEX + 1	; pointer on block change
IDS=2 DO \$R[10+INDEX] = \$AC_TIME	; Write current time
	; of block start in each case to R parameter

5.6 "Centering" with continuous measurement

Introduction

The gaps between gear teeth are measured sequentially. The gap dimension is calculated from the sum of all gaps and the number of teeth. The center position required to continue machining is the position of the first measuring point plus 1/2 the average gap size. When measuring, the speed is selected so that for each interpolation clock cycle a measured value can be reliably detected.

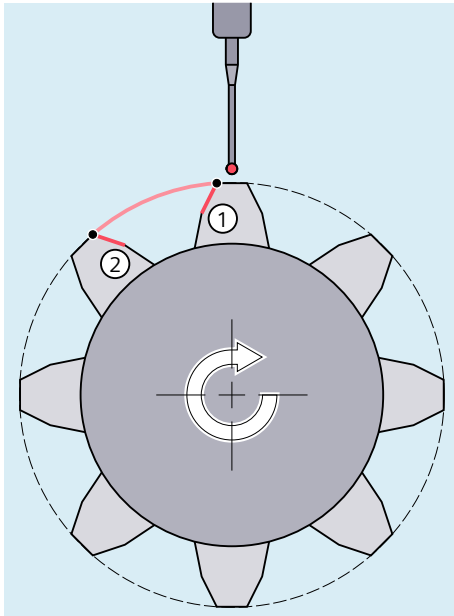


Figure 5-4 Diagrammatic representation showing how gaps between the gear teeth are measured

%_N_MEAC_MITTEN_MPF

```
; Measure using rotary axis B (BACH) where the difference is displayed
; between the measured values
;*** Define local user variables ***
N1 DEF INT ZAEHNEZahl          ; Input number of gear teeth
N5 DEF REAL HYS_POS_Flanke     ; Hysteresis positive edge probe
N6 DEF REAL HYS_NEG_Flanke     ; Hysteresis negative edge probe
;*** Define short names for synchronized action markers ***
define M_ZAEHNE as $AC_Marker[1] ; ID marker for calculation: neg/pos edge per
                                  ; tooth
define Z_MW as $AC_Marker[2]     ; Read ID counter MW FIFO
define Z_RW as $AC_Marker[3]     ; Calculate ID Counter MW tooth gaps
;*** Input values for ZAHNRADMESSEN ***
N50 ZAEHNEZahl=26               ; Enter number of gear teeth to be measured
N70 HYS_POS_Flanke = 0.160      ; Hysteresis positive edge probe
N80 HYS_NEG_Flanke = 0.140      ; Hysteresis negative edge probe
```

5.6 "Centering" with continuous measurement

```

Start:                                     *** Assign variables ***
R1=0                                       ; ID2 calculation result for gap dimension
R2=0                                       ; ID2 calculation result addition of all gaps
R3=0                                       ; Content of the first element read
R4=0                                       ; R4 corresponds to a tooth distance
R5=0                                       ; Gap position calculated, final result
R6=1                                       ; Switch-on ID 3 BACH with MOV
R7=1                                       ; Switch-on ID 5 MEAC
M_ZAEHNE=ZAEHNEZAH*2                     ; Calculate ID neg./pos. edge per tooth
Z_MW=0                                    ; Read ID counter MW FIFO till the number of
                                         ; teeth
Z_RW=2                                    ; Calculate ID counter difference of tooth gap
R13=HYS_POS_FLANKE                       ; Hysteresis in calculation register
R14=HYS_NEG_FLANKE                       ; Hysteresis in calculation register
;*** Traverse axis, measure, calculate ***
N100 MEAC[BACH]=(0)                      ; Reset measurement job
; Reset FIFO1[4] variables and ensure a defined measurement trace
N105 $AC_FIFO1[4]=0                      ; Reset FIFO1
STOPRE
;*** Read out FIFO until the number of teeth reached ***
; If FIFO1 is not empty and all teeth are still not measured, save measured value
; from FIFO variable in
; synchronization parameter and increment counter of measured values

ID=1 WHENEVER ($AC_FIFO1[4]>=1) AND (Z_MW<M_ZAEHNE)
    DO $AC_PARAM[0+Z_MW]=$AC_FIFO1[0]    Z_MW=Z_MW+1
; if 2 measured values are present, start calculation, calculate ONLY gap dimension
; and gap sum, increment calculation value counter by 2
ID=2 WHENEVER (Z_MW>=Z_RW) AND (Z_RW<M_ZAEHNE)
    DO $R1=($AC_PARAM[-1+Z_RW]-$R13)-($AC_PARAM[-2+Z_RW]-$R14)    Z_RW=Z_RW+2
    $R2=$R2+$R1
;*** Switch-on the axis BACH as endless rotating rotary axis with MOV ***
WAITP(BACH)
; Switch on
ID=3 EVERY $R6==1 DO MOV[BACH]=1 FA[BACH]=1000
; Switch off
ID=4 EVERY $R6==0 and ($AA_STAT[BACH]==1) DO MOV[BACH]=0
; Measure sequentially, store in FIFO 1, MT2 neg, MT2 pos edge;the distance between
two teeth is measured
;falling edge-...-rising edge, probe 2
N310 ID=5 WHEN $R7==1 DO MEAC[BACH]=(2, 1, -2, 2)
; Cancel measurement
N320 ID=6 WHEN (Z_MW>=M_ZAEHNE) DO MEAC[BACH]=(0)
M00
STOPRE

```

5.7 Axis couplings via synchronized actions

```

;*** Fetch FIFO values and save***
; Content of the first element read
; Reset FIFO1[4] variables and ensure a
; defined measurement trace for the next measuring job
N400 R3=$AC_PARAM[0]
N500 $AC_FIFO1[4]=0

;*** Calculate difference between the individual teeth ***
; R4 corresponds to an average tooth distance
; division "/1000" removed in later SW versions
N510 R4=R2/(ZAEHNEZahl)/1000

;*** Calculate center position ***
N520 R3=R3/1000 ; First measurement position converted to de-
; gree
N530 R3=R3 MOD 360 ; first measurement point modulo
N540 R5=(R3-R14)+(R4/2) ; Calculate gap position
M00
STOPRE
R6=0 ; Deactivate axis rotation from BACH
GOTOB START
M30

```

5.7 Axis couplings via synchronized actions

5.7.1 Coupling to leading axis

Task assignment

A cyclic curve table is defined by means of polynomial segments. Controlled by means of arithmetic variables, the movement of the master axis and the coupling process between master and slave (following) axes is activated/deactivated.

%_N_KOP_SINUS_MPF

```

N5 R1=1 ; ID 1, 2 activate/deactivate coupling: LEADON
; (CACE, BACH)
N6 R2=1 ; ID 3, 4 Move leading axis on/off: MOV BACH
N7 R5=36000 ; BACH Feedrate/min
N8 STOPRE

```


5.7 Axis couplings via synchronized actions

```

;*** Define periodic table No. 4 through polynomial segments ***
N10 CTABDEF (YGEO,XGEO,4,1)
N16 G1 F1200 XGEO=0.000 YGEO=0.000 ; Go to basic position
N17 POLY PO[XGEO]=(79,944.30.420,00.210) PO[YGEO]=(24,634.00.871,-9,670)
N18 PO[XGEO]=(116.059,0.749,-0.656) PO[YGEO]=(22.429,-5.201,0.345)
N19 PO[XGEO]=(243.941,-17.234,11.489) PO[YGEO]=(-22.429,-58.844,39.229)
N20 PO[XGEO]=(280.056,1.220,-0.656) PO[YGEO]=(-24.634,4.165,0.345)
N21 PO[XGEO]=(360.000,-4.050,0.210) PO[YGEO]=(0.000,28.139,-9.670)
N22 CTABEND ; *** End of table definition ***

; Travel axis leading axis and coupled axis in quick motion in basic position
N80 G0 BACH=0 CACH=0 ; Channel axis names
N50 LEADOF(CACH,BACH) ; existing coupling OFF

N235 ;*** Switch-on the coupling movement for the axis CACH ***
N240 WAITP(CACH) ; Synchronize axis to channel
N245 ID=1 EVERY $R1==1 DO LEA- ; Coupling via table 4
DON(CACH, BACH, 4)
N250 ID=2 EVERY $R1==0 DO LEA- ; Deactivate coupling
DOF(CACH, BACH)

N265 WAITP(BACH)
N270 ID=3 EVERY $R2==1 DO ; Rotate leading axis with feedrate endlessly
MOV[BACH]=1 FA[BACH]=R5 in R5
N275 ID=4 EVERY $R2==0 DO ; Stop leading axis
MOV[BACH]=0
N280 M00
N285 STOPRE
N290 R1=0 ; Disable coupling condition
N295 R2=0 ; Disable condition for rotating leading axis
N300 R5=180 ; New feedrate for BACH
N305 M30

```

5.7.2 Non-circular grinding via master value coupling

Task assignment

A non-circular workpiece that is rotating on axis CACH must be machined by grinding. The distance between the grinding wheel and workpiece is controlled by axis XACH and depends on the angle of rotation of the workpiece. The interrelationship between angles of rotation and assigned movements is defined in curve table 2. The workpiece must move at velocities that are determined by the workpiece contour defined in curve table 1.

Solution

CACH is designated as the leading axis in a master value coupling. It controls:

- via table 2 the compensatory movement of the axis XACH
- via table 1 the "software axis" CASW.

The axis override of axis CACH is determined by the actual values of axis CASW, thus providing the required contour-dependent velocity of axis CACH.

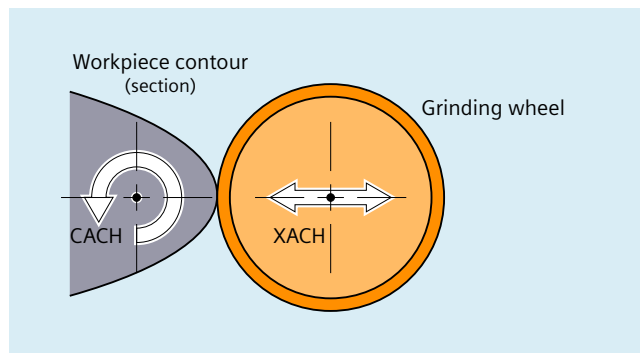


Figure 5-5 Diagrammatic representation of non-circular contour grinding

%_N_CURV_TABS_SPF		
PROC CURV_TABS		
N160 ; *** Define table 1 override ***		
N165 CTABDEF(CASW,CACH,1.1)	;	Table 1 periodic
N170 CACH=0 CASW=10		
N175 CACH=90 CASW=10		
N180 CACH=180 CASW=100		
N185 CACH=350 CASW=10		
N190 CACH=359.999 CASW=10		
N195 CTABEND		

N160 ; *** Define table 2 linear compensatory movement of XACH ***		
CTABDEF(YGEO,XGEO,2.1)	;	Table 2 periodic
N16 XGEO=0.000 YGEO=0.000		

5.7 Axis couplings via synchronized actions

N16 XGEO=0.001 YGEO=0.000	
N17 POLY PO[XGEO]=(116.000,0.024,0.012) PO[YGEO]=(4.251,0.067,-0.828)	
N18 PO[XGEO]=(244.000,0.072,-0.048) PO[YGEO]=(4.251,-2.937)	
N19 PO[XGEO]=(359.999,-0.060,0.012) PO[YGEO]=(0.000,-2.415,0.828)	
N16 XGEO=360.000 YGEO=0.000	
N20 CTABEND	
M17	

%_N_UNRUND_MPF

; Coupling group for a non-circular machining

; XACH is the infeed axis of the grinding disk

; CACH is the workpiece axis as rotary axis and master value axis

; Application: Grind non-circular contours

; Table 1 maps the override for axis CACH as function of the position of CACH

; Overlay of the XGEO axis with handwheel infeed for scratching

N100 DRFOF	; deselect handwheel overlay
N200 MSG(Select "DRF, (Handwheel 1 active) and Select INKREMENT.== Handwheel overlay AKTIV")	
N300 M00	
N500 MSG()	; Reset message
N600 R2=1	; LEADON Table 2, Activate with ID=3/4 CACH to XACH
N700 R3=1	; LEADON Table 1, Activate with ID=5/6 CACH to CASW, override
N800 R4=1	; Endless rotating axis CACH, start with ID=7/8
N900 R5=36000	; FA[CACH] Endless rotating rotary axis speed

N1100 STOPRE	
N1200	; *** Set axis and leading axis to FA ***
	; Move Axis, master axis and following axis
	; to the initial position
N1300 G0 XGEO=0 CASW=10 CACH=0	
N1400 LEADOF(XACH,CACH)	; Coupling AUS XACH compensatory movement
N1500 LEADOF(CASW,CACH)	; Coupling AUS CASW override table
N1600 CURV_TABS	; Sub-program with definition of the tables

N1700	; *** On-off switch of the LEADON compensatory movement XACH ***
N1800 WAITP(XGEO)	; Synchronize axis to channel
N1900 ID=3 EVERY \$R2==1 DO LEADON(XACH,CACH,2)	
N2000 ID=4 EVERY \$R2==0 DO LEADOF(XACH,CACH)	

5.7 Axis couplings via synchronized actions

N2100	;	*** On-off switch of the LEADON CASW override table ***
N2200 WAITP(CASW)		
N2300 ID=5 EVERY \$R3==1 DO LEA- DON(CASW,CACH,1)	;	CTAB Coupling ON leading axis CACH
N2400 ID=6 EVERY \$R3==0 DO LEA- DOF(CASW,CACH)	;	CTAB Coupling OFF leading axis CACH

N2500	;	*** Control override of the CACH from position CASW with ID 10 ***
N2700 ID=11 DO \$ \$AA_OVR[CACH]=\$AA_IM[CASW]	;	Assign "axis position" CASW to OVR CACH

N2900 WAITP(CACH)		
N3000 ID=7 EVERY \$R4==1 DO MOV[CACH]=1 FA[CACH]=R5	;	Start as endless rotating rotary axis
N3100 ID=8 EVERY \$R4==0 DO MOV[CACH]=0	;	Stop as endless rotating rotary axis

N3200 STOPRE		
N3300 R90=\$AA_COUP_ACT[CASW]	;	State of the coupling for CASW for checking
N3400 MSG("Override table CASW activated with LEADON "<<R90<<," further ENDE with NC-START")		

N3500 M00	;	*** NC HALT ***
N3600 MSG()		
N3700 STOPRE	;	Preprocessing stop
N3800 R1=0	;	Stop with ID=2 CASW axis as endless rotating rotary axis
N3900 R2=0	;	LEADOF with ID=6 FA XACH and leading axis CACH
N4000 R3=0	;	LEADOF TAB1 CASW with ID=7/8 CACH to CASW override table
N4100 R4=0	;	Stop axis as endless rotating rotary axis , ID=4 CACH
N4200 M30		

Expansion options

The example above can be expanded by the following components:

- Introduction of a Z axis to move the grinding wheel or workpiece from one non-circular operation to the next on the same shaft (cam shaft).
- Table switchovers, if the cams for inlet and outlet have different contours.
ID = ... <Condition> DO LEADOF(XACH, CACH) LEADON(XACH, CACH, <new table number>)
- Dressing of grinding wheel by means of online tool offset acc. to Subsection "Online tool offset FTOC".

5.7.3 On-the-fly parting

Task assignment

An extruded material which passes continuously through the operating area of a cutting tool must be cut into parts of equal length.

X axis: Axis in which the extruded material moves, WCS

X1 axis: Machine axis of the extruded material, MCS

Y axis: Axis in which the cutting tool "tracks" the extruded material

It is assumed that the infeed and control of the cutting tool are controlled via the PLC. The signals at the PLC interface can be evaluated to determine whether the extruded material and cutting tool are synchronized.

Actions

Activate coupling, LEADON

Deactivate coupling, LEADOF

Set actual values, PRESETON

Program code	Comment
%_N_SCHERE1_MPF	
;\$PATH=/_N_WKS_DIR/_N_DEMOFBE_WPD	
N100 R3=1500	; Length of a part to be cut off
N200 R2=100000 R13=R2/300	
N300 R4=100000	
N400 R6=30	; Start position Y axis
N500 R1=1	; Start condition for conveyor axis
N600 LEADOF(Y,X)	; Delete any existing coupling
N700 CTABDEF(Y,X,1,0)	; Table definition
N800 X=30 Y=30	; Value pairs
N900 X=R13 Y=R13	
N1000 X=2*R13 Y=30	

5.8 Technology cycles position spindle

Program code	Comment
N1100 CTABEND	; End of table definition
N1200 PRESETON(X1,0)	; Preset offset at the beginning
N1300 Y=R6 G0	; Start position Y axis
	; The axis is a linear axis
N1400 ID=1 EVERY \$AA_IW[X]>\$R3 DO PRESETON(X1,0)	; Preset offset after length R3, PRESETON must only be executed with WHEN and EVERY
	; New start after parting
N1500 WAITP(Y)	
N1800 ID=6 EVERY \$AA_IM[X]<10 DO LEADON(Y,X,1)	; For X < 10, couple Y to X via table 1
N1900 ID=10 EVERY \$AA_IM[X]>\$R3-30 DO LEADOF(Y,X)	; > 30 before traversed parting distance, deactivate coupling
N2000 WAITP(X)	
N2100 ID=7 WHEN \$R1==1 DO MOV[X]=1 FA[X]=\$R4	; Set extruded material axis continuously in motion
N2200 M30	

5.8 Technology cycles position spindle

Application

Interacting with the PLC program, the spindle which initiates a tool change should be:

- Traversed to an initial position,
- Positioned at a specific point at which the tool to be inserted is located.

See Section "Starting command axes" and Section "Control via PLC".

Coordination

The PLC and NCK are coordinated by means of the common data that are provided in SW version 4 and later (see Section "List of system variables relevant to synchronized actions")

- \$A_DBB[0]: Go to basic position 1,
- \$A_DBB[1]: Go to target position 1,
- \$A_DBW[1]: Value to be positioned +/- , PLC calculates the shortest route.

Synchronized actions

%_N_MAIN_MPF

...

IDS=1 EVERY \$A_DBB[0]==1 DO NULL_POS

; When \$A_DBB[0] set by PLC, go to basic position.

5.8 Technology cycles position spindle

<pre>IDS=2 EVERY \$A_DBB[1]==1 DO ZIEL_POS ... </pre>	<pre>; When \$A_DBB[1] set by PLC, po- sition spindle to the value stor- ed in \$A_DBW[1]. </pre>
---	---

Technology cycle NULL_POS

%_N_NULL_POS_SPF

<pre>PROC NULL_POS SPOS=0 \$A_DBB[0]=0 </pre>	<pre>; Put drive for the tool change into basic posi- tion. ; Basic position executed in NC. </pre>
---	---

Technology cycle ZIEL_POS

%_N_ZIEL_POS_SPF

<pre>PROC ZIEL_POS SPOS=IC(\$A_DBW[1]) \$A_DBB[1]=0 </pre>	<pre>; Traverse spindle to position value that has been stored in \$A_DBW[1] by the PLC, incremental dimen- sion ; Target positions executed in NC. </pre>
--	--

5.9 Synchronized actions in the TC/MC area

Introduction

The following figure shows the schematic structure of a tool-changing cycle.

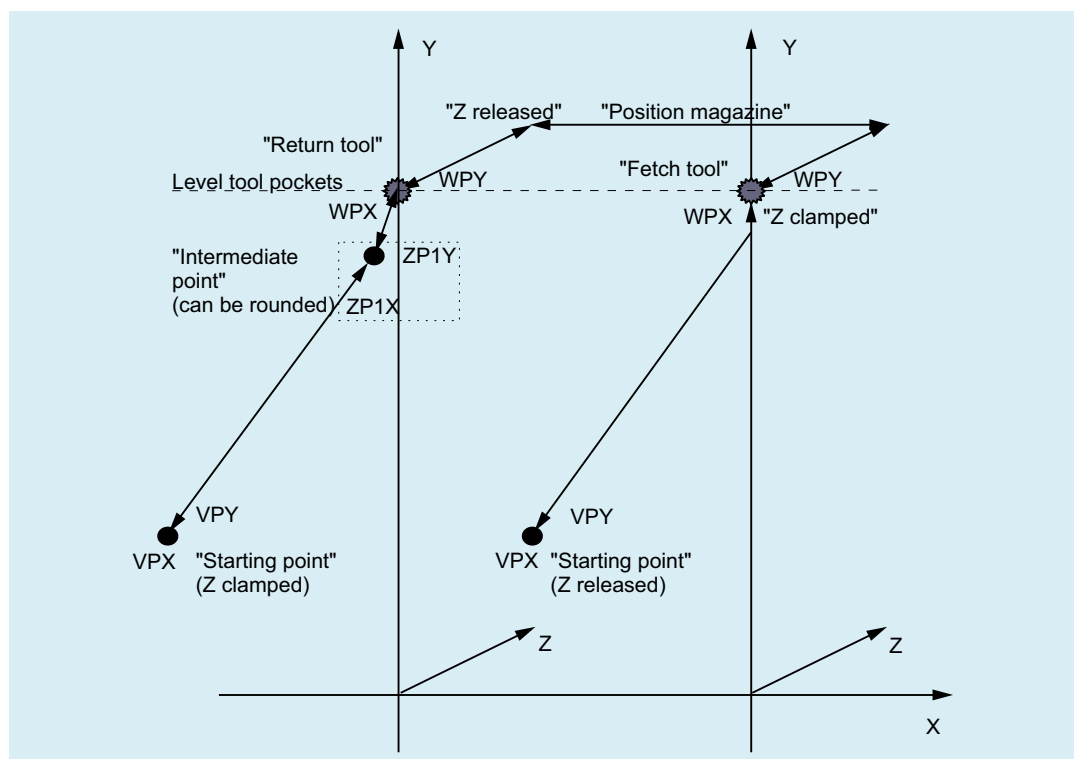


Figure 5-6 Schematic sequence for tool-changing cycle

Flow diagram

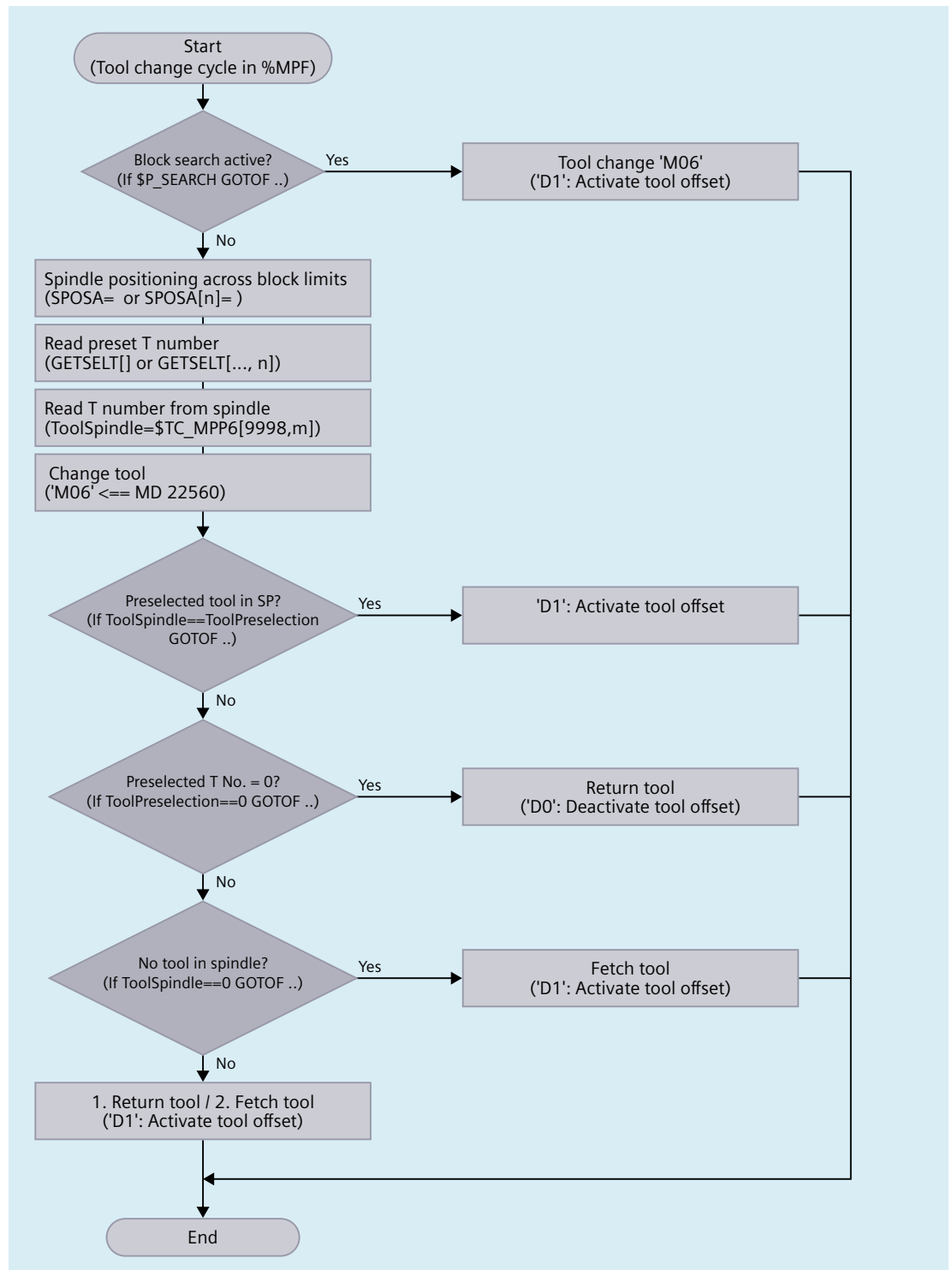


Figure 5-7 Flow diagram for tool changing cycle

5.9 Synchronized actions in the TC/MC area

NC program	Comment
%_N_WZW_SPF	
;\$PATH=/_N_SPF_DIR	
N10 DEF INT WZVorwahl,WZSpindel	;Marker on = 1 when MagAxis traversed
N15 WHEN \$AC_PATHN<10 DO \$AC_MARKER[0]=0 \$AC_MARKER[1]=0 \$AC_MARKER[2]=0	
N20 ID=3 WHENEVER \$A_IN[9]==TRUE DO \$AC_MARKER[1]=1	
N25 ID=4 WHENEVER \$A_IN[10]==TRUE DO \$AC_MARKER[2]=1	;Marker on = 1 when MagAxis traversed
N30 IF \$P_SEARCH GOTOF wzw_vorlauf	;Block preprocessing active ? ->
N35 SPOSA=0 DO	
N40 GETSELT(WZVorwahl)	;Read preselected T no.
N45 WZSpindel=\$TC_MPP6[9998,1]	;Read tool in spindle
N50 M06	
N55 IF WZSpindel==WZVorwahl GOTOF wz_in_spindel IF WZVorwahl==0 GOTOF ablegen1 IF WZSpindel==0 GOTOF holen1	
;*** Fetch and store tool***	
ablegen1holen1:	
N65 WHENEVER \$AA_VACTM[C2]<>0 DO \$AC_MARKER[1]=1	;when MagAxis travels Marker = 1
N70 G01 G40 G53 G64 G90 X=Magazin1VPX Y=Magazin1VPY Z=Magazin1ZGespannt F70000 M=QU(120) M=QU(123) M=QU(9)	
N75 WHENEVER \$AA_STAT[S1]<>4 DO \$AC_OVR=0	;Spindle in position
N80 WHENEVER \$AA_VACTM[C2]<>0 DO \$AC_MARKER[1]=1	;Query MagAxis travel
N85 WHENEVER \$AC_MARKER[1]==0 DO \$AC_OVR=0	;Override=0 when axis not traversed
N90 WHENEVER \$AA_STAT[C2]<>4 DO \$AC_OVR=0	;Override=0 when MagAxis not in position fine
N95 WHENEVER \$AA_DTEB[C2]>0 DO \$AC_OVR=0	;Override=0 when distance-to-go MagAxis > 0
N100 G53 G64 X=Magazin1ZP1X Y=Magazin1ZP1Y F60000	
N105 G53 G64 X=Magazin1WPX Y=Magazin1WPY F60000	
N110 M20	;Release tool
N115 G53 G64 Z=MR_Magazin1ZGeloest F40000	
N120 WHENEVER \$AA_VACTM[C2]<>0 DO \$AC_MARKER[2]=1;	
N125 WHENEVER \$AC_MARKER[2]==0 DO \$AC_OVR=0	
N130 WHENEVER \$AA_STAT[C2]<>4 DO \$AC_OVR=0	
N135 WHENEVER \$AA_DTEB[C2]>0 DO \$AC_OVR=0	
N140 G53 G64 Z=Magazin1ZGespannt F40000	
N145 M18	;Clamp tool
N150 WHEN \$AC_PATHN<10 DO M=QU(150) M=QU(121)	;Condition always fulfilled
N155 G53 G64 X=Magazin1VPX Y=Magazin1VPY F60000 D1 M17	

5.9 Synchronized actions in the TC/MC area

NC program	Comment
*** Store tool***	
ablegen1:	
N160 WHENEVER \$AA_VACTM[C2]<>0 DO \$AC_MARKER[1]=1	
N165 G01 G40 G53 G64 G90 X=Magazin1VPX Y=Magazin1VPY Z=Maga-	
zin1ZGespannt F70000 M=QU(120) M=QU(123) M=QU(9)	
N170 WHENEVER \$AA_STAT[S1]<>4 DO \$AC_OVR=0	
N175 WHENEVER \$AA_VACTM[C2]<>0 DO \$AC_MARKER[1]=1	
N180 WHENEVER \$AC_MARKER[1]==0 DO \$AC_OVR=0	
N185 WHENEVER \$AA_STAT[C2]<>4 DO \$AC_OVR=0	
N190 WHENEVER \$AA_DTEB[C2]>0 DO \$AC_OVR=0	
N195 G53 G64 X=Magazin1ZPlX Y=Magazin1ZPlY F60000	
N200 G53 G64 X=Magazin1WPX Y=Magazin1WPY F60000	
N205 M20	;Release tool
N210 G53 G64 Z=Magazin1ZGeloest F40000	
N215 G53 G64 X=Magazin1VPX Y=Magazin1VPY F60000 M=QU(150)	
M=QU(121) D0 M17	
*** Fetch tool***	
holen1:	
N220 WHENEVER \$AA_VACTM[C2]<>0 DO \$AC_MARKER[2]=1	
N225 G01 G40 G53 G64 G90 X=Magazin1VPX Y=Magazin1VPY Z=Maga-	
zin1ZGeloest F70000 M=QU(120) M=QU(123) M=QU(9)	
N230 G53 G64 X=Magazin1WPX Y=Magazin1WPY F60000	
N235 WHENEVER \$AA_STAT[S1]<>4 DO \$AC_OVR=0	
N240 WHENEVER \$AA_VACTM[C2]<>0 DO \$AC_MARKER[2]=1	
N245 WHENEVER \$AC_MARKER[2]==0 DO \$AC_OVR=0	
N250 WHENEVER \$AA_STAT[C2]<>4 DO \$AC_OVR=0	
N255 WHENEVER \$AA_DTEB[C2]>0 DO \$AC_OVR=0	
N260 G53 G64 Z=Magazin1ZGespannt F40000	
N265 M18	;Clamp tool
N270 G53 G64 X=Magazin1VPX Y=Magazin1VPY F60000 M=QU(150)	
M=QU(121) D1 M17	
Tool in spindle	
wz_in_spindle:	
N275 M=QU(121) D1 M17	
Block preprocessing	
wzw_feed:	
N280 STOPRE	
N285 D0	
N290 M06	
N295 D1 M17	

Appendix

A.1 List of abbreviations

A	
A	Output
AFIS	Automatic Filter Switch: Automatic filter switch
ASCII	American Standard Code for Information Interchange: American coding standard for the exchange of information
ASIC	Application Specific Integrated Circuit: User switching circuit
ASUP	Asynchronous subprogram
AUTO	Operating mode "Automatic"
AUXFU	Auxiliary Function: Auxiliary function
AWL	Statement list

B	
BAG	Mode group
BCD	Binary Coded Decimals: Decimal numbers encoded in binary code
BICO	Binector Connector
BIN	Binary Files: Binary files
BKS	Basic coordinate system
BM	Operating alarm
BO	Binector Output
BTSS	Operator panel interface

C	
CLC	Clearance control
CNC	Computerized Numerical Control: Computerunterstützte numerische Steuerung
COM	Communication
CP	Communication Processor
CPU	Central Processing Unit: Central processing unit
CST	Configured Stop: Configured stop

D	
DDS	Drive Data Set: Drive data set
DIR	Directory: Directory
DO	Drive Object
DRF	Differential Resolver Function: Differential resolver function (handwheel)

D	
DRY	Dry Run: Dry run feedrate
DWORD	Double word (currently 32 bits)

E	
E	Input
EES	Execution from External Storage
E/A	Input/Output
ESR	Extended stop and retract
ETC	ETC key ">"; Softkey bar extension in the same menu

F	
FDD	Feed Disable: Feed disable
FdStop	Feed Stop: Feed stop
FIFO	First In First Out: Memory that works without address specification and whose data is read in the same order in which they was stored
FM	Error message
FUP	Control system flowchart (PLC programming method)
FW	Firmware

G	
GEO	Geometry, e.g. geometry axis
GP	Basic program (PLC)
GUD	Global User Data: Global user data

H	
HEX	Abbreviation for hexadecimal number
HiFu	Auxiliary function
HMI	Human Machine Interface: SINUMERIK user interface
HSA	Main spindle drive
HT	Handheld Terminal
HW	Hardware

I	
IBN	Commissioning
INC	Increment: Increment
INI	Initializing Data: Initializing data
IPO	Interpolator

J	
JOG	JOGging: Setup mode

K	
KOP	Ladder diagram (PLC programming method)

L	
LED	Light Emitting Diode: Light-emitting diode
LMS	Position measuring system
LR	Position controller

M	
Main	Main program: Main program (OB1, PLC)
MCP	Machine Control Panel: Machine control panel
MD	Machine Data
MDA	Manual Data Automatic: Manual input
MDS	Motor Data Set: Motor data set
MELDW	Message word
MKS	Machine coordinate system
MM	Motor Module
MPF	Main Program File: Main program (NC)
MPI	Multi Point Interface
MSTT	Machine control panel

N	
NC	Numerical Control: Numerical control with block preparation, traversing range, etc.
NCU	Numerical Control Unit: NC hardware unit
NCK	Numerical Control Kernel
NCSD	NC Start Disable
NST	Interface signal
NV	Work offset
NX	Numerical Extension: Axis expansion module

O	
OB	Organization block in the PLC
OEM	Original Equipment Manufacturer
OP	Operation Panel: Operating equipment

P	
PCU	PC Unit: PC box (computer unit)
PG	Programming device
PI	Program instance
PLC	Programmable Logic Control: Controller
PN	PROFINET
PO	Power On
POS	Position/positioning
PPO	Parameter process data object: Cyclic data telegram for PROFIBUS DP transmission and "Variable speed drives" profile
PPU	Panel Processing Unit (central hardware for a panel-based CNC, e.g. SINUMERIK 828D)
PROFIBUS	Process Field Bus: Serial data bus
PRT	Program test
PTP	Point to Point: Point-to-point
PZD	Process data: Process data part of a PPO

R	
REF	REference point approach function
REPOS	REPOSition function
RESU	Retrace support
RID	Read In Disable
RP	R Parameter, arithmetic parameter, predefined user variable

S	
SA	Synchronized action
SBL	Single Block: Single block
SBT	Safe Brake Test
SCC	Safety Control Channel
SCL	Structured Control Language
SD	Settingdatum or setting data
SDI	Safe Direction
SERUPRO	Search-Run by Program Test: Block search via program test
SIC	Safety Info Channel
SKP	Skip: Function for skipping a part program block
SLP	Safe Limited Position
SLS	Safely Limited Speed
SMI	Sensor Module Integrated
SOS	Safe Operating Stop
SPF	Sub Program File: Subprogram (NC)
SS1	Safe Stop 1
SS2	Safe Stop 2
STO	Safe Torque Off

S	
STW	Control word
SUG	Grinding wheel peripheral speed
SW	Software

T	
TCU	Thin Client Unit
TIA	Totally Integrated Automation
TM	Terminal Module (SINAMICS)
TO	Tool Offset: Tool offset
TOA	Tool Offset Active: Identifier (file type) for tool offsets
TOFF	Online tool length offset
TRANSMIT	Transform Milling Into Turning: Coordination transformation for milling operations on a turning machine

U	
UP	Subprogram
USB	Universal Serial Bus

V	
VDI	Internal communication interface between NC and PLC

W	
WKS	Workpiece coordinate system
WPD	Work Piece Directory: Workpiece directory
WZ	Tool
WZV	Tool management

Z	
ZSW	Status word (of drive)

Index

\$

\$A_INA, 75
\$AA_AXCHANGE_STAT, 92
\$AA_AXCHANGE_TYP, 92
\$AA_JERK_COUNT, 46
\$AA_JERK_TIME, 46
\$AA_JERK_TOT, 46
\$AA_MEAAct, 114
\$AA_OFF, 49
\$AA_OFF_LIMIT, 50
\$AA_OVR, 41
\$AA_PLC_OVR, 41
\$AA_TOFF, 52
\$AA_TOFF_VAL, 53
\$AA_TOTAL_OVR, 42
\$AA_TRAVEL_COUNT, 46
\$AA_TRAVEL_COUNT_HS, 46
\$AA_TRAVEL_DIST, 46
\$AA_TRAVEL_DIST_HS, 46
\$AA_TRAVEL_TIME, 46
\$AA_TRAVEL_TIME_HS, 46
\$AC_BLOCKTYPE, 56
\$AC_BLOCKTYPEINFO, 56
\$AC_DTEB, 80
\$AC_FCT0, 47
\$AC_FCT1, 47
\$AC_FCT2, 48
\$AC_FCT3, 48
\$AC_FCTLL, 47
\$AC_FCTUL, 47
\$AC_FIFO, 35
\$AC_MARKER, 30
\$AC_OVR, 40
\$AC_PARAM, 31
\$AC_PLC_OVR, 41
\$AC_SPLITBLOCK, 57
\$AC_SYNA_STATE, 62
\$AC_SYNC_ACT_LOAD, 42
\$AC_SYNC_AVERAGE_LOAD, 42
\$AC_SYNC_MAX_LOAD, 42
\$AC_TANEB, 40
\$AC_TIMER, 34
\$AC_TOTAL_OVR, 41
\$AN_IPO_ACT_LOAD, 42
\$AN_IPO_LOAD_LIMIT, 43
\$AN_IPO_LOAD_PERCENT, 42
\$AN_IPO_MAX_LOAD, 42

\$AN_IPO_MIN_LOAD, 42
\$AN_SERVO_ACT_LOAD, 42
\$AN_SERVO_MAX_LOAD, 42
\$AN_SERVO_MIN_LOAD, 42
\$AN_SYNC_ACT_LOAD, 42
\$AN_SYNC_MAX_LOAD, 42
\$AN_SYNC_TO_IPO, 42
\$P_TECCYCLE, 124
\$SA_WORKAREA_MINUS_ENABLE, 45
\$SA_WORKAREA_PLUS_ENABLE, 45
\$SN_SW_CAM_MINUS_POS_TAB_1, 45
\$SN_SW_CAM_MINUS_POS_TAB_2, 45
\$SN_SW_CAM_MINUS_TIME_TAB_1, 45
\$SN_SW_CAM_MINUS_TIME_TAB_2, 45
\$SN_SW_CAM_PLUS_POS_TAB_1, 45
\$SN_SW_CAM_PLUS_POS_TAB_2, 45
\$SN_SW_CAM_PLUS_TIME_TAB_1, 45
\$SN_SW_CAM_PLUS_TIME_TAB_2, 45

A

Adaptive control, 137
 Example, 138
AXTOCHAN (SA), 90

B

Boolean operations, 23

C

CANCEL, 127
Cancels the current subprogram level
 CANCELSUB, 119
CANCELSUB, 119
CP..., 109
CTAB..., 109

D

DELDTG, 80
DO, 24

E

ELSE (SA), 24
EVERY, 21

F

FA (SA), 89
FOC, 116
FOCOF, 116
FOCON, 116
FROM, 21
FTOC, 77
FXS, 116
FXST, 116
FXSW, 116

G

G functions
 Action, 23
 Condition, 22
G70 (SA), 86
G700 (SA), 86
G71 (SA), 86
G710 (SA), 86
GET (SA), 90
GUD (SA), 63

I

ICYCOF, 122
ICYCON, 122
ID, 20
Identification number, 21
IDS, 20

L

LEAD..., 109
LOCK, 127

M

M, 95
MD10070, 129
MD10722, 92
MD11110, 70
MD11510, 43
MD18660, 63
MD18661, 63
MD18662, 63
MD18663, 63
MD18664, 63

MD18665, 63
MD20110, 54, 131, 132
MD21190, 52
MD21194, 52
MD21196, 52
MD22200, 70
MD22210, 70
MD22230, 70
MD28050, 32
MD28250, 128
MD28252, 128
MD28254, 31
MD28255, 31
MD28256, 30
MD28257, 30
MD28258, 34
MD28260, 38
MD28262, 38
MD28264, 38
MD28266, 38
MD30450, 85
MD30460, 103
MD32060, 90
MD32070, 49
MD32074, 84
MD32420, 49
MD32430, 50
MD35040, 132
MD36750, 50, 137
MEAC (SA), 113
MEAWA (SA), 113
Modal synchronized action, 20
MOV, 88

N

NC reset, 130
NC stop, 130
Non-modal synchronized action, 20

P

POS (SA), 82
POSRANGE, 87
Power On, 129
PRESETON (SA), 97
PRESETONS (SA), 103

R

RDISABLE, 79

RELEASE (SA), 90
REPOS, 133

S

S (SA), 95
SD42122, 40
SD43300, 90
SD43350, 50, 137
Sequence of execution, 21
SPOS (SA), 95
Static synchronized action, 20
STOPRE (SA), 82
STOPREOF, 80
Subprogram level is canceled
 CANCELSUB, 119
Synchronized actions
 Example Adaptive control, 137
 Example Control via dyn. override, 139
 Example Presses, coupled axes, 144
 Example: Path feedrate control, 138
Synchronous actions
 Additive adjustment via SYNFACT, 72
SYNFACT, 72
 Examples, 137

T

Technology cycle, 24
Technology cycles, 120
TRAIL..., 109

U

UNLOCK, 127

W

WHEN, 21
WHENEVER, 21

